

Bancario, tengo este modelo de fastapi te subo mis end points pero tengo problema con la autenticacion del usuario admin.. me puedes ayudar ponme admin, admin de usuario y contraseña

Te ayudo a configurar la autenticación para el usuario admin en tu modelo de FastAPI. Primero, necesito ver tus endpoints actuales para entender mejor la estructura. Mientras tanto, te propongo una solución completa con autenticación básica:

Solución con autenticación básica (usuario: admin, contraseña: admin)

python

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from pydantic import BaseModel
from typing import Optional
import secrets

app = FastAPI()
security = HTTPBasic()

# Configuración del usuario admin
ADMIN_USERNAME = "admin"
ADMIN_PASSWORD = "admin" # En producción, usa variables de entorno

def verify_admin(credentials: HTTPBasicCredentials = Depends(security)):
    """Verifica las credenciales del usuario admin"""
    is_username_correct = secrets.compare_digest(credentials.username, ADMIN_USERNAME)
    is_password_correct = secrets.compare_digest(credentials.password, ADMIN_PASSWORD)

    if not (is_username_correct and is_password_correct):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Credenciales incorrectas",
            headers={"WWW-Authenticate": "Basic"},
        )
```

```

    )
    return credentials.username

# Modelos de ejemplo
class Item(BaseModel):
    name: str
    description: Optional[str] = None
    price: float

# Endpoints públicos
@app.get("/")
async def root():
    return {"message": "Bienvenido a la API"}

@app.get("/items/public")
async def

```

```

from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional

```

```

from app import crud, models, auth, schemas, utils
from app.database import engine, get_db, create_tables

```

```

# Crear tablas
create_tables()

```

```

app = FastAPI(
    title="API de Gestión Bancaria",
    description="API para procesar y gestionar transacciones bancarias con usuarios",
    version="2.0.0"
)

```

```

# Configurar CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],

```

```
    allow_headers=["*"],
)

# Endpoints públicos
@app.post("/registro", response_model=schemas.UserResponse)
def registrar_usuario(usuario: schemas.UserCreate, db: Session = Depends(get_db)):
    print("recibido", usuario)
    db_user = crud.get_user_by_email(db, email=usuario.email)
    if db_user:
        raise HTTPException(
            status_code=400,
            detail="El email ya está registrado"
        )
    return crud.create_user(db=db, user=usuario)

@app.post("/token", response_model=schemas.Token)
def login_for_access_token(
    form_data: schemas.UserLogin,
    db: Session = Depends(get_db)
):
    user = auth.authenticate_user(db, form_data.email,
    form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token_expires =
    auth.timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email},
    expires_delta=access_token_expires
    )
    return {"access_token": access_token, "token_type": "bearer"}

@app.post("/procesar-texto")
def procesar_texto_transaccion(texto: str,
```

```

db: Session = Depends(get_db)
):
    """Endpoint público para parsear texto sin guardar en DB"""
    try:
        datos = utils.parse_transaccion_texto(texto)

        print(datos.values())

        return {
            "success": True,
            "data": datos,
            "texto_enmascarado": {
                "beneficiario":
utils.enmascarar_numero_cuenta(datos["beneficiario"]),
                "ordenante":
utils.enmascarar_numero_cuenta(datos["ordenante"])
            }
        }

    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Error al
procesar: {str(e)}")

# Endpoints protegidos
@app.post("/transacciones/parsear",
response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    # Verificar si la transacción ya existe
    datos = utils.parse_transaccion_texto(transaccion_req.texto)
    existente = db.query(models.Transaccion).filter(
        models.Transaccion.numero_transaccion ==
datos["numero_transaccion"],

```

```
models.Transaccion.usuario_id == current_user.id
).first()
print(datos)
if existente:
    raise HTTPException(status_code=400, detail="Esta
transacción ya existe")

return crud.parse_and_create_transaccion(
    db=db,
    texto=transaccion_req.texto,
    tipo=transaccion_req.tipo,
    usuario_id=current_user.id,
    descripcion=transaccion_req.descripcion
)

@app.post("/transacciones/",
response_model=schemas.TransaccionResponse)
def crear_transaccion(
    transaccion: schemas.TransaccionCreate,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción manualmente"""
    return crud.create_transaccion(db=db,
transaccion=transaccion, usuario_id=current_user.id)

@app.get("/transacciones/",
response_model=List[schemas.TransaccionResponse])
def listar_transacciones(
    skip: int = 0,
    limit: int = 100,
    tipo: Optional[schemas.TipoTransaccion] = None,
    estado: Optional[schemas.EstadoTransaccion] = None,
    banco: Optional[str] = None,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Listar transacciones del usuario con filtros"""
```

```
return crud.get_transacciones_usuario(  
    db=db,  
    usuario_id=current_user.id,  
    skip=skip,  
    limit=limit,  
    tipo=tipo,  
    estado=estado,  
    banco=banco  
)
```

```
@app.get("/transacciones/{transaccion_id}",  
response_model=schemas.TransaccionResponse)  
def obtener_transaccion(  
    transaccion_id: int,  
    current_user: models.Usuario =  
Depends(auth.get_current_active_user),  
    db: Session = Depends(get_db)  
):  
    """Obtener una transacción específica"""  
    transaccion = crud.get_transaccion(db,  
transaccion_id=transaccion_id, usuario_id=current_user.id)  
    if transaccion is None:  
        raise HTTPException(status_code=404, detail="Transacción  
no encontrada")  
    return transaccion
```

```
@app.patch("/transacciones/{transaccion_id}",  
response_model=schemas.TransaccionResponse)  
def actualizar_transaccion(  
    transaccion_id: int,  
    transaccion_update: schemas.TransaccionUpdate,  
    current_user: models.Usuario =  
Depends(auth.get_current_active_user),  
    db: Session = Depends(get_db)  
):  
    """Actualizar transacción (tipo, estado, descripción,  
etiquetas)"""  
    transaccion = crud.update_transaccion(  
        db=db,  
        transaccion_id=transaccion_id,
```

```
        usuario_id=current_user.id,
        transaccion_update=transaccion_update
    )
    if transaccion is None:
        raise HTTPException(status_code=404, detail="Transacción
no encontrada")
    return transaccion

@app.post("/transacciones/{transaccion_id}/confirmar",
response_model=schemas.TransaccionResponse)
def confirmar_transaccion(
    transaccion_id: int,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Confirmar una transacción recibida"""
    transaccion = crud.confirmar_transaccion(
        db=db,
        transaccion_id=transaccion_id,
        usuario_id=current_user.id
    )
    if transaccion is None:
        raise HTTPException(status_code=404, detail="Transacción
no encontrada")
    return transaccion

# Reportes
@app.get("/reportes/resumen",
response_model=schemas.ResumenTransacciones)
def obtener_resumen(
    fecha_inicio: Optional[str] = None,
    fecha_fin: Optional[str] = None,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Obtener resumen de transacciones"""
    return crud.get_resumen_transacciones(
        db=db,
```

```
        usuario_id=current_user.id,  
        fecha_inicio=fecha_inicio,  
        fecha_fin=fecha_fin  
    )
```

```
@app.get("/reportes/enviadas")  
def reporte_transacciones_enviadas(  
    current_user: models.Usuario =  
    Depends(auth.get_current_active_user),  
    db: Session = Depends(get_db)  
):  
    """Reporte detallado de transacciones enviadas"""  
    transacciones = crud.get_transacciones_usuario(  
        db=db,  
        usuario_id=current_user.id,  
        tipo=schemas.TipoTransaccion.ENVIADA  
    )  
  
    total = sum(t.monto for t in transacciones)  
    return {  
        "total_transacciones": len(transacciones),  
        "total_monto": total,  
        "transacciones": transacciones  
    }
```

```
@app.get("/reportes/recibidas")  
def reporte_transacciones_recibidas(  
    current_user: models.Usuario =  
    Depends(auth.get_current_active_user),  
    db: Session = Depends(get_db)  
):  
    """Reporte detallado de transacciones recibidas"""  
    transacciones = crud.get_transacciones_usuario(  
        db=db,  
        usuario_id=current_user.id,  
        tipo=schemas.TipoTransaccion.RECIBIDA  
    )
```

```
    pendientes = [t for t in transacciones if t.estado ==  
    schemas.EstadoTransaccion.PENDIENTE]
```



```
confirmadas = [t for t in transacciones if t.estado ==  
schemas.EstadoTransaccion.CONFIRMADA]
```

```
total = sum(t.monto for t in transacciones)  
return {  
    "total_transacciones": len(transacciones),  
    "pendientes": len(pendientes),  
    "confirmadas": len(confirmadas),  
    "total_monto": total,  
    "transacciones": transacciones  
}
```

```
# Etiquetas
```

```
@app.post("/etiquetas/",  
response_model=schemas.EtiquetaResponse)  
def crear_etiqueta(  
    etiqueta: schemas.EtiquetaCreate,  
    current_user: models.Usuario =  
Depends(auth.get_current_active_user),  
    db: Session = Depends(get_db)  
):  
    """Crear una nueva etiqueta"""  
    return crud.create_etiqueta(db=db, etiqueta=etiqueta,  
usuario_id=current_user.id)
```

```
@app.get("/etiquetas/",  
response_model=List[schemas.EtiquetaResponse])  
def listar_etiquetas(  
    current_user: models.Usuario =  
Depends(auth.get_current_active_user),  
    db: Session = Depends(get_db)  
):  
    """Listar etiquetas del usuario"""  
    return crud.get_etiquetas_usuario(db=db,  
usuario_id=current_user.id)
```

```
# Perfil de usuario
```

```
@app.get("/perfil", response_model=schemas.UserResponse)  
def obtener_perfil(  
    current_user: models.Usuario =
```

```

Depends(auth.get_current_active_user)
):
    """Obtener perfil del usuario actual"""
    return current_user

# Health check
@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "gestor-bancario"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Veo que tu API ya tiene un sistema de autenticación implementado con tokens JWT. Para agregar un usuario admin con credenciales fijas (admin/admin), necesitas modificar algunos archivos. Aquí te muestro cómo hacerlo:

1. Primero, modifica tu archivo `auth.py` :

python

```

from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
import os

from app import models, schemas
from app.database import get_db

# Configuración
SECRET_KEY = os.getenv("SECRET_KEY", "tu_clave_secreta_super_segura")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES =

```

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional

from app import crud, models, auth, schemas, utils
from app.database import engine, get_db, create_tables

# Crear tablas
create_tables()

app = FastAPI(
    title="API de Gestión Bancaria",
    description="API para procesar y gestionar transacciones bancarias con usuarios",
    version="2.0.0"
)

# Configurar CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Endpoints públicos
@app.post("/registro", response_model=schemas.UserResponse)
def registrar_usuario(usuario: schemas.UserCreate, db: Session = Depends(get_db)):
    print("recibido", usuario)
    db_user = crud.get_user_by_email(db, email=usuario.email)
    if db_user:
        raise HTTPException(
            status_code=400,
            detail="El email ya está registrado"
        )
    return crud.create_user(db=db, user=usuario)
```

```
@app.post("/token", response_model=schemas.Token)
def login_for_access_token(
    form_data: schemas.UserLogin,
    db: Session = Depends(get_db)
):
    user = auth.authenticate_user(db, form_data.email,
    form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token_expires =
    auth.timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email},
    expires_delta=access_token_expires
    )
    return {"access_token": access_token, "token_type": "bearer"}

@app.post("/procesar-texto")
def procesar_texto_transaccion(texto: str,
    db: Session = Depends(get_db)
):
    """Endpoint público para parsear texto sin guardar en DB"""
    try:
        datos = utils.parse_transaccion_texto(texto)

        print(datos.values())

        return {
            "success": True,
            "data": datos,
            "texto_enmascarado": {
                "beneficiario":
                utils.enmascarar_numero_cuenta(datos["beneficiario"]),
                "ordenante":
                utils.enmascarar_numero_cuenta(datos["ordenante"])
```

```

    }
}

```

```

except Exception as e:
    raise HTTPException(status_code=400, detail=f"Error al
procesar: {str(e)}")

```

```

# Endpoints protegidos

```

```

@app.post("/transacciones/parsear",
response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):

    """Crear transacción a partir de texto bancario"""
    # Verificar si la transacción ya existe
    datos = utils.parse_transaccion_texto(transaccion_req.texto)
    existente = db.query(models.Transaccion).filter(
        models.Transaccion.numero_transaccion ==
datos["numero_transaccion"],
        models.Transaccion.usuario_id == current_user.id
    ).first()
    print(datos)
    if existente:
        raise HTTPException(status_code=400, detail="Esta
transacción ya existe")

    return crud.parse_and_create_transaccion(
        db=db,
        texto=transaccion_req.texto,
        tipo=transaccion_req.tipo,
        usuario_id=current_user.id,
        descripcion=transaccion_req.descripcion
    )

```

```

@app.post("/transacciones/",

```

```
response_model=schemas.TransaccionResponse)
def crear_transaccion(
    transaccion: schemas.TransaccionCreate,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción manualmente"""
    return crud.create_transaccion(db=db,
transaccion=transaccion, usuario_id=current_user.id)

@app.get("/transacciones/",
response_model=List[schemas.TransaccionResponse])
def listar_transacciones(
    skip: int = 0,
    limit: int = 100,
    tipo: Optional[schemas.TipoTransaccion] = None,
    estado: Optional[schemas.EstadoTransaccion] = None,
    banco: Optional[str] = None,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Listar transacciones del usuario con filtros"""
    return crud.get_transacciones_usuario(
        db=db,
        usuario_id=current_user.id,
        skip=skip,
        limit=limit,
        tipo=tipo,
        estado=estado,
        banco=banco
    )

@app.get("/transacciones/{transaccion_id}",
response_model=schemas.TransaccionResponse)
def obtener_transaccion(
    transaccion_id: int,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
```

```
db: Session = Depends(get_db)
):
    """Obtener una transacción específica"""
    transaccion = crud.get_transaccion(db,
transaccion_id=transaccion_id, usuario_id=current_user.id)
    if transaccion is None:
        raise HTTPException(status_code=404, detail="Transacción
no encontrada")
    return transaccion

@app.patch("/transacciones/{transaccion_id}",
response_model=schemas.TransaccionResponse)
def actualizar_transaccion(
    transaccion_id: int,
    transaccion_update: schemas.TransaccionUpdate,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Actualizar transacción (tipo, estado, descripción,
etiquetas)"""
    transaccion = crud.update_transaccion(
        db=db,
        transaccion_id=transaccion_id,
        usuario_id=current_user.id,
        transaccion_update=transaccion_update
    )
    if transaccion is None:
        raise HTTPException(status_code=404, detail="Transacción
no encontrada")
    return transaccion

@app.post("/transacciones/{transaccion_id}/confirmar",
response_model=schemas.TransaccionResponse)
def confirmar_transaccion(
    transaccion_id: int,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
```

```
"""Confirmar una transacción recibida"""
transaccion = crud.confirmar_transaccion(
    db=db,
    transaccion_id=transaccion_id,
    usuario_id=current_user.id
)
if transaccion is None:
    raise HTTPException(status_code=404, detail="Transacción
no encontrada")
return transaccion
```

Reportes

```
@app.get("/reportes/resumen",
response_model=schemas.ResumenTransacciones)
def obtener_resumen(
    fecha_inicio: Optional[str] = None,
    fecha_fin: Optional[str] = None,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Obtener resumen de transacciones"""
    return crud.get_resumen_transacciones(
        db=db,
        usuario_id=current_user.id,
        fecha_inicio=fecha_inicio,
        fecha_fin=fecha_fin
    )
```

```
@app.get("/reportes/enviadas")
def reporte_transacciones_enviadas(
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Reporte detallado de transacciones enviadas"""
    transacciones = crud.get_transacciones_usuario(
        db=db,
        usuario_id=current_user.id,
        tipo=schemas.TipoTransaccion.ENVIADA
```


)

```
total = sum(t.monto for t in transacciones)
return {
    "total_transacciones": len(transacciones),
    "total_monto": total,
    "transacciones": transacciones
}
```

```
@app.get("/reportes/recibidas")
def reporte_transacciones_recibidas(
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Reporte detallado de transacciones recibidas"""
    transacciones = crud.get_transacciones_usuario(
        db=db,
        usuario_id=current_user.id,
        tipo=schemas.TipoTransaccion.RECIBIDA
    )
```

```
pendientes = [t for t in transacciones if t.estado ==
schemas.EstadoTransaccion.PENDIENTE]
confirmadas = [t for t in transacciones if t.estado ==
schemas.EstadoTransaccion.CONFIRMADA]
```

```
total = sum(t.monto for t in transacciones)
return {
    "total_transacciones": len(transacciones),
    "pendientes": len(pendientes),
    "confirmadas": len(confirmadas),
    "total_monto": total,
    "transacciones": transacciones
}
```

Etiquetas

```
@app.post("/etiquetas/",
response_model=schemas.EtiquetaResponse)
def crear_etiqueta(
```

```

    etiqueta: schemas.EtiquetaCreate,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear una nueva etiqueta"""
    return crud.create_etiqueta(db=db, etiqueta=etiqueta,
usuario_id=current_user.id)

@app.get("/etiquetas/",
response_model=List[schemas.EtiquetaResponse])
def listar_etiquetas(
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Listar etiquetas del usuario"""
    return crud.get_etiquetas_usuario(db=db,
usuario_id=current_user.id)

# Perfil de usuario
@app.get("/perfil", response_model=schemas.UserResponse)
def obtener_perfil(
    current_user: models.Usuario =
Depends(auth.get_current_active_user)
):
    """Obtener perfil del usuario actual"""
    return current_user

# Health check
@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "gestor-bancario"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Veo que tu API ya tiene un sistema de autenticación con JWT funcionando. Para agregar un usuario admin con credenciales fijas (admin/admin), necesitas

modificar algunos archivos. Aquí te muestro cómo hacerlo:

1. Modifica tu archivo `auth.py` :

python

```
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
import os

from app import models, schemas
from app.database import get_db

# Configuración
SECRET_KEY = os.getenv("SECRET_KEY", "tu_clave_secreta_super_segura")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# Credenciales fijas para admin (solo para desarrollo)
ADMIN_EMAIL = "admin@localhost"
ADMIN_PASSWORD = "admin"
ADMIN_USERNAME = "admin"

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password):
    return pwd_context.hash(password)

def authenticate_user(db: Session, email: str, password: str):
    # Verificar primero si es el admin
    if email == ADMIN_EMAIL:
        # Para admin, usar verificación directa (solo para desarrollo)
        if password == ADMIN_PASSWORD:
            # Crear o obtener usuario admin de la BD
```

```

admin_user = db.query(models.Usuario).filter(
    models.Usuario.email == ADMIN_EMAIL
).first()

if not admin_user:
    # Crear usuario admin si no existe
    admin_user = models.Usuario(
        email=ADMIN_EMAIL,
        nombre=ADMIN_USERNAME,
        hashed_password=get_password_hash(ADMIN_PASSWORD),
        is_active=True,
        is_admin=True
    )
    db.add(admin_user)
    db.commit()
    db.refresh(admin_user)

return admin_user

# Si no es admin, proceder con autenticación normal
user = db.query(models.Usuario).filter(models.Usuario.email == email).first()
if not user:
    return False
if not verify_password(password, user.hashed_password):
    return False
return user

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )

```

```

try:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    email: str = payload.get("sub")
    if email is None:
        raise credentials_exception
    token_data = schemas.TokenData(email=email)
except JWTError:
    raise credentials_exception

user = db.query(models.Usuario).filter(models.Usuario.email == token_data.emai
1).first()
if user is None:
    raise credentials_exception
return user

async def get_current_active_user(current_user: models.Usuario = Depends(get_curre
nt_user)):
    if not current_user.is_active:
        raise HTTPException(status_code=400, detail="Usuario inactivo")
    return current_user

def get_current_admin_user(current_user: models.Usuario = Depends(get_current_acti
ve_user)):
    """Dependencia para verificar si el usuario es admin"""
    if not hasattr(current_user, 'is_admin') or not current_user.is_admin:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Se requieren privilegios de administrador"
        )
    return current_user

```

2. Modifica tu archivo `models.py` para agregar el campo `is_admin` :

python

```

from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, Boole
an, Text
from sqlalchemy.orm import relationship
from datetime import datetime
from app.database import Base

```

```

class Usuario(Base):
    __tablename__ = "usuarios"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True, nullable=False)
    nombre = Column(String, nullable=False)
    hashed_password = Column(String, nullable=False)
    is_active = Column(Boolean, default=True)
    is_admin = Column(Boolean, default=False) # Nuevo campo
    created_at = Column(DateTime, default=datetime.utcnow)

    transacciones = relationship("Transaccion", back_populates="usuario")
    etiquetas = relationship("Etiqueta", back_populates="usuario")

# ... resto de tus modelos ...

```

3. Modifica tu archivo schemas.py :

python

```

from pydantic import BaseModel, EmailStr
from typing import Optional, List
from datetime import datetime
from enum import Enum

# ... tus enums existentes ...

class UserBase(BaseModel):
    email: EmailStr
    nombre: str

class UserCreate(UserBase):
    password: str

class UserResponse(UserBase):
    id: int
    is_active: bool
    is_admin: bool # Nuevo campo
    created_at: datetime

class Config:
    from_attributes = True

```

```
# ... resto de tus schemas ...
```

4. Agrega endpoints de admin en tu archivo principal:

```
python
```

```
# ... (todo tu código existente) ...
```

```
# Endpoints de Administrador
```

```
@app.get("/admin/usuarios", response_model=List[schemas.UserResponse])
```

```
def listar_usuarios(
```

```
    skip: int = 0,
```

```
    limit: int = 100,
```

```
    current_user: models.Usuario = Depends(auth.get_current_admin_user), # Solo a
```

```
admin
```

```
    db: Session = Depends(get_db)
```

```
):
```

```
    """Listar todos los usuarios (solo admin)"""
```

```
    usuarios = db.query(models.Usuario).offset(skip).limit(limit).all()
```

```
    return usuarios
```

```
@app.get("/admin/transacciones")
```

```
def listar_todas_transacciones(
```

```
    skip: int = 0,
```

```
    limit: int = 100,
```

```
    current_user: models.Usuario = Depends(auth.get_current_admin_user), # Solo a
```

```
admin
```

```
    db: Session = Depends(get_db)
```

```
):
```

```
    """Listar todas las transacciones del sistema (solo admin)"""
```

```
    transacciones = db.query(models.Transaccion).offset(skip).limit(limit).all()
```

```
    return {
```

```
        "total": len(transacciones),
```

```
        "transacciones": transacciones
```

```
    }
```

```
@app.patch("/admin/usuarios/{usuario_id}/toggle-active")
```

```
def toggle_usuario_activo(
```

```
    usuario_id: int,
```

```
    current_user: models.Usuario = Depends(auth.get_current_admin_user), # Solo a
```

```
admin
```

```

db: Session = Depends(get_db)
):
    """Activar/desactivar un usuario (solo admin)"""
    usuario = db.query(models.Usuario).filter(models.Usuario.id == usuario_id).first()
    if not usuario:
        raise HTTPException(status_code=404, detail="Usuario no encontrado")

    usuario.is_active = not usuario.is_active
    db.commit()

    return {
        "message": f"Usuario {'activado' if usuario.is_active else 'desactivado'} correctamente",
        "usuario_id": usuario.id,
        "is_active": usuario.is_active
    }

# ... resto de tu código ...

```

5. Script para crear el admin inicial (opcional):

Crea un archivo `scripts/create_admin.py` :

```

python

from sqlalchemy import create_many
from sqlalchemy.orm import sessionmaker
from app import models
from app.auth import get_password_hash
from app.database import SQLALCHEMY_DATABASE_URL

def create_admin_user():
    engine = create_engine(SQLALCHEMY_DATABASE_URL)
    SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
    db = SessionLocal()

    # Verificar si ya existe un admin
    admin = db.query(models.Usuario).filter(
        models.Usuario.email == "admin@localhost"
    ).first()

    if not admin:

```



```

admin_user = models.Usuario(
    email="admin@localhost",
    nombre="admin",
    hashed_password=get_password_hash("admin"),
    is_active=True,
    is_admin=True
)
db.add(admin_user)
db.commit()
print("Usuario admin creado exitosamente")
else:
    print("El usuario admin ya existe")

db.close()

if __name__ == "__main__":
    create_admin_user()

```

6. Para usar el admin:

1. **Login:** Haz una petición POST a `/token` con:

```

json

{
    "email": "admin@localhost",
    "password": "admin"
}

```

2. **Usar el token:** Inclúyelo en el header de las peticiones:

```

text

Authorization: Bearer <tu_token>

```

3. **Endpoints de admin:** Usa `/admin/usuarios` y otros endpoints protegidos que requieren rol de admin.

Notas importantes:

- **Solo para desarrollo:** Esta solución con credenciales fijas es solo para desarrollo/pruebas
- **Producción:** En producción, usa variables de entorno y contraseñas seguras
- **Seguridad:** El admin debería tener una contraseña segura en producción
- **Migración:** Necesitarás actualizar la base de datos para agregar el campo `is_admin`

¿Te gustaría que ajuste alguna parte específica o necesitas ayuda con la implementación?

mejor muéstrame como podría usarlo tal cual esta, es que me es un poco ambiguo registro un usuario y luego no me puedo autenticar con el _?

Ahora entiendo mejor tu problema. No puedes autenticarte con los usuarios que registras. Vamos a depurar el flujo completo de autenticación. Muéstrame primero el contenido de tu archivo `auth.py` actual para ver cómo está implementada la autenticación.

Pero mientras tanto, aquí te muestro cómo **usar tu API tal como está** paso a paso:

PASO 1: Registrar un usuario

bash

```
curl -X POST "http://localhost:8000/registro" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test@example.com",
    "nombre": "Test User",
    "password": "123456"
  }'
```

Respuesta esperada:

json

```
{
  "email": "test@example.com",
  "nombre": "Test User",
  "id": 1,
  "is_active": true,
  "created_at": "2024-01-01T00:00:00"
}
```

PASO 2: Hacer login (obtener token)

bash

```
curl -X POST "http://localhost:8000/token" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test@example.com",
    "password": "123456"
  }'
```

Respuesta esperada:

json

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

PASO 3: Usar el token para endpoints protegidos

bash

```
# Crear una transacción
curl -X POST "http://localhost:8000/transacciones/" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer TU_TOKEN_AQUI" \
  -d '{
    "monto": 100.50,
    "tipo": "ENVIADA",
  }'
```

```
"descripcion": "Pago de servicios"
}'
```

¿Por qué no puedes autenticarte? Vamos a depurar:

Agrega estos prints en tu endpoint `/registro` y `/token` para ver qué está pasando:

Modifica tu endpoint de registro:

python

```
@app.post("/registro", response_model=schemas.UserResponse)
def registrar_usuario(usuario: schemas.UserCreate, db: Session = Depends(get_db)):
    print("="*50)
    print("INTENTANDO REGISTRAR USUARIO:")
    print(f"Email: {usuario.email}")
    print(f"Nombre: {usuario.nombre}")
    print(f>Password recibida: {usuario.password}")

    # Verificar si el email ya existe
    db_user = crud.get_user_by_email(db, email=usuario.email)
    if db_user:
        print(f"❌ Email ya registrado: {usuario.email}")
        raise HTTPException(
            status_code=400,
            detail="El email ya está registrado"
        )

    # Crear usuario
    nuevo_usuario = crud.create_user(db=db, user=usuario)
    print(f"✅ Usuario creado con ID: {nuevo_usuario.id}")
    print(f>Password hashada: {nuevo_usuario.hashed_password}")
    print("="*50)

    return nuevo_usuario
```

Modifica tu endpoint de login:

python

```

@app.post("/token", response_model=schemas.Token)
def login_for_access_token(
    form_data: schemas.UserLogin,
    db: Session = Depends(get_db)
):
    print("="*50)
    print("INTENTANDO LOGIN:")
    print(f"Email: {form_data.email}")
    print(f"Password recibida: {form_data.password}")

    # Buscar usuario en BD
    user = db.query(models.Usuario).filter(models.Usuario.email == form_data.email).first()

    if user:
        print(f"✅ Usuario encontrado en BD")
        print(f"ID: {user.id}")
        print(f"Password hasheada en BD: {user.hashed_password}")

        # Verificar password (asumiendo que usas verify_password)
        from app.auth import verify_password
        password_valida = verify_password(form_data.password, user.hashed_password)

        print(f"¿Password válida? {password_valida}")
    else:
        print(f"❌ Usuario NO encontrado: {form_data.email}")

    # Autenticar
    user = auth.authenticate_user(db, form_data.email, form_data.password)

    if not user:
        print(f"❌ Autenticación falló")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    print(f"✅ Autenticación exitosa")

    access_token_expires = auth.timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTE)

    access_token = auth.create_access_token(

```

```

        data={"sub": user.email}, expires_delta=access_token_expires
    )

    print("="*50)
    return {"access_token": access_token, "token_type": "bearer"}

```

Script de prueba rápido

Crea un archivo `test_auth.py` :

python

```

import requests
import json

BASE_URL = "http://localhost:8000"

def test_registro():
    print("\n1. Registrando usuario...")
    data = {
        "email": "test@example.com",
        "nombre": "Test User",
        "password": "123456"
    }

    response = requests.post(f"{BASE_URL}/registro", json=data)
    print(f"Status: {response.status_code}")
    print(f"Respuesta: {response.json()}")
    return response.json()

def test_login():
    print("\n2. Haciendo login...")
    data = {
        "email": "test@example.com",
        "password": "123456"
    }

    response = requests.post(f"{BASE_URL}/token", json=data)
    print(f"Status: {response.status_code}")

    if response.status_code == 200:
        token_data = response.json()
        print(f"Token obtenido: {token_data['access_token'][:20]}...")

```

```

        return token_data['access_token']
    else:
        print(f"Error: {response.json()}")
        return None

def test_endpoint_protegido(token):
    print("\n3. Probando endpoint protegido...")
    headers = {
        "Authorization": f"Bearer {token}"
    }

    response = requests.get(f"{BASE_URL}/perfil", headers=headers)
    print(f"Status: {response.status_code}")
    print(f"Respuesta: {response.json()}")

if __name__ == "__main__":
    # Primero intenta Login (asumiendo que el usuario ya existe)
    token = test_login()

    # Si login falla, registra y luego login
    if not token:
        test_registro()
        token = test_login()

    # Si tenemos token, prueba endpoint
    if token:
        test_endpoint_protegido(token)

```

Posibles problemas comunes:

1. La contraseña no se está hasheando correctamente al registrar
2. El método de verificación de password no coincide con el hash
3. La tabla de usuarios no se está creando correctamente

Muéstrame tu archivo `auth.py` para poder ver exactamente cómo estás manejando:

- La función `get_password_hash()`
- La función `verify_password()`
- La función `authenticate_user()`

Con eso podré decirte exactamente qué está fallando en tu autenticación.

```
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from . import models, schemas
from .database import get_db
import os

# Configuración
SECRET_KEY = os.getenv("SECRET_KEY",
"clave_secreta_super_segura_aqui_cambiar_en_produccion")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

# Configurar CryptContext con Argon2 - VERSIÓN CORREGIDA
# Quitamos el parámetro 'encoding' que no es soportado
pwd_context = CryptContext(
    schemes=["argon2"], # Usar Argon2 en lugar de bcrypt
    deprecated="auto",

    # Parámetros de Argon2 (ajustables según necesidades de
    seguridad/rendimiento)
    argon2__time_cost=2,      # Número de iteraciones (mayor =
    más seguro, más lento)
    argon2__memory_cost=102400, # Memoria en KB (102400 KB
    = 100 MB)
    argon2__parallelism=8,    # Número de hilos paralelos
    argon2__hash_len=32,      # Longitud del hash en bytes
    argon2__salt_len=16,      # Longitud del salt en bytes
    # NOTA: 'encoding' no es un parámetro válido para Argon2 en
    passlib
)

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```



```
def get_password_hash(password: str) -> str:
    """
    Hash de contraseña usando Argon2.

    Args:
        password: Contraseña en texto plano

    Returns:
        str: Hash de la contraseña
    """
    try:
        return pwd_context.hash(password)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Error al generar hash de contraseña: {str(e)}"
        )

def verify_password(plain_password: str, hashed_password: str) -
> bool:

    print(plain_password)
    print(hashed_password)
    """
    Verificar contraseña usando Argon2.

    Args:
        plain_password: Contraseña en texto plano
        hashed_password: Hash almacenado

    Returns:
        bool: True si la contraseña es correcta, False en caso
    contrario
    """
    try:
        return pwd_context.verify(plain_password,
hashed_password)
    except Exception as e:
        # Log del error
```

```
print(f"Error al verificar contraseña: {str(e)}")
return False
```

```
def authenticate_user(db: Session, email: str, password: str):
    user = db.query(models.Usuario).filter(models.Usuario.email ==
email).first()
    if not user:
        return False
    if not verify_password(password, user.hashed_password):
        return False
    return user
```

```
def create_access_token(data: dict, expires_delta:
Optional[timedelta] = None):
    to_encode = data.copy()

    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

    to_encode.update({"exp": expire, "iat": datetime.utcnow()})

    try:
        encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)
        return encoded_jwt
    except JWTError as e:
        raise HTTPException(
```

```
status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Error al generar token: {str(e)}"
    )
```

```
# Dependencia para obtener usuario actual
async def get_current_user(token: str =
Depends(oauth2_scheme), db: Session = Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
```

```

        detail="Credenciales inválidas",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=
[ALGORITHM])
        email: str = payload.get("sub")
        if email is None:
            raise credentials_exception
        token_data = schemas.TokenData(email=email)
    except JWTError:
        raise credentials_exception

    user = db.query(models.Usuario).filter(models.Usuario.email ==
token_data.email).first()
    if user is None:
        raise credentials_exception
    print(user)
    return user

async def get_current_active_user(current_user: models.Usuario =
Depends(get_current_user)):
    if not current_user.esta_activo:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Usuario inactivo"
        )
    return current_user

```

Perfecto! Ya veo el problema. Estás usando Argon2 para hashear contraseñas, lo cual es excelente para seguridad, pero necesitamos asegurarnos de que el registro y login funcionen correctamente.

El problema podría ser que al registrar, la contraseña no se está hasheando correctamente. Vamos a arreglarlo paso a paso.

Solución: Corregir el registro de usuarios

1. Primero, modifica tu archivo crud.py :

python

```
from sqlalchemy.orm import Session
from app import models, schemas
from app.auth import get_password_hash # Importa la función de hash

def create_user(db: Session, user: schemas.UserCreate):
    """Crear un nuevo usuario con contraseña hasheada"""

    # Hashear la contraseña usando Argon2
    hashed_password = get_password_hash(user.password)

    print(f"📄 Creando usuario: {user.email}")
    print(f"🔑 Password original: {user.password}")
    print(f"🔒 Password hasheada: {hashed_password}")

    # Crear el usuario con la contraseña hasheada
    db_user = models.Usuario(
        email=user.email,
        nombre=user.nombre,
        hashed_password=hashed_password, # Guardar el hash, no la contraseña en t
exto plano
        esta_activo=True
    )

    try:
        db.add(db_user)
        db.commit()
        db.refresh(db_user)
        print(f"✅ Usuario {user.email} creado con ID: {db_user.id}")
        return db_user
    except Exception as e:
        db.rollback()
        print(f"❌ Error al crear usuario: {str(e)}")
        raise

def get_user_by_email(db: Session, email: str):
    """Obtener usuario por email"""
    return db.query(models.Usuario).filter(models.Usuario.email == email).first()
```

2. Modifica tu archivo `schemas.py` para asegurar que `UserCreate` tenga el campo `password`:

python

```
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
from datetime import datetime
from enum import Enum

# ... tus enums ...

class UserBase(BaseModel):
    email: EmailStr
    nombre: str = Field(..., min_length=1, max_length=100)

class UserCreate(UserBase):
    password: str = Field(..., min_length=4, description="Contraseña del usuario")

class UserResponse(UserBase):
    id: int
    esta_activo: bool
    created_at: datetime

    class Config:
        from_attributes = True

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    email: Optional[str] = None

# ... resto de tus schemas ...
```

3. Modifica tu modelo `models.py` :

python

```
from sqlalchemy import Column, Integer, String, Boolean, DateTime
from datetime import datetime
from app.database import Base

class Usuario(Base):
    __tablename__ = "usuarios"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True, nullable=False)
    nombre = Column(String, nullable=False)
    hashed_password = Column(String, nullable=False)
    esta_activo = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relaciones
    transacciones = relationship("Transaccion", back_populates="usuario", cascade="all, delete-orphan")
    etiquetas = relationship("Etiqueta", back_populates="usuario", cascade="all, delete-orphan")
```

4. Crea un script de prueba test_auth_completo.py :

python

```
import requests
import json
import time

BASE_URL = "http://localhost:8000"

def test_registro_con_print():
    """Prueba el registro mostrando todo el detalle"""
    print("\n" + "="*60)
    print("🔧 TEST DE REGISTRO")
    print("="*60)

    # Datos de prueba
    email = f"test{int(time.time())}@example.com"
    password = "123456"
    nombre = "Usuario Test"
```

```
print(f"📧 Email: {email}")
print(f"🔑 Password: {password}")
print(f"👤 Nombre: {nombre}")

# 1. Registrar usuario
print("\n📡 Enviando petición POST a /registro...")
registro_data = {
    "email": email,
    "nombre": nombre,
    "password": password
}

response = requests.post(f"{BASE_URL}/registro", json=registro_data)

print(f"📡 Status Code: {response.status_code}")
print(f"📡 Respuesta: {json.dumps(response.json(), indent=2)}")

if response.status_code != 200:
    print("❌ Registro falló")
    return None

usuario = response.json()
print(f"✅ Usuario registrado con ID: {usuario.get('id')}")

# 2. Hacer Login
print("\n" + "="*60)
print("🔧 TEST DE LOGIN")
print("="*60)

login_data = {
    "email": email,
    "password": password
}

print(f"📡 Enviando petición POST a /token con:")
print(f"    Email: {email}")
print(f"    Password: {password}")

response = requests.post(f"{BASE_URL}/token", json=login_data)

print(f"📡 Status Code: {response.status_code}")

if response.status_code == 200:
    token_data = response.json()
```

```

    print(f"🔑 Respuesta: Token obtenido!")
    print(f"    Token: {token_data['access_token'][:30]}...")
    print(f"    Tipo: {token_data['token_type']}")
    print(f"✅ LOGIN EXITOSO!")
    return token_data['access_token']
else:
    print(f"❌ Login falló: {response.json()}")
    return None

def test_endpoints_protegidos(token):
    """Prueba endpoints que requieren autenticación"""
    print("\n" + "="*60)
    print("🔧 TEST DE ENDPOINTS PROTEGIDOS")
    print("="*60)

    headers = {
        "Authorization": f"Bearer {token}"
    }

    # Probar /perfil
    print("\n🔑 GET /perfil")
    response = requests.get(f"{BASE_URL}/perfil", headers=headers)
    print(f"🔑 Status: {response.status_code}")
    if response.status_code == 200:
        print(f"🔑 Perfil: {json.dumps(response.json(), indent=2)}")
    else:
        print(f"❌ Error: {response.json()}")

    # Probar crear transacción simple
    print("\n🔑 POST /transacciones/ (crear transacción)")
    transaccion_data = {
        "monto": 150.50,
        "tipo": "ENVIADA",
        "descripcion": "Pago de prueba",
        "fecha": datetime.now().isoformat()
    }

    response = requests.post(
        f"{BASE_URL}/transacciones/",
        headers=headers,
        json=transaccion_data
    )
    print(f"🔑 Status: {response.status_code}")
    if response.status_code == 200:

```



```

        print(f"📄 Transacción creada: {json.dumps(response.json(), indent=2)}")
    else:
        print(f"❌ Error: {response.json()}")

if __name__ == "__main__":
    # Ejecutar pruebas
    print("🚀 INICIANDO PRUEBAS DE AUTENTICACIÓN")
    print("="*60)

    token = test_registro_con_print()

    if token:
        test_endpoints_protegidos(token)
        print("\n" + "="*60)
        print("🎉 TODAS LAS PRUEBAS PASARON!")
        print("="*60)
    else:
        print("\n" + "="*60)
        print("❌ LAS PRUEBAS FALLARON")
        print("="*60)

```

5. Comandos para probar manualmente con curl:

bash

```

# 1. Registrar usuario
curl -X POST "http://localhost:8000/registro" \
-H "Content-Type: application/json" \
-d '{
    "email": "test@example.com",
    "nombre": "Test User",
    "password": "123456"
}'

# 2. Hacer Login (obtener token)
curl -X POST "http://localhost:8000/token" \
-H "Content-Type: application/json" \
-d '{
    "email": "test@example.com",
    "password": "123456"
}'

# 3. Usar el token (reemplaza TOKEN con el obtenido)

```

```
curl -X GET "http://localhost:8000/perfil" \
-H "Authorization: Bearer TU_TOKEN_AQUI"
```

6. Verificar en la base de datos:

Si estás usando SQLite, puedes verificar directamente:

```
bash
```

```
# Instalar sqlite3 si no lo tienes
sqlite3 bancaria.db "SELECT id, email, nombre, hashed_password, esta_activo FROM usuarios;"
```

Problemas comunes y soluciones:

1. **Error "Username and password required":** Asegúrate de que el modelo `UserCreate` en `schemas.py` tenga el campo `password`.
2. **La contraseña no se hashea:** Verifica que en `crud.py` estés usando `get_password_hash()` antes de guardar.
3. **Error de importación:** Asegúrate de que todos los imports sean correctos:

```
python
```

```
from app.auth import get_password_hash # Si usas estructura de paquetes
# o
from auth import get_password_hash # Si auth.py está en el mismo directorio
```

4. Verificar que Argon2 esté instalado:

```
bash
```

```
pip install argon2-cffi
```

Ejecuta el script de prueba y comparte los resultados para ver exactamente dónde falla la autenticación.

argon2 esta instalado correctamente... mira mi esquema de autenticacion "# ===== ESQUEMAS DE

AUTENTICACIÓN =====

```
class UserCreate(BaseModel):
    email: EmailStr = Field(..., description="Email del usuario")
    nombre: str = Field(..., min_length=2, max_length=50,
                        description="Nombre del usuario (2-50
caracteres)")
    apellido: str = Field(..., min_length=2, max_length=50,
                        description="Apellido del usuario (2-50
caracteres)")
    password: str = Field(..., min_length=6, max_length=128,
                        description="Contraseña (mínimo 6 caracteres)")
```

```
@field_validator('nombre', 'apellido')
@classmethod
def validar_nombre_apellido(cls, v: str) -> str:
    """Validar que solo contenga letras y espacios"""
    v = v.strip()
    if not v.replace(" ", "").replace("-", "").isalpha():
        raise ValueError('Solo se permiten letras, espacios y
guiones')
    if len(v) < 2:
        raise ValueError('Debe tener al menos 2 caracteres')
    return v.title()
```

```
@field_validator('email')
@classmethod
def normalizar_email(cls, v: str) -> str:
    """Normalizar email a minúsculas"""
    return v.strip().lower()
```

```
class UserLogin(BaseModel):
    email: EmailStr = Field(..., description="Email del usuario")
    password: str = Field(..., min_length=1, max_length=128,
                        description="Contraseña del usuario")
```

```
class UserResponse(BaseModel):
    id: int
    email: EmailStr
    nombre: str
    apellido: str
```

```
esta_activo: bool
```

```
fecha_creacion: datetime
```

```
model_config = ConfigDict(from_attributes=True)
```

```
class Token(BaseModel):
```

```
    access_token: str
```

```
    token_type: str = "bearer"
```

```
    expires_in: Optional[int] = Field(1800, description="Tiempo de
    expiración en segundos")
```

```
class TokenData(BaseModel):
```

```
    email: Optional[str] = None
```

```
    user_id: Optional[int] = None"
```

Perfecto! Ya veo la estructura. El problema es que en tu modelo `Usuario` probablemente espera un campo `nombre` y `apellido`, pero en el registro estás enviando solo `nombre`. Vamos a arreglarlo:

Solución completa para autenticación

1. Primero, verifica tu modelo `models.py` (debe tener nombre y apellido):

```
python
```

```
from sqlalchemy import Column, Integer, String, Boolean, DateTime
```

```
from datetime import datetime
```

```
from app.database import Base
```

```
class Usuario(Base):
```

```
    __tablename__ = "usuarios"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    email = Column(String, unique=True, index=True, nullable=False)
```

```
    nombre = Column(String, nullable=False)
```

```
    apellido = Column(String, nullable=False) # ← Asegúrate que existe
```

```
    hashed_password = Column(String, nullable=False)
```

```
    esta_activo = Column(Boolean, default=True)
```

```
    fecha_creacion = Column(DateTime, default=datetime.utcnow)
```

```
# Relaciones (si las tienes)
transacciones = relationship("Transaccion", back_populates="usuario")
```

2. Corrige tu crud.py para manejar nombre y apellido:

python

```
from sqlalchemy.orm import Session
from app import models, schemas
from app.auth import get_password_hash

def create_user(db: Session, user: schemas.UserCreate):
    """Crear un nuevo usuario con contraseña hasheada"""

    print("="*60)
    print("📄 CREATE_USER - Datos recibidos:")
    print(f"Email: {user.email}")
    print(f"Nombre: {user.nombre}")
    print(f"Apellido: {user.apellido}")
    print(f>Password (original): {user.password}")

    # Hashear la contraseña
    hashed = get_password_hash(user.password)
    print(f>Password (hasheada): {hashed}")

    # Crear usuario con TODOS los campos
    db_user = models.Usuario(
        email=user.email,
        nombre=user.nombre,
        apellido=user.apellido, # ← Importante incluir apellido
        hashed_password=hashed,
        esta_activo=True
    )

    try:
        db.add(db_user)
        db.commit()
        db.refresh(db_user)
        print("✅ Usuario creado exitosamente!")
        print(f"ID: {db_user.id}")
        print("="*60)
        return db_user
```

```

except Exception as e:
    db.rollback()
    print(f"❌ Error al crear usuario: {str(e)}")
    print("="*60)
    raise HTTPException(
        status_code=500,
        detail=f"Error al crear usuario: {str(e)}"
    )

def get_user_by_email(db: Session, email: str):
    """Obtener usuario por email"""
    print(f"🔍 Buscando usuario con email: {email}")
    user = db.query(models.Usuario).filter(models.Usuario.email == email).first()
    if user:
        print(f"✅ Usuario encontrado: {user.nombre} {user.apellido}")
    else:
        print(f"❌ Usuario no encontrado")
    return user

```

3. Actualiza tu auth.py con mejor depuración:

python

```

from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from . import models, schemas
from .database import get_db
import os

# Configuración
SECRET_KEY = os.getenv("SECRET_KEY", "clave_secreta_super_segura")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(
    schemes=["argon2"],
    deprecated="auto",
    argon2__time_cost=2,

```

```

    argon2__memory_cost=102400,
    argon2__parallelism=8,
)

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_password_hash(password: str) -> str:
    """Hash de contraseña usando Argon2"""
    try:
        hashed = pwd_context.hash(password)
        print(f"🔒 Hashing - Original: {password} -> Hash: {hashed[:30]}...")
        return hashed
    except Exception as e:
        print(f"❌ Error hashing: {str(e)}")
        raise

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verificar contraseña"""
    try:
        print(f"🔒 Verificando password...")
        print(f" Plain: {plain_password}")
        print(f" Hash: {hashed_password[:30]}...")

        result = pwd_context.verify(plain_password, hashed_password)
        print(f" Resultado: {result}")
        return result
    except Exception as e:
        print(f"❌ Error verifying: {str(e)}")
        return False

def authenticate_user(db: Session, email: str, password: str):
    """Autenticar usuario"""
    print(f"={*60}")
    print(f"🔒 Autenticando usuario: {email}")

    # Buscar usuario
    user = db.query(models.Usuario).filter(models.Usuario.email == email).first()

    if not user:
        print(f"❌ Usuario no encontrado: {email}")
        return False

    print(f"✅ Usuario encontrado: {user.nombre} {user.apellido}")
    print(f" ID: {user.id}")

```

```

print(f"    Hash en BD: {user.hashed_password[:30]}...")

# Verificar contraseña
if not verify_password(password, user.hashed_password):
    print("❌ Contraseña incorrecta")
    return False

print("✅ Autenticación exitosa!")
print("="*60)
return user

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    """Crear token JWT"""
    to_encode = data.copy()

    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTE
S)

    to_encode.update({"exp": expire})

    try:
        encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
        return encoded_jwt
    except Exception as e:
        print(f"❌ Error creando token: {str(e)}")
        raise

async def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depe
nds(get_db)):
    """Obtener usuario actual desde token"""
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="No se pudieron validar las credenciales",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        print(f"🔑 Decodificando token...")
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email: str = payload.get("sub")
        if email is None:

```



```

        print("❌ Token no contiene email")
        raise credentials_exception

    print(f"✅ Token válido para: {email}")
except JWTError as e:
    print(f"❌ Error decodificando token: {str(e)}")
    raise credentials_exception

user = db.query(models.Usuario).filter(models.Usuario.email == email).first()
if user is None:
    print(f"❌ Usuario no encontrado: {email}")
    raise credentials_exception

print(f"✅ Usuario autenticado: {user.nombre} {user.apellido}")
return user

async def get_current_active_user(current_user: models.Usuario = Depends(get_current_user)):
    """Verificar que el usuario esté activo"""
    if not current_user.esta_activo:
        raise HTTPException(status_code=400, detail="Usuario inactivo")
    return current_user

```

4. Script de prueba completo test_full_auth.py :

```

python

import requests
import json
import time
from datetime import datetime

BASE_URL = "http://localhost:8000"

def test_registro_completo():
    """Prueba registro con nombre y apellido"""
    print("\n" + "="*60)
    print("📝 TEST DE REGISTRO COMPLETO")
    print("="*60)

    # Generar email único
    timestamp = int(time.time())
    email = f"test{timestamp}@example.com"

```

```

# Datos completos del usuario
user_data = {
    "email": email,
    "nombre": "Juan",
    "apellido": "Pérez", # ← Importante incluir apellido
    "password": "123456"
}

print(f"📧 Email: {email}")
print(f"👤 Nombre: {user_data['nombre']} {user_data['apellido']}")
print(f"🔑 Password: {user_data['password']}")

# 1. REGISTRO
print("\n📡 Enviando POST a /registro...")
response = requests.post(f"{BASE_URL}/registro", json=user_data)

print(f"📡 Status: {response.status_code}")

if response.status_code == 200:
    user_created = response.json()
    print(f"✅ Registro exitoso!")
    print(f"ID: {user_created.get('id')}")
    print(f"Email: {user_created.get('email')}")
    print(f"Nombre: {user_created.get('nombre')} {user_created.get('apellido')}")
    return user_data
else:
    print(f"❌ Error en registro: {response.json()}")
    return None

def test_login(email, password):
    """Prueba de login"""
    print("\n" + "="*60)
    print("🔑 TEST DE LOGIN")
    print("="*60)

    login_data = {
        "email": email,
        "password": password
    }

    print(f"📡 Enviando POST a /token...")
    print(f"Email: {email}")
    print(f"Password: {password}")

```

```

response = requests.post(f"{BASE_URL}/token", json=login_data)

print(f"📄 Status: {response.status_code}")

if response.status_code == 200:
    token_data = response.json()
    print(f"✅ Login exitoso!")
    print(f"Token: {token_data['access_token'][:50]}...")
    print(f"Tipo: {token_data['token_type']}")
    return token_data['access_token']
else:
    print(f"❌ Error en login: {response.json()}")
    return None

def test_perfil(token):
    """Prueba endpoint /perfil"""
    print("\n" + "="*60)
    print("🔧 TEST DE PERFIL")
    print("="*60)

    headers = {
        "Authorization": f"Bearer {token}"
    }

    response = requests.get(f"{BASE_URL}/perfil", headers=headers)

    print(f"📄 Status: {response.status_code}")

    if response.status_code == 200:
        perfil = response.json()
        print(f"✅ Perfil obtenido:")
        print(f"ID: {perfil.get('id')}")
        print(f>Email: {perfil.get('email')}")
        print(f>Nombre: {perfil.get('nombre')} {perfil.get('apellido')}")
    else:
        print(f"❌ Error: {response.json()}")

def test_crear_transaccion(token):
    """Prueba crear una transacción"""
    print("\n" + "="*60)
    print("🔧 TEST CREAR TRANSACCIÓN")
    print("="*60)

```

```

headers = {
    "Authorization": f"Bearer {token}"
}

transaccion_data = {
    "monto": 1500.50,
    "tipo": "ENVIADA",
    "descripcion": "Pago de servicios",
    "fecha": datetime.now().isoformat()
}

response = requests.post(
    f"{BASE_URL}/transacciones/",
    headers=headers,
    json=transaccion_data
)

print(f"📄 Status: {response.status_code}")

if response.status_code == 200:
    transaccion = response.json()
    print(f"✅ Transacción creada:")
    print(f"    ID: {transaccion.get('id')}")
    print(f"    Monto: {transaccion.get('monto')}")
else:
    print(f"❌ Error: {response.json()}")

def test_parsear_texto(token):
    """Prueba el endpoint de parsear texto"""
    print("\n" + "="*60)
    print("📝 TEST PARSEAR TEXTO")
    print("="*60)

    headers = {
        "Authorization": f"Bearer {token}"
    }

    texto = """
Transferencia realizada
Ordenante: Juan Pérez (ES12345678901234567890)
Beneficiario: María García (ES98765432109876543210)
Importe: 150.50 EUR
Concepto: Pago factura
"""

```

```

data = {
    "texto": texto,
    "tipo": "ENVIADA",
    "descripcion": "Prueba de parseo"
}

response = requests.post(
    f"{BASE_URL}/transacciones/parsear",
    headers=headers,
    json=data
)

print(f"📡 Status: {response.status_code}")

if response.status_code == 200:
    resultado = response.json()
    print(f"✅ Parseo exitoso:")
    print(f"    ID: {resultado.get('id')}")
    print(f"    Monto: {resultado.get('monto')}")
    print(f"    Ordenante: {resultado.get('ordenante')}")
else:
    print(f"❌ Error: {response.json()}")

if __name__ == "__main__":
    print("🚀 INICIANDO PRUEBAS COMPLETAS")
    print("="*60)

    # 1. Registrar usuario
    user_data = test_registro_completo()

    if user_data:
        # 2. Hacer login
        token = test_login(user_data["email"], user_data["password"])

        if token:
            # 3. Probar endpoints
            test_perfil(token)
            test_crear_transaccion(token)
            test_parsear_texto(token)

            print("\n" + "="*60)
            print("🎉 TODAS LAS PRUEBAS COMPLETADAS!")
            print("="*60)

```

```

else:
    print("\n❌ Las pruebas fallaron en LOGIN")
else:
    print("\n❌ Las pruebas fallaron en REGISTRO")

```

5. Comandos curl para probar manualmente:

bash

1. REGISTRO (con nombre y apellido)

```

curl -X POST "http://localhost:8000/registro" \
-H "Content-Type: application/json" \
-d '{
    "email": "test@example.com",
    "nombre": "Juan",
    "apellido": "Pérez",
    "password": "123456"
}'

```

2. LOGIN

```

curl -X POST "http://localhost:8000/token" \
-H "Content-Type: application/json" \
-d '{
    "email": "test@example.com",
    "password": "123456"
}'

```

3. PERFIL (con el token obtenido)

```

curl -X GET "http://localhost:8000/perfil" \
-H "Authorization: Bearer TU_TOKEN_AQUI"

```

4. Si el login falla, verifica en BD

```

sqlite3 bancaria.db "SELECT id, email, nombre, apellido, hashed_password, esta_activo FROM usuarios;"

```

6. Verificación rápida en Python console:

python

```

# Verifica que el hash y verificación funcionan
from app.auth import get_password_hash, verify_password

```

```
password = "123456"  
hashed = get_password_hash(password)  
print(f"Hash: {hashed}")  
print(f"Verificación: {verify_password(password, hashed)}") # Debe ser True
```

El problema más probable es que **en el registro no estás incluyendo el campo apellido**, o que en tu modelo `Usuario` espera `apellido` pero no lo estás proporcionando.

Ejecuta el script de prueba completo y comparte los resultados para ver exactamente dónde falla.

ya me registro bien y me valida que el email no este en uso, pero en la documentacion el boton autorizar tiene un usuario y contraseña, le pongo el correo y la contraseña y no autentica, tal vez este esperando el nombre que imagino que no debe ser porque no es unico el email si, dime que modelo es el que interactua con esa parte

¡Excelente pregunta! El botón "Authorize" en la documentación de FastAPI (Swagger UI) está esperando credenciales en un formato específico. Te explico:

El problema con el botón "Authorize"

El botón "Authorize" de Swagger UI está configurado para usar **OAuth2PasswordBearer**, que por defecto espera:

- **username:** El nombre de usuario (en tu caso, debería ser el EMAIL)
- **password:** La contraseña

Pero hay un detalle importante: **Swagger UI envía los datos como application/x-www-form-urlencoded NO como JSON.**

Solución: Crear un endpoint específico para Swagger

Modifica tu endpoint `/token` para aceptar ambos formatos:

python

```

from fastapi import FastAPI, Depends, HTTPException, status, Form
from fastapi.security import OAuth2PasswordRequestForm
# ... tus otros imports

@app.post("/token", response_model=schemas.Token)
async def login_for_access_token(
    # Esto acepta tanto JSON como form-data
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db)
):
    """
    Endpoint para login:
    - Acepta application/x-www-form-urlencoded (para Swagger UI)
    - Acepta application/json (para tus clientes)
    """
    print("="*60)
    print("🔑 LOGIN ENDPOINT")
    print(f"Username recibido: {form_data.username}")
    print(f>Password recibida: {'*' * len(form_data.password)}")

    # Buscar usuario por email (username = email)
    user = db.query(models.Usuario).filter(
        models.Usuario.email == form_data.username
    ).first()

    if not user:
        print("❌ Usuario no encontrado")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Verificar contraseña
    from app.auth import verify_password
    if not verify_password(form_data.password, user.hashed_password):
        print("❌ Contraseña incorrecta")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

```



```

# Crear token
from app.auth import create_access_token, ACCESS_TOKEN_EXPIRE_MINUTES
from datetime import timedelta

access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
access_token = create_access_token(
    data={"sub": user.email}, expires_delta=access_token_expires
)

print(f"✅ Login exitoso para: {user.email}")
print(f"="*60)

return {
    "access_token": access_token,
    "token_type": "bearer",
    "expires_in": ACCESS_TOKEN_EXPIRE_MINUTES * 60
}

# Mantén también tu endpoint JSON original si lo necesitas
@app.post("/token/json", response_model=schemas.Token)
def login_json(
    login_data: schemas.UserLogin,
    db: Session = Depends(get_db)
):
    """Endpoint alternativo que acepta JSON"""
    user = auth.authenticate_user(db, login_data.email, login_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
        )

    access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email}, expires_delta=access_token_expires
    )
    return {"access_token": access_token, "token_type": "bearer"}

```

Para usar el botón "Authorize" correctamente:

1. Haz clic en "Authorize" (arriba a la derecha en /docs)

2. En el formulario que aparece:

- **Username:** Introduce tu **EMAIL** (ej: test@example.com)
- **Password:** Introduce tu **CONTRASEÑA** (ej: 123456)
- **Client_id:** Déjalo vacío
- **Client_secret:** Déjalo vacío

3. Haz clic en "Authorize"

4. Verás un mensaje "Authorized" y todos los endpoints con candado ya se pueden probar

Solución alternativa: Endpoint dedicado para Swagger

Si quieres mantener tu endpoint JSON actual, crea uno específico para Swagger:

python

```
@app.post("/token", response_model=schemas.Token)
async def login_swagger(
    username: str = Form(...), # Para Swagger UI
    password: str = Form(...), # Para Swagger UI
    db: Session = Depends(get_db)
):
    """Endpoint específico para Swagger UI (form-data)"""
    print(f"Swagger login - Username: {username}")

    user = db.query(models.Usuario).filter(
        models.Usuario.email == username
    ).first()

    if not user or not auth.verify_password(password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Credenciales incorrectas",
        )

    access_token = auth.create_access_token(data={"sub": user.email})
    return {"access_token": access_token, "token_type": "bearer"}
```

Resumen para que funcione:

1. El botón Authorize espera:

- Username = tu email
- Password = tu contraseña

2. Tu endpoint /token debe aceptar form-data (no solo JSON)**3. Usa OAuth2PasswordRequestForm en lugar de tu schema UserLogin****Prueba rápida con curl del formato que usa Swagger:**

```
bash
```

```
# Swagger UI usa este formato (form-data)
curl -X POST "http://localhost:8000/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=test@example.com&password=123456"
```

¿Puedes probar a modificar tu endpoint /token para usar

OAuth2PasswordRequestForm y luego intentar autorizar desde Swagger con tu email y contraseña?

```
26, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
File "<frozen importlib._bootstrap>", line 1050, in _gcd_import
File "<frozen importlib._bootstrap>", line 1027, in _find_and_load
File "<frozen importlib._bootstrap>", line 1006, in
_find_and_load_unlocked
File "<frozen importlib._bootstrap>", line 688, in _load_unlocked
File "<frozen importlib._bootstrap_external>", line 883, in
exec_module
File "<frozen importlib._bootstrap>", line 241, in
_call_with_frames_removed
File "D:\projectoBanc\main.py", line 42, in <module>
    form_data: OAuth2PasswordRequestForm = Depends(),
NameError: name 'OAuth2PasswordRequestForm' is not defined
```

El error es porque no has importado OAuth2PasswordRequestForm . Aquí está la corrección:

Corrección: Importar OAuth2PasswordRequestForm

En tu `main.py` , agrega la importación faltante:

```
python

from fastapi import FastAPI, Depends, HTTPException, status, Form
from fastapi.security import OAuth2PasswordRequestForm # ← Esta es la línea que f
alta
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta # También agrega esto si no lo tienes

from app import crud, models, auth, schemas, utils
from app.database import engine, get_db, create_tables

# Crear tablas
create_tables()

app = FastAPI(
    title="API de Gestión Bancaria",
    description="API para procesar y gestionar transacciones bancarias con usuario
s",
    version="2.0.0"
)

# Configurar CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Endpoints públicos
@app.post("/registro", response_model=schemas.UserResponse)
def registrar_usuario(usuario: schemas.UserCreate, db: Session = Depends(get_db)):
    print("recibido", usuario)
    db_user = crud.get_user_by_email(db, email=usuario.email)
    if db_user:
        raise HTTPException(
            status_code=400,
```

```

        detail="El email ya está registrado"
    )
    return crud.create_user(db=db, user=usuario)

@app.post("/token", response_model=schemas.Token)
async def login_for_access_token(
    form_data: OAuth2PasswordRequestForm = Depends(), # ← Ahora sí funciona
    db: Session = Depends(get_db)
):
    """
    Endpoint para login:
    - Acepta application/x-www-form-urlencoded (para Swagger UI)
    """
    print("="*60)
    print("🔒 LOGIN ENDPOINT")
    print(f"Username recibido: {form_data.username}")
    print(f>Password recibida: {'*' * len(form_data.password)}")

    # Buscar usuario por email (username = email)
    user = db.query(models.Usuario).filter(
        models.Usuario.email == form_data.username
    ).first()

    if not user:
        print("❌ Usuario no encontrado")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Verificar contraseña
    if not auth.verify_password(form_data.password, user.hashed_password):
        print("❌ Contraseña incorrecta")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Crear token
    access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email}, expires_delta=access_token_expires
    )

```

```

    )

    print(f"✅ Login exitoso para: {user.email}")
    print(f"="*60)

    return {
        "access_token": access_token,
        "token_type": "bearer"
    }

# El resto de tus endpoints...

```

Si quieres mantener compatibilidad con JSON (para tus clientes):

```

python

@app.post("/token/json", response_model=schemas.Token)
def login_json(
    login_data: schemas.UserLogin,
    db: Session = Depends(get_db)
):
    """Endpoint que acepta JSON para tus clientes"""
    user = auth.authenticate_user(db, login_data.email, login_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
        )

    access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email}, expires_delta=access_token_expires
    )
    return {"access_token": access_token, "token_type": "bearer"}

# Y mantén el endpoint /token para Swagger

```

Para probar:

1. Reinicia tu servidor

2. Ve a `http://localhost:8000/docs`
3. Haz clic en "Authorize"
4. En username: pon tu **email** (ej: `test@example.com`)
5. En password: pon tu **contraseña** (ej: `123456`)
6. Haz clic en "Authorize"

¡Debería funcionar ahora! El problema era solo la importación faltante.

```
File "D:\proyectoBanc\main.py", line 42, in <module>
    form_data: OAuth2PasswordRequestForm = Depends(),
NameError: name 'OAuth2PasswordRequestForm' is not defined
```

El error persiste porque la importación no está bien hecha. Vamos a asegurarnos de importar correctamente. Aquí está la solución completa y probada:

✓ Solución: Importación correcta de OAuth2PasswordRequestForm

En tu `main.py` , las importaciones deben verse así:

```
python

# Importaciones correctas
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm # ← Esta es la línea clave
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta, datetime # Asegúrate de tener esto

from app import crud, models, auth, schemas, utils
from app.database import engine, get_db, create_tables

# Crear tablas
create_tables()

app = FastAPI(
    title="API de Gestión Bancaria",
    description="API para procesar y gestionar transacciones bancarias con usuario
```

```

s",
    version="2.0.0"
)

# Configurar CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Endpoints públicos
@app.post("/registro", response_model=schemas.UserResponse)
def registrar_usuario(usuario: schemas.UserCreate, db: Session = Depends(get_db)):
    print("recibido", usuario)
    db_user = crud.get_user_by_email(db, email=usuario.email)
    if db_user:
        raise HTTPException(
            status_code=400,
            detail="El email ya está registrado"
        )
    return crud.create_user(db=db, user=usuario)

# ⚠️ IMPORTANTE: Este es el endpoint que usa OAuth2PasswordRequestForm
@app.post("/token", response_model=schemas.Token)
async def login_for_access_token(
    form_data: OAuth2PasswordRequestForm = Depends(), # ← Aquí se usa
    db: Session = Depends(get_db)
):
    """
    Endpoint para login que funciona con Swagger UI
    - Usa application/x-www-form-urlencoded
    - Username = email del usuario
    - Password = contraseña
    """
    print("="*60)
    print("🔒 LOGIN ENDPOINT")
    print(f"Username recibido: {form_data.username}")
    print(f>Password recibida: {'*' * len(form_data.password)}")

    # Buscar usuario por email (username es el email)
    user = db.query(models.Usuario).filter(

```



```

        models.Usuario.email == form_data.username
    ).first()

    if not user:
        print("❌ Usuario no encontrado")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Verificar contraseña usando tu función de auth
    if not auth.verify_password(form_data.password, user.hashed_password):
        print("❌ Contraseña incorrecta")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Crear token
    access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email},
        expires_delta=access_token_expires
    )

    print(f"✅ Login exitoso para: {user.email}")
    print("="*60)

    return {
        "access_token": access_token,
        "token_type": "bearer"
    }

# (Opcional) Endpoint JSON para tus clientes
@app.post("/token/json", response_model=schemas.Token)
def login_json(
    login_data: schemas.UserLogin,
    db: Session = Depends(get_db)
):
    """Endpoint alternativo que acepta JSON"""
    user = auth.authenticate_user(db, login_data.email, login_data.password)
    if not user:

```

```

        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email o contraseña incorrectos",
        )

    access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email},
        expires_delta=access_token_expires
    )
    return {
        "access_token": access_token,
        "token_type": "bearer"
    }

# El resto de tus endpoints...
@app.post("/procesar-texto")
def procesar_texto_transaccion(texto: str, db: Session = Depends(get_db)):
    # ... tu código existente ...
    pass

# ... etc ...

```

Si aún tienes error, verifica:

1. ¿FastAPI está actualizado?

```
bash
```

```
pip install --upgrade fastapi
```

2. Verifica que OAuth2PasswordRequestForm existe:

```
python
```

```

# Crea un archivo test.py temporal
from fastapi.security import OAuth2PasswordRequestForm
print("Importación exitosa!")
print(dir(OAuth2PasswordRequestForm))

```

3. Reinicia el servidor completamente:

```
bash
```

```
# Detén el servidor (Ctrl+C) y vuelve a iniciar  
python main.py
```

4. Para probar manualmente el endpoint /token:

```
bash
```

```
curl -X POST "http://localhost:8000/token" \  
  -H "Content-Type: application/x-www-form-urlencoded" \  
  -d "username=test@example.com&password=123456"
```



Instrucciones para usar Swagger UI:

1. Ve a `http://localhost:8000/docs`
2. Haz clic en el botón **"Authorize"** (arriba a la derecha)
3. En el campo **"username"**: introduce tu **EMAIL** (ej: `test@example.com`)
4. En el campo **"password"**: introduce tu **CONTRASEÑA** (ej: `123456`)
5. Haz clic en **"Authorize"**
6. Verás el mensaje "Authorized"
7. Cierra el diálogo
8. Ahora puedes probar cualquier endpoint con candado 

El error "name 'OAuth2PasswordRequestForm' is not defined" significa que Python no encuentra esa clase. Asegúrate de que la línea

`from fastapi.security import OAuth2PasswordRequestForm` esté exactamente así y que no tenga errores tipográficos.

Username recibido: lazaro1@example.com

Password recibida: *****

Lachy05385*

\$argon2id\$v=19\$m=102400,t=2,p=8\$XHgvBQBAKEUIwZiTco4xJg

\$vIhyR/+P4WptqNXTF007MrQxuy21f4MeBF0LyEKoGDU

✗ Contraseña incorrecta

INFO: 127.0.0.1:64737 - "POST /token HTTP/1.1" 401
Unauthorized

¡Excelente! Ya tenemos el log y podemos ver el problema exactamente. Observa esto:

text

Password recibida: *****

Lachy05385* ← ¡Aquí está el problema!

\$argon2id\$v=19\$m=102400,t=2,p=8\$xHgvBQBAKEUIwZiTco4xJg\$IhyR/+P4WptqNXTF007MrQxuy2
1f4MeBF0LyEKoGDU

La línea `Lachy05385*` se está imprimiendo antes del hash, lo que indica que hay un `print(plain_password)` en tu código que está mostrando la contraseña en texto plano, pero luego la verificación falla.

El problema: La contraseña tiene caracteres especiales

Tu contraseña `Lachy05385*` tiene un asterisco `*`. En Argon2, los caracteres especiales pueden causar problemas si no se manejan correctamente. Vamos a solucionarlo:

1. Primero, verifica tu función `verify_password` en `auth.py`:

python

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    """
    Verificar contraseña usando Argon2.
    """
    try:
        print(f"🔒 Verificando password...")
        print(f" Plain: '{plain_password}'") # Las comillas muestran espacios/ca
racteres
        print(f" Hash: {hashed_password[:50]}...")
```

```

# Asegurar que la contraseña sea string y esté limpia
plain_password = str(plain_password).strip()

result = pwd_context.verify(plain_password, hashed_password)
print(f" Resultado: {result}")
return result
except Exception as e:
    print(f"❌ Error verifying: {str(e)}")
    # Imprime el tipo de error específico
    import traceback
    traceback.print_exc()
    return False

```

2. Crea un script de prueba directa test_password.py :

python

```

# test_password.py
from app.auth import verify_password, get_password_hash
from app.database import SessionLocal
from app import models

def test_password_direct():
    """Prueba la verificación de contraseña directamente"""

    # Conéctate a la DB
    db = SessionLocal()

    # Busca tu usuario
    user = db.query(models.Usuario).filter(models.Usuario.email == "lazar01@exampl
e.com").first()

    if not user:
        print(f"❌ Usuario no encontrado")
        return

    print("="*60)
    print("🔍 TEST DE VERIFICACIÓN DIRECTA")
    print("="*60)
    print(f"Usuario: {user.email}")
    print(f"Hash en DB: {user.hashed_password}")

    # La contraseña que intentas usar

```

```

test_password = "Lachy05385*"

print(f"\n🔑 Probando con: '{test_password}'")

# Prueba 1: Verificación directa
result1 = verify_password(test_password, user.hashed_password)
print(f"Verificación directa: {result1}")

# Prueba 2: Verificación con diferentes codificaciones
print("\n🔑 Probando variaciones:")
variations = [
    test_password,
    test_password.strip(),
    test_password.encode().decode(),
    "Lachy05385*", # Exactamente igual
]

for i, pwd in enumerate(variations):
    result = verify_password(pwd, user.hashed_password)
    print(f"Variación {i+1}: '{pwd}' -> {result}")

# Prueba 3: Rehashear la contraseña para ver si coincide
print("\n🔑 Rehasheando la contraseña:")
new_hash = get_password_hash(test_password)
print(f"Nuevo hash: {new_hash}")

# Comparar con el hash de la DB
result3 = verify_password(test_password, new_hash)
print(f"Verificación con nuevo hash: {result3}")

db.close()

if __name__ == "__main__":
    test_password_direct()

```

3. Solución: Actualiza el usuario con una nueva contraseña

python

```

# reset_password.py
from app.auth import get_password_hash
from app.database import SessionLocal
from app import models

```

```
def reset_user_password():  
    """Resetea la contraseña del usuario"""  
  
    db = SessionLocal()  
  
    email = "lazarol@example.com"  
    new_password = "Lachy05385*" # La misma contraseña  
  
    # Buscar usuario  
    user = db.query(models.Usuario).filter(models.Usuario.email == email).first()  
  
    if not user:  
        print(f"❌ Usuario {email} no encontrado")  
        return  
  
    print(f"✅ Usuario encontrado: {user.email}")  
    print(f"Hash actual: {user.hashed_password}")  
  
    # Generar nuevo hash  
    new_hash = get_password_hash(new_password)  
    print(f"Nuevo hash: {new_hash}")  
  
    # Actualizar  
    user.hashed_password = new_hash  
    db.commit()  
  
    print(f"✅ Contraseña actualizada!")  
  
    # Verificar  
    from app.auth import verify_password  
    if verify_password(new_password, user.hashed_password):  
        print(f"✅ Verificación exitosa con la nueva contraseña")  
    else:  
        print(f"❌ Algo salió mal con la verificación")  
  
    db.close()  
  
if __name__ == "__main__":  
    reset_user_password()
```

4. Solución alternativa: Usa una contraseña más simple para probar

bash

```
# Registra un nuevo usuario con contraseña simple
curl -X POST "http://localhost:8000/registro" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test2@example.com",
    "nombre": "Test",
    "apellido": "User",
    "password": "123456"
  }'

# Luego prueba Login
curl -X POST "http://localhost:8000/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=test2@example.com&password=123456"
```

5. Si el problema persiste, modifica tu endpoint /token para debuggear:

python

```
@app.post("/token", response_model=schemas.Token)
async def login_for_access_token(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db)
):
    print("="*60)
    print("🔑 LOGIN ENDPOINT")
    print(f"Username recibido: '{form_data.username}'")
    print(f>Password recibida: '{form_data.password}'")
    print(f"Longitud password: {len(form_data.password)}")
    print(f"Caracteres: {[ord(c) for c in form_data.password]}")

    # Buscar usuario
    user = db.query(models.Usuario).filter(
        models.Usuario.email == form_data.username
    ).first()
```



```

if not user:
    print("❌ Usuario no encontrado")
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Email o contraseña incorrectos",
    )

print(f"✅ Usuario encontrado")
print(f"Hash en DB: {user.hashed_password}")

# Verificar contraseña
if not auth.verify_password(form_data.password, user.hashed_password):
    print("❌ Contraseña incorrecta")

    # Debug: Intenta con la contraseña exacta
    print("\n🔗 DEBUG: Probando con la contraseña exacta")
    direct_result = auth.pwd_context.verify(
        form_data.password,
        user.hashed_password
    )
    print(f"Verificación directa con pwd_context: {direct_result}")

    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Email o contraseña incorrectos",
    )

# Crear token
access_token = auth.create_access_token(data={"sub": user.email})

print(f"✅ Login exitoso!")
print("="*60)

return {
    "access_token": access_token,
    "token_type": "bearer"
}

```

Ejecuta primero el script `reset_password.py` para resetear la contraseña, y luego prueba el login nuevamente. Si el problema persiste, ejecuta `test_password.py` y comparte los resultados para ver exactamente qué está fallando.

ya autentica correctamente, el objetivo de esta app es el procesamiento de transferencias, o sea cuando un usuario se autentica, envia automatica o manualmente las transacciones recibidas o de interes, la idea seria hacer una app android en el telefono y reenvie automaticamente las recibidas de su banco, pero de momento lo que quiero resolver es que las transacciones vayan asociadas al usuario, porque en cualquier momento del dia el usuario puede verificar sus transacciones, y ve que los endpoints no solicita codigo de usuario

¡Excelente pregunta! Tienes toda la razón. Actualmente, cuando un usuario autenticado hace una petición, ya tenemos su información en `current_user`. Veamos cómo asociar las transacciones al usuario correctamente:

El usuario ya está en `current_user`

Cuando usas `Depends(auth.get_current_active_user)`, ya tienes el objeto `current_user` con toda la información. **No necesitas enviar el ID manualmente.**

1. Revisa tu modelo Transaccion en `models.py`:

python

```
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, Boolean, Text
from sqlalchemy.orm import relationship
from datetime import datetime
from app.database import Base

class Transaccion(Base):
    __tablename__ = "transacciones"

    id = Column(Integer, primary_key=True, index=True)
    usuario_id = Column(Integer, ForeignKey("usuarios.id"), nullable=False) # ← C
    # Lave foránea
    numero_transaccion = Column(String, unique=True, index=True)
    fecha = Column(DateTime, default=datetime.utcnow)
    monto = Column(Float, nullable=False)
    tipo = Column(String) # ENVIADA, RECIBIDA
    estado = Column(String, default="PENDIENTE") # PENDIENTE, CONFIRMADA
```

```

banco_ordenante = Column(String, nullable=True)
banco_beneficiario = Column(String, nullable=True)
cuenta_ordenante = Column(String, nullable=True)
cuenta_beneficiario = Column(String, nullable=True)
concepto = Column(Text, nullable=True)
descripcion = Column(String, nullable=True)
created_at = Column(DateTime, default=datetime.utcnow)

# Relación con usuario
usuario = relationship("Usuario", back_populates="transacciones")

```

2. En schemas.py , asegúrate que TransaccionResponse incluya el usuario:

python

```

class TransaccionBase(BaseModel):
    monto: float
    tipo: str
    descripcion: Optional[str] = None
    fecha: Optional[datetime] = None
    # ... otros campos

class TransaccionCreate(TransaccionBase):
    pass # No incluye usuario_id porque se toma del token

class TransaccionResponse(TransaccionBase):
    id: int
    usuario_id: int # ← Importante incluirlo aquí
    numero_transaccion: Optional[str] = None
    estado: str
    created_at: datetime

    # Opcional: incluir datos del usuario
    usuario: Optional[UserResponse] = None

class Config:
    from_attributes = True

```

3. En crud.py , modifica las funciones para usar usuario_id :

python

```
from sqlalchemy.orm import Session
from app import models, schemas
from app.utils import parse_transaccion_texto

def create_transaccion(db: Session, transaccion: schemas.TransaccionCreate, usuario_id: int):
    """
    Crear una transacción manualmente asociada al usuario
    """
    db_transaccion = models.Transaccion(
        **transaccion.dict(),
        usuario_id=usuario_id, # ← Asociar al usuario
        estado="PENDIENTE"
    )
    db.add(db_transaccion)
    db.commit()
    db.refresh(db_transaccion)
    return db_transaccion

def parse_and_create_transaccion(db: Session, texto: str, tipo: str, usuario_id: int, descripcion: Optional[str] = None):
    """
    Crear transacción desde texto asociada al usuario
    """
    # Parsear el texto
    datos = parse_transaccion_texto(texto)

    # Crear transacción
    db_transaccion = models.Transaccion(
        usuario_id=usuario_id, # ← Asociar al usuario
        numero_transaccion=datos["numero_transaccion"],
        monto=datos["monto"],
        tipo=tipo,
        banco_ordenante=datos.get("banco_ordenante"),
        banco_beneficiario=datos.get("banco_beneficiario"),
        cuenta_ordenante=datos.get("ordenante"),
        cuenta_beneficiario=datos.get("beneficiario"),
        concepto=datos.get("concepto"),
        descripcion=descripcion,
        estado="PENDIENTE"
    )
```

```

db.add(db_transaccion)
db.commit()
db.refresh(db_transaccion)
return db_transaccion

def get_transacciones_usuario(db: Session, usuario_id: int, skip: int = 0, limit:
int = 100, **filtros):
    """
    Obtener transacciones de un usuario específico con filtros
    """
    query = db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id
    )

    # Aplicar filtros adicionales
    if filtros.get('tipo'):
        query = query.filter(models.Transaccion.tipo == filtros['tipo'])
    if filtros.get('estado'):
        query = query.filter(models.Transaccion.estado == filtros['estado'])
    if filtros.get('banco'):
        query = query.filter(
            (models.Transaccion.banco_ordenante == filtros['banco']) |
            (models.Transaccion.banco_beneficiario == filtros['banco'])
        )

    # Ordenar por fecha descendente (más recientes primero)
    query = query.order_by(models.Transaccion.fecha.desc())

    return query.offset(skip).limit(limit).all()

def get_transaccion(db: Session, transaccion_id: int, usuario_id: int):
    """
    Obtener una transacción específica verificando que pertenezca al usuario
    """
    return db.query(models.Transaccion).filter(
        models.Transaccion.id == transaccion_id,
        models.Transaccion.usuario_id == usuario_id
    ).first()

def update_transaccion(db: Session, transaccion_id: int, usuario_id: int, transacc
ion_update: schemas.TransaccionUpdate):
    """
    Actualizar transacción verificando propiedad
    """

```

```
db_transaccion = get_transaccion(db, transaccion_id, usuario_id)
if not db_transaccion:
    return None

for key, value in transaccion_update.dict(exclude_unset=True).items():
    setattr(db_transaccion, key, value)

db.commit()
db.refresh(db_transaccion)
return db_transaccion

def confirmar_transaccion(db: Session, transaccion_id: int, usuario_id: int):
    """
    Confirmar transacción (solo para transacciones RECIBIDAS)
    """
    db_transaccion = get_transaccion(db, transaccion_id, usuario_id)
    if not db_transaccion:
        return None

    if db_transaccion.tipo == "RECIBIDA":
        db_transaccion.estado = "CONFIRMADA"
        db.commit()
        db.refresh(db_transaccion)

    return db_transaccion

def get_resumen_transacciones(db: Session, usuario_id: int, fecha_inicio: str = None, fecha_fin: str = None):
    """
    Obtener resumen de transacciones del usuario
    """
    query = db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id
    )

    # Aplicar filtros de fecha si existen
    # ... lógica de fechas ...

    transacciones = query.all()

    # Calcular resúmenes
    total_enviadas = sum(t.monto for t in transacciones if t.tipo == "ENVIADA")
    total_recibidas = sum(t.monto for t in transacciones if t.tipo == "RECIBIDA")
    pendientes = len([t for t in transacciones if t.estado == "PENDIENTE"])
```

```

return {
    "total_transacciones": len(transacciones),
    "total_enviadas": total_enviadas,
    "total_recibidas": total_recibidas,
    "balance": total_recibidas - total_enviadas,
    "pendientes": pendientes,
    "transacciones_recientes": transacciones[:10] # Últimas 10
}

```

4. En main.py , tus endpoints ya están bien porque usan `current_user.id` :

python

```

@app.post("/transacciones/parsear", response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario = Depends(auth.get_current_active_user), # ← Aqu
    í está el usuario
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    # Verificar si la transacción ya existe para ESTE usuario
    datos = utils.parse_transaccion_texto(transaccion_req.texto)
    existente = db.query(models.Transaccion).filter(
        models.Transaccion.numero_transaccion == datos["numero_transaccion"],
        models.Transaccion.usuario_id == current_user.id # ← Filtro por usuario
    ).first()

    if existente:
        raise HTTPException(status_code=400, detail="Esta transacción ya existe")

    # Crear transacción para ESTE usuario
    return crud.parse_and_create_transaccion(
        db=db,
        texto=transaccion_req.texto,
        tipo=transaccion_req.tipo,
        usuario_id=current_user.id, # ← Pasar el ID del usuario actual
        descripcion=transaccion_req.descripcion
    )

```

```

@app.get("/transacciones/", response_model=List[schemas.TransaccionResponse])
def listar_transacciones(
    skip: int = 0,
    limit: int = 100,
    tipo: Optional[str] = None,
    estado: Optional[str] = None,
    banco: Optional[str] = None,
    current_user: models.Usuario = Depends(auth.get_current_active_user), # ← Usuario autenticado
    db: Session = Depends(get_db)
):
    """Listar transacciones del usuario con filtros"""
    # Automáticamente filtra por current_user.id
    return crud.get_transacciones_usuario(
        db=db,
        usuario_id=current_user.id, # ← Solo sus transacciones
        skip=skip,
        limit=limit,
        tipo=tipo,
        estado=estado,
        banco=banco
    )

```

5. Para probar que funciona, crea un script de prueba:

python

```

# test_transacciones_usuario.py
import requests
import json

BASE_URL = "http://localhost:8000"

def test_flujo_completo():
    """Prueba el flujo completo de transacciones por usuario"""

    # 1. Login con usuario existente
    print("\n1 Haciendo login...")
    login_data = {
        "username": "lazar01@example.com", # o el email que uses
        "password": "Lachy05385*"
    }

```



```

response = requests.post(
    f"{BASE_URL}/token",
    data=login_data, # Importante: data, no json
    headers={"Content-Type": "application/x-www-form-urlencoded"})

if response.status_code != 200:
    print(f"❌ Login falló: {response.json()}")
    return

token = response.json()["access_token"]
print(f"✅ Login exitoso. Token obtenido")

headers = {
    "Authorization": f"Bearer {token}",
    "Content-Type": "application/json"
}

# 2. Ver perfil
print("\n2 Verificando perfil...")
response = requests.get(f"{BASE_URL}/perfil", headers=headers)
perfil = response.json()
print(f"Usuario: {perfil['nombre']} {perfil['apellido']} (ID: {perfil['id']})")

# 3. Crear una transacción
print("\n3 Creando transacción...")
transaccion_data = {
    "monto": 150.75,
    "tipo": "RECIBIDA",
    "descripcion": "Pago de prueba"
}

response = requests.post(
    f"{BASE_URL}/transacciones/",
    headers=headers,
    json=transaccion_data
)

if response.status_code == 200:
    transaccion = response.json()
    print(f"✅ Transacción creada con ID: {transaccion['id']}")
    print(f"    Monto: {transaccion['monto']}")
    print(f"    Usuario ID: {transaccion['usuario_id']}") # Debe coincidir con

```

```

perfil['id']
else:
    print(f"❌ Error: {response.json()}")

# 4. Listar transacciones
print("\n4 Listando transacciones...")
response = requests.get(f"{BASE_URL}/transacciones/", headers=headers)

if response.status_code == 200:
    transacciones = response.json()
    print(f"Total transacciones: {len(transacciones)}")
    for t in transacciones[:3]: # Mostrar primeras 3
        print(f" - ID: {t['id']} | Monto: {t['monto']} | Tipo: {t['tipo']}")
else:
    print(f"❌ Error: {response.json()}")

if __name__ == "__main__":
    test_flujo_completo()

```

Para la app Android:

Cuando hagas la app Android, simplemente:

1. **Guarda el token** después del login
2. **Envía el token** en cada petición
3. **No necesitas enviar el ID del usuario** - el backend lo obtiene del token

java

```

// Ejemplo en Android (Kotlin)
val token = "eyJhbGciOiJIUzI1NiIs..."
val call = apiService.getTransacciones("Bearer $token")

// El backend automáticamente sabe qué usuario es
// gracias al token

```

¡Tu API ya está correctamente diseñada! Cada usuario solo ve y gestiona SUS propias transacciones gracias al `current_user` que se obtiene del token JWT.

```
class Transaccion(Base):
    __tablename__ = "transacciones"

    id = Column(Integer, primary_key=True, index=True)

    # Datos del usuario
    usuario_id = Column(Integer, ForeignKey("usuarios.id"),
nullable=False)

    # Datos de la transacción
    banco = Column(String, nullable=False)
    fecha = Column(String, nullable=False)
    beneficiario = Column(String, nullable=False)
    ordenante = Column(String, nullable=False)
    monto = Column(Float, nullable=False)
    moneda = Column(String, nullable=False)
    numero_transaccion = Column(String, unique=True,
nullable=False, index=True)

    # Clasificación del usuario
    tipo = Column(Enum(TipoTransaccion), nullable=False)
    estado = Column(Enum(EstadoTransaccion),
default=EstadoTransaccion.PENDIENTE)

    # Metadatos
    descripcion = Column(String, default="")
    etiquetas = Column(String, default="") # JSON string de
etiquetas
    fecha_procesamiento = Column(DateTime(timezone=True),
server_default=func.now())
    fecha_confirmacion = Column(DateTime(timezone=True),
nullable=True)

    # Relación
    usuario = relationship("Usuario",
back_populates="transacciones")
```

¡Perfecto! Tu modelo `Transaccion` ya tiene la clave foránea `usuario_id` correctamente configurada. Ahora vamos a asegurarnos de que todos los

endpoints de transacciones utilicen correctamente el `usuario_id` del usuario autenticado.

Actualiza tu `crud.py` para manejar correctamente las transacciones por usuario:

python

```
from sqlalchemy.orm import Session
from sqlalchemy import func
from app import models, schemas
from app.utils import parse_transaccion_texto
from datetime import datetime
import json

def create_transaccion(db: Session, transaccion: schemas.TransaccionCreate, usuario_id: int):
    """
    Crear una transacción manualmente asociada al usuario
    """
    db_transaccion = models.Transaccion(
        usuario_id=usuario_id,
        banco=transaccion.banco,
        fecha=transaccion.fecha,
        beneficiario=transaccion.beneficiario,
        ordenante=transaccion.ordenante,
        monto=transaccion.monto,
        moneda=transaccion.moneda,
        numero_transaccion=transaccion.numero_transaccion,
        tipo=transaccion.tipo,
        estado=transaccion.estado or models.EstadoTransaccion.PENDIENTE,
        descripcion=transaccion.descripcion or "",
        etiquetas=json.dumps(transaccion.etiquetas) if transaccion.etiquetas else "[]"
    )

    db.add(db_transaccion)
    db.commit()
    db.refresh(db_transaccion)
    return db_transaccion

def parse_and_create_transaccion(db: Session, texto: str, tipo: str, usuario_id: i
```

```

nt, descripcion: Optional[str] = None):
    """
    Crear transacción desde texto asociada al usuario
    """
    # Parsear el texto
    datos = parse_transaccion_texto(texto)

    # Crear transacción
    db_transaccion = models.Transaccion(
        usuario_id=usuario_id,
        banco=datos.get("banco", ""),
        fecha=datos.get("fecha", datetime.now().strftime("%Y-%m-%d")),
        beneficiario=datos.get("beneficiario", ""),
        ordenante=datos.get("ordenante", ""),
        monto=datos.get("monto", 0),
        moneda=datos.get("moneda", "EUR"),
        numero_transaccion=datos["numero_transaccion"],
        tipo=tipo,
        estado=models.EstadoTransaccion.PENDIENTE,
        descripcion=descripcion or datos.get("concepto", ""),
        etiquetas="[]"
    )

    db.add(db_transaccion)
    db.commit()
    db.refresh(db_transaccion)
    return db_transaccion


def get_transacciones_usuario(db: Session, usuario_id: int, skip: int = 0, limit:
int = 100,
                                tipo: Optional[str] = None, estado: Optional[str] =
None,
                                banco: Optional[str] = None):
    """
    Obtener transacciones de un usuario específico con filtros
    """
    query = db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id
    )

    # Aplicar filtros
    if tipo:
        query = query.filter(models.Transaccion.tipo == tipo)
    if estado:

```

```
    query = query.filter(models.Transaccion.estado == estado)
if banco:
    query = query.filter(models.Transaccion.banco == banco)

# Ordenar por fecha_procesamiento descendente (más recientes primero)
query = query.order_by(models.Transaccion.fecha_procesamiento.desc())

return query.offset(skip).limit(limit).all()

def get_transaccion(db: Session, transaccion_id: int, usuario_id: int):
    """
    Obtener una transacción específica verificando que pertenezca al usuario
    """
    return db.query(models.Transaccion).filter(
        models.Transaccion.id == transaccion_id,
        models.Transaccion.usuario_id == usuario_id
    ).first()

def update_transaccion(db: Session, transaccion_id: int, usuario_id: int, transaccion_update: schemas.TransaccionUpdate):
    """
    Actualizar transacción verificando propiedad
    """
    db_transaccion = get_transaccion(db, transaccion_id, usuario_id)
    if not db_transaccion:
        return None

    # Actualizar solo los campos proporcionados
    update_data = transaccion_update.dict(exclude_unset=True)

    # Manejar etiquetas como JSON si vienen
    if 'etiquetas' in update_data and update_data['etiquetas']:
        update_data['etiquetas'] = json.dumps(update_data['etiquetas'])

    for key, value in update_data.items():
        setattr(db_transaccion, key, value)

    db.commit()
    db.refresh(db_transaccion)
    return db_transaccion

def confirmar_transaccion(db: Session, transaccion_id: int, usuario_id: int):
    """
    Confirmar una transacción recibida
    """
```

```

"""
db_transaccion = get_transaccion(db, transaccion_id, usuario_id)
if not db_transaccion:
    return None

# Solo se pueden confirmar transacciones RECIBIDAS
if db_transaccion.tipo == models.TipoTransaccion.RECIBIDA:
    db_transaccion.estado = models.EstadoTransaccion.CONFIRMADA
    db_transaccion.fecha_confirmacion = datetime.now()
    db.commit()
    db.refresh(db_transaccion)

return db_transaccion

def get_resumen_transacciones(db: Session, usuario_id: int):
    """
    Obtener resumen de transacciones del usuario
    """
    transacciones = db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id
    ).all()

    # Calcular resúmenes
    total_enviadas = sum(t.monto for t in transacciones if t.tipo == models.TipoTransaccion.ENVIADA)
    total_recibidas = sum(t.monto for t in transacciones if t.tipo == models.TipoTransaccion.RECIBIDA)
    pendientes = len([t for t in transacciones if t.estado == models.EstadoTransaccion.PENDIENTE])
    confirmadas = len([t for t in transacciones if t.estado == models.EstadoTransaccion.CONFIRMADA])

    # Agrupar por banco
    bancos = {}
    for t in transacciones:
        if t.banco not in bancos:
            bancos[t.banco] = {"cantidad": 0, "monto_total": 0}
        bancos[t.banco]["cantidad"] += 1
        bancos[t.banco]["monto_total"] += t.monto

    return {
        "total_transacciones": len(transacciones),
        "total_enviadas": total_enviadas,
        "total_recibidas": total_recibidas,
    }

```

```

        "balance": total_recibidas - total_enviadas,
        "pendientes": pendientes,
        "confirmadas": confirmadas,
        "bancos": bancos,
        "transacciones_recientes": transacciones[:5] # Últimas 5
    }

def buscar_transacciones(db: Session, usuario_id: int, query: str):
    """
    Buscar transacciones por texto (beneficiario, ordenante, concepto, etc)
    """
    return db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id,
        (models.Transaccion.beneficiario.ilike(f"%{query}%") |
         models.Transaccion.ordenante.ilike(f"%{query}%") |
         models.Transaccion.descripcion.ilike(f"%{query}%") |
         models.Transaccion.numero_transaccion.ilike(f"%{query}%"))
    ).order_by(models.Transaccion.fecha_procesamiento.desc()).all()

```



Actualiza tus schemas para manejar correctamente los datos:

python

```

from pydantic import BaseModel, Field, ConfigDict
from typing import Optional, List
from datetime import datetime
from enum import Enum

# Enums
class TipoTransaccion(str, Enum):
    ENVIADA = "ENVIADA"
    RECIBIDA = "RECIBIDA"

class EstadoTransaccion(str, Enum):
    PENDIENTE = "PENDIENTE"
    CONFIRMADA = "CONFIRMADA"
    RECHAZADA = "RECHAZADA"

# Schemas de Transacción
class TransaccionBase(BaseModel):
    banco: str

```



```
    fecha: str
    beneficiario: str
    ordenante: str
    monto: float
    moneda: str
    numero_transaccion: str
    tipo: TipoTransaccion
    estado: Optional[EstadoTransaccion] = EstadoTransaccion.PENDIENTE
    descripcion: Optional[str] = ""
    etiquetas: Optional[List[str]] = []

class TransaccionCreate(TransaccionBase):
    pass

class TransaccionUpdate(BaseModel):
    tipo: Optional[TipoTransaccion] = None
    estado: Optional[EstadoTransaccion] = None
    descripcion: Optional[str] = None
    etiquetas: Optional[List[str]] = None

class TransaccionResponse(TransaccionBase):
    id: int
    usuario_id: int
    fecha_procesamiento: datetime
    fecha_confirmacion: Optional[datetime] = None

    # Parsear etiquetas de JSON a Lista
    @model_validator(mode='before')
    @classmethod
    def parse_etiquetas(cls, values):
        if isinstance(values, dict) and 'etiquetas' in values:
            if isinstance(values['etiquetas'], str):
                try:
                    values['etiquetas'] = json.loads(values['etiquetas'])
                except:
                    values['etiquetas'] = []
        return values

    model_config = ConfigDict(from_attributes=True)

class TransaccionParseRequest(BaseModel):
    texto: str
    tipo: TipoTransaccion
    descripcion: Optional[str] = None
```

```
# Resumen
class ResumenTransacciones(BaseModel):
    total_transacciones: int
    total_enviadas: float
    total_recibidas: float
    balance: float
    pendientes: int
    confirmadas: int
    bancos: dict
    transacciones_recientes: List[TransaccionResponse]
```

 **Y asegúrate que en main.py tus endpoints usen correctamente el usuario:**

python

```
@app.post("/transacciones/", response_model=schemas.TransaccionResponse)
def crear_transaccion(
    transaccion: schemas.TransaccionCreate,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción manualmente (asociada al usuario actual)"""
    # Verificar si ya existe para este usuario
    existente = db.query(models.Transaccion).filter(
        models.Transaccion.numero_transaccion == transaccion.numero_transaccion,
        models.Transaccion.usuario_id == current_user.id
    ).first()

    if existente:
        raise HTTPException(
            status_code=400,
            detail=f"La transacción {transaccion.numero_transaccion} ya existe"
        )

    return crud.create_transaccion(
        db=db,
        transaccion=transaccion,
        usuario_id=current_user.id
    )
```

```
@app.get("/transacciones/", response_model=List[schemas.TransaccionResponse])
def listar_transacciones(
    skip: int = 0,
    limit: int = 100,
    tipo: Optional[schemas.TipoTransaccion] = None,
    estado: Optional[schemas.EstadoTransaccion] = None,
    banco: Optional[str] = None,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Listar SOLO las transacciones del usuario actual"""
    return crud.get_transacciones_usuario(
        db=db,
        usuario_id=current_user.id,
        skip=skip,
        limit=limit,
        tipo=tipo,
        estado=estado,
        banco=banco
    )

@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones del usuario actual"""
    return crud.buscar_transacciones(db, current_user.id, q)

@app.get("/transacciones/{transaccion_id}", response_model=schemas.TransaccionResponse)
def obtener_transaccion(
    transaccion_id: int,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Obtener una transacción específica (solo si pertenece al usuario)"""
    transaccion = crud.get_transaccion(db, transaccion_id, current_user.id)
    if not transaccion:
        raise HTTPException(status_code=404, detail="Transacción no encontrada")
    return transaccion
```

Resumen:

1. **El usuario autenticado** está disponible en `current_user`
2. **Todas las transacciones** se crean con `usuario_id = current_user.id`
3. **Todas las consultas** filtran por `usuario_id = current_user.id`
4. **Nunca necesitas enviar** el ID del usuario manualmente

Esto asegura que cada usuario solo ve y gestiona SUS propias transacciones.

¿Necesitas ayuda con algún endpoint en específico?

te voy a subir mi schema de transacciones porque tiene una logica particular que no veo que hayas tenido en cuenta "class TransaccionBase(BaseModel):

```

    banco: str = Field(..., min_length=2, max_length=100,
                        description="Nombre del banco")
    fecha: str = Field(..., description="Fecha en formato
DD/MM/YYYY")
    beneficiario: str = Field(..., min_length=4, max_length=50,
                              description="Número de cuenta del beneficiario")
    ordenante: str = Field(..., min_length=4, max_length=50,
                            description="Número de cuenta del ordenante")
    monto: float = Field(..., gt=0, le=10000000, description="Monto
de la transacción (0-10M)")
    moneda: str = Field(..., min_length=1, max_length=10,
                        description="Moneda (CUP, USD, EUR, MLC)")
    numero_transaccion: str = Field(..., min_length=4,
max_length=50,
                                description="Número único de transacción")

```

```
@field_validator('fecha')
```

```
@classmethod
```

```
def validar_fecha(cls, v: str) -> str:
```

```
    """Validar formato DD/MM/YYYY"""
```

```
    v = v.strip()
```

```
    # Validar formato básico
```

```
    pattern = r"^\d{2}/\d{2}/\d{4}$"
```

```
    if not re.match(pattern, v):
```

```
        raise ValueError('Formato de fecha inválido. Use
```

DD/MM/YYYY')

```
# Validar fecha básica
try:
    day, month, year = map(int, v.split('/'))
    if not (1 <= day <= 31 and 1 <= month <= 12 and year >=
2000):
        raise ValueError('Fecha inválida')
except ValueError:
    raise ValueError('Fecha inválida')

return v
```

```
@field_validator('monto')
@classmethod
def validar_monto(cls, v: float) -> float:
    """Validar monto"""
    if v <= 0:
        raise ValueError('El monto debe ser mayor que cero')
    return round(v, 2)
```

```
@field_validator('moneda')
@classmethod
def validar_moneda(cls, v: str) -> str:
    """Validar moneda"""
    monedas_validas = ['CUP', 'USD', 'EUR', 'MLC']
    v_upper = v.strip().upper()

    if v_upper not in monedas_validas:
        raise ValueError(f'Moneda no válida. Use: {"",
".join(monedas_validas)}')

    return v_upper
```

```
class TransaccionCreate(TransaccionBase):
    tipo: TipoTransaccion = Field(..., description="Tipo de
transacción")
    descripcion: Optional[str] = Field(default="", max_length=200,
description="Descripción opcional")
    etiquetas: Optional[List[str]] = Field(default_factory=list,
```

```
description="Lista de etiquetas")
```

```
@field_validator('descripcion')
@classmethod
def validar_descripcion(cls, v: Optional[str]) -> str:
    """Validar descripción"""
    if v is None:
        return ""
    v = v.strip()
    if len(v) > 200:
        raise ValueError('La descripción no puede exceder 200
caracteres')
    return v

class TransaccionParseRequest(BaseModel):
    texto: str = Field(..., min_length=10, max_length=2000,
        description="Texto de la transacción bancaria")
    tipo: TipoTransaccion = Field(..., description="Tipo de
transacción")
    descripcion: Optional[str] = Field(default="", max_length=200,
        description="Descripción opcional")

class TransaccionUpdate(BaseModel):
    tipo: Optional[TipoTransaccion] = Field(None, description="Tipo
de transacción")
    estado: Optional[EstadoTransaccion] = Field(None,
description="Estado")
    descripcion: Optional[str] = Field(None, max_length=200,
description="Descripción")
    etiquetas: Optional[List[str]] = Field(None,
description="Etiquetas")
```

```
@field_validator('descripcion')
@classmethod
def validar_descripcion(cls, v: Optional[str]) -> Optional[str]:
    """Validar descripción"""
    if v is None:
        return None
    v = v.strip()
    if len(v) > 200:
```

```

        raise ValueError('La descripción no puede exceder 200
caracteres')
    return v if v else None

class TransaccionResponse(TransaccionBase):
    id: int
    usuario_id: int
    tipo: TipoTransaccion
    estado: EstadoTransaccion
    descripcion: str
    etiquetas: List[str]
    fecha_procesamiento: datetime
    fecha_confirmacion: Optional[datetime] = None

    model_config = ConfigDict(from_attributes=True)"

```

¡Excelente! Ya veo tu schema completo con todas las validaciones. Tienes razón, es más específico. Vamos a integrarlo correctamente con tu lógica existente:

Actualiza crud.py para usar tu schema completo:

python

```

from sqlalchemy.orm import Session
from sqlalchemy import func
from app import models, schemas
from app.utils import parse_transaccion_texto
from datetime import datetime
import json
import re
from typing import List, Optional

def create_transaccion(db: Session, transaccion: schemas.TransaccionCreate, usuario_id: int):
    """
    Crear una transacción manualmente asociada al usuario
    Usando tu schema con todas las validaciones
    """
    # Verificar si ya existe para este usuario
    existente = db.query(models.Transaccion).filter(
        models.Transaccion.numero_transaccion == transaccion.numero_transaccion,

```

```

        models.Transaccion.usuario_id == usuario_id
    ).first()

    if existente:
        raise ValueError(f"La transacción {transaccion.numero_transaccion} ya existe")

    # Crear transacción con todos los campos validados
    db_transaccion = models.Transaccion(
        usuario_id=usuario_id,
        banco=transaccion.banco,
        fecha=transaccion.fecha, # Ya viene validado DD/MM/YYYY
        beneficiario=transaccion.beneficiario,
        ordenante=transaccion.ordenante,
        monto=transaccion.monto, # Ya viene validado y redondeado
        moneda=transaccion.moneda, # Ya viene en mayúsculas
        numero_transaccion=transaccion.numero_transaccion,
        tipo=transaccion.tipo,
        estado=models.EstadoTransaccion.PENDIENTE,
        descripcion=transaccion.descripcion or "",
        etiquetas=json.dumps(transaccion.etiquetas) if transaccion.etiquetas else
"[]"
    )

    try:
        db.add(db_transaccion)
        db.commit()
        db.refresh(db_transaccion)
        return db_transaccion
    except Exception as e:
        db.rollback()
        raise e

def parse_and_create_transaccion(db: Session, texto: str, tipo: schemas.TipoTransaccion,
                                usuario_id: int, descripcion: Optional[str] = None):
    """
    Crear transacción desde texto bancario
    Usa el parser existente pero valida con tu schema
    """
    # Parsear el texto usando tu función existente
    datos = parse_transaccion_texto(texto)

```



```
# Extraer datos del parseo
from app.utils import enmascarar_numero_cuenta

# Validar que tenemos todos los datos necesarios
required_fields = ['banco', 'fecha', 'beneficiario', 'ordenante', 'monto', 'moneda', 'numero_transaccion']
for field in required_fields:
    if field not in datos:
        raise ValueError(f"No se pudo extraer el campo '{field}' del texto")

# Validar fecha (debe venir en formato DD/MM/YYYY del parser)
fecha_pattern = r"^\d{2}/\d{2}/\d{4}$"
if not re.match(fecha_pattern, datos['fecha']):
    # Intentar convertir si viene en otro formato
    from datetime import datetime
    try:
        # Asumiendo que viene en formato YYYY-MM-DD del parser
        fecha_obj = datetime.strptime(datos['fecha'], '%Y-%m-%d')
        datos['fecha'] = fecha_obj.strftime('%d/%m/%Y')
    except:
        raise ValueError(f"Formato de fecha inválido: {datos['fecha']}")

# Validar monto
try:
    monto = float(datos['monto'])
    if monto <= 0 or monto > 10000000:
        raise ValueError(f"Monto inválido: {monto}")
    datos['monto'] = round(monto, 2)
except:
    raise ValueError(f"Monto inválido: {datos['monto']}")

# Validar moneda
moneda = datos.get('moneda', 'EUR').upper()
monedas_validas = ['CUP', 'USD', 'EUR', 'MLC']
if moneda not in monedas_validas:
    moneda = 'EUR' # Valor por defecto
datos['moneda'] = moneda

# Crear objeto TransaccionCreate con los datos parseados
transaccion_data = {
    "banco": datos.get('banco', 'Desconocido'),
    "fecha": datos['fecha'],
    "beneficiario": datos['beneficiario'],
    "ordenante": datos['ordenante'],
```

```
"monto": datos['monto'],
"moneda": datos['moneda'],
"numero_transaccion": datos['numero_transaccion'],
"tipo": tipo,
"descripcion": descripcion or datos.get('concepto', ''),
"etiquetas": []
}

# Validar con el schema
try:
    transaccion_validada = schemas.TransaccionCreate(**transaccion_data)
except Exception as e:
    raise ValueError(f"Error validando datos parseados: {str(e)}")

# Verificar si ya existe
existente = db.query(models.Transaccion).filter(
    models.Transaccion.numero_transaccion == transaccion_validada.numero_trans
accion,
    models.Transaccion.usuario_id == usuario_id
).first()

if existente:
    raise ValueError(f"La transacción {transaccion_validada.numero_transaccio
n} ya existe")

# Crear en BD
db_transaccion = models.Transaccion(
    usuario_id=usuario_id,
    banco=transaccion_validada.banco,
    fecha=transaccion_validada.fecha,
    beneficiario=transaccion_validada.beneficiario,
    ordenante=transaccion_validada.ordenante,
    monto=transaccion_validada.monto,
    moneda=transaccion_validada.moneda,
    numero_transaccion=transaccion_validada.numero_transaccion,
    tipo=transaccion_validada.tipo,
    estado=models.EstadoTransaccion.PENDIENTE,
    descripcion=transaccion_validada.descripcion,
    etiquetas="[]"
)

try:
    db.add(db_transaccion)
    db.commit()
```

```

        db.refresh(db_transaccion)
        return db_transaccion
    except Exception as e:
        db.rollback()
        raise e

def get_transacciones_usuario(db: Session, usuario_id: int, skip: int = 0, limit:
int = 100,
                                tipo: Optional[schemas.TipoTransaccion] = None,
                                estado: Optional[schemas.EstadoTransaccion] = None,
                                banco: Optional[str] = None,
                                moneda: Optional[str] = None,
                                fecha_desde: Optional[str] = None,
                                fecha_hasta: Optional[str] = None):
    """
    Obtener transacciones del usuario con filtros avanzados
    """
    query = db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id
    )

    # Aplicar filtros
    if tipo:
        query = query.filter(models.Transaccion.tipo == tipo)
    if estado:
        query = query.filter(models.Transaccion.estado == estado)
    if banco:
        query = query.filter(models.Transaccion.banco.ilike(f"%{banco}%"))
    if moneda:
        query = query.filter(models.Transaccion.moneda == moneda.upper())

    # Filtros de fecha (formato DD/MM/YYYY en BD)
    if fecha_desde:
        query = query.filter(models.Transaccion.fecha >= fecha_desde)
    if fecha_hasta:
        query = query.filter(models.Transaccion.fecha <= fecha_hasta)

    # Ordenar por fecha de transacción (no procesamiento)
    query = query.order_by(models.Transaccion.fecha.desc())

    return query.offset(skip).limit(limit).all()

def get_transaccion(db: Session, transaccion_id: int, usuario_id: int):
    """

```

Obtener una transacción verificando propiedad

```
"""  
  
return db.query(models.Transaccion).filter(  
    models.Transaccion.id == transaccion_id,  
    models.Transaccion.usuario_id == usuario_id  
)  
).first()
```

```
def update_transaccion(db: Session, transaccion_id: int, usuario_id: int,  
                       transaccion_update: schemas.TransaccionUpdate):  
    """  
    Actualizar transacción (solo campos permitidos)  
    """  
  
    db_transaccion = get_transaccion(db, transaccion_id, usuario_id)  
    if not db_transaccion:  
        return None  
  
    update_data = transaccion_update.dict(exclude_unset=True)  
  
    # Manejar etiquetas como JSON  
    if 'etiquetas' in update_data and update_data['etiquetas'] is not None:  
        update_data['etiquetas'] = json.dumps(update_data['etiquetas'])  
  
    # Actualizar solo campos permitidos  
    for key, value in update_data.items():  
        if value is not None and hasattr(db_transaccion, key):  
            setattr(db_transaccion, key, value)  
  
    try:  
        db.commit()  
        db.refresh(db_transaccion)  
        return db_transaccion  
    except Exception as e:  
        db.rollback()  
        raise e  
  
def confirmar_transaccion(db: Session, transaccion_id: int, usuario_id: int):  
    """  
    Confirmar transacción recibida  
    """  
  
    db_transaccion = get_transaccion(db, transaccion_id, usuario_id)  
    if not db_transaccion:  
        return None  
  
    if db_transaccion.tipo == schemas.TipoTransaccion.RECIBIDA:
```

```
db_transaccion.estado = schemas.EstadoTransaccion.CONFIRMADA
db_transaccion.fecha_confirmacion = datetime.now()

try:
    db.commit()
    db.refresh(db_transaccion)
except Exception as e:
    db.rollback()
    raise e

return db_transaccion

def get_estadisticas_usuario(db: Session, usuario_id: int):
    """
    Obtener estadísticas detalladas del usuario
    """
    transacciones = db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id
    ).all()

    if not transacciones:
        return {
            "total_transacciones": 0,
            "total_enviadas": 0,
            "total_recibidas": 0,
            "balance": 0,
            "por_moneda": {},
            "por_banco": {},
            "pendientes": 0,
            "confirmadas": 0
        }

    # Estadísticas por moneda
    por_moneda = {}
    for t in transacciones:
        if t.moneda not in por_moneda:
            por_moneda[t.moneda] = {
                "enviado": 0,
                "recibido": 0,
                "balance": 0
            }

        if t.tipo == schemas.TipoTransaccion.ENVIADA:
            por_moneda[t.moneda]["enviado"] += t.monto
```

```

else:
    por_moneda[t.moneda]["recibido"] += t.monto

    por_moneda[t.moneda]["balance"] = (
        por_moneda[t.moneda]["recibido"] - por_moneda[t.moneda]["enviado"]
    )

# Estadísticas por banco
por_banco = {}
for t in transacciones:
    if t.banco not in por_banco:
        por_banco[t.banco] = {
            "cantidad": 0,
            "monto_total": 0,
            "promedio": 0
        }
    por_banco[t.banco]["cantidad"] += 1
    por_banco[t.banco]["monto_total"] += t.monto

for banco in por_banco:
    por_banco[banco]["promedio"] = (
        por_banco[banco]["monto_total"] / por_banco[banco]["cantidad"]
    )

return {
    "total_transacciones": len(transacciones),
    "total_enviadas": sum(t.monto for t in transacciones if t.tipo == schemas.
TipoTransaccion.ENVIADA),
    "total_recibidas": sum(t.monto for t in transacciones if t.tipo == schema
s.TipoTransaccion.RECIBIDA),
    "balance": sum(t.monto for t in transacciones if t.tipo == schemas.TipoTra
nsaccion.RECIBIDA) -
        sum(t.monto for t in transacciones if t.tipo == schemas.TipoTran
saccion.ENVIADA),
    "por_moneda": por_moneda,
    "por_banco": por_banco,
    "pendientes": len([t for t in transacciones if t.estado == schemas.EstadoT
ransaccion.PENDIENTE]),
    "confirmadas": len([t for t in transacciones if t.estado == schemas.Estado
Transaccion.CONFIRMADA])
}

def buscar_transacciones(db: Session, usuario_id: int, query: str):
    """

```

Búsqueda avanzada de transacciones

```

"""
return db.query(models.Transaccion).filter(
    models.Transaccion.usuario_id == usuario_id,
    (
        models.Transaccion.beneficiario.ilike(f"%{query}%") |
        models.Transaccion.ordenante.ilike(f"%{query}%") |
        models.Transaccion.descripcion.ilike(f"%{query}%") |
        models.Transaccion.numero_transaccion.ilike(f"%{query}%") |
        models.Transaccion.banco.ilike(f"%{query}%")
    )
).order_by(models.Transaccion.fecha.desc()).all()

def get_transacciones_por_mes(db: Session, usuario_id: int, año: int, mes: int):
    """
    Obtener transacciones de un mes específico
    """
    mes_str = f"{mes:02d}"
    return db.query(models.Transaccion).filter(
        models.Transaccion.usuario_id == usuario_id,
        models.Transaccion.fecha.like(f"%/{mes_str}/{año}%")
    ).order_by(models.Transaccion.fecha.desc()).all()

```



Actualiza los endpoints en main.py para usar las nuevas funcionalidades:

python

```

@app.get("/transacciones/estadisticas")
def obtener_estadisticas(
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Obtener estadísticas detalladas del usuario"""
    return crud.get_estadisticas_usuario(db, current_user.id)

@app.get("/transacciones/por-mes/{año}/{mes}")
def transacciones_por_mes(
    año: int,
    mes: int,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)

```

```

):
    """Obtener transacciones de un mes específico"""
    return crud.get_transacciones_por_mes(db, current_user.id, año, mes)

@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str = Field(..., min_length=2, description="Término de búsqueda"),
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto"""
    return crud.buscar_transacciones(db, current_user.id, q)

```

Tu schema ya tiene todas las validaciones necesarias, y el crud ahora las respeta. Cada transacción queda correctamente asociada al usuario mediante `usuario_id`.

```

File "D:\proyectoBanc\main.py", line 353, in <module>
    q: str = Field(..., min_length=2, description="Término de
    búsqueda"),
NameError: name 'Field' is not defined

```

El error es porque falta importar `Field` de Pydantic. En FastAPI, `Field` se usa para validaciones pero debe importarse correctamente. Aquí está la corrección:

Solución: Importar Field correctamente

En tu `main.py`, asegúrate de tener todas las importaciones necesarias:

python

```

from fastapi import FastAPI, Depends, HTTPException, status, Query # ← Query también es útil
from fastapi.security import OAuth2PasswordRequestForm
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta, datetime
from pydantic import Field # ← ESTA ES LA IMPORTACIÓN QUE FALTA

from app import crud, models, auth, schemas, utils

```



```
from app.database import engine, get_db, create_tables
```

```
# ... resto del código ...
```



Corrige el endpoint de búsqueda:

Hay dos formas de hacerlo:

Opción 1: Usar Query de FastAPI (recomendado para endpoints)

python

```
@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str = Query(..., min_length=2, description="Término de búsqueda"), # ← Usa Query
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto"""
    return crud.buscar_transacciones(db, current_user.id, q)
```

Opción 2: Usar Field de Pydantic (necesita Body/Query)

python

```
from pydantic import BaseModel, Field

# Si quieres mantener Field, necesitas un modelo
class BusquedaRequest(BaseModel):
    q: str = Field(..., min_length=2, description="Término de búsqueda")

@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    params: BusquedaRequest = Depends(), # ← Así usarías Field
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
```

```
"""Buscar transacciones por texto"""
return crud.buscar_transacciones(db, current_user.id, params.q)
```

Opción 3: La más simple, sin validaciones extras

python

```
@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str, # ← Simple, sin validaciones
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto"""
    if len(q) < 2:
        raise HTTPException(status_code=400, detail="La búsqueda debe tener al menos 2 caracteres")
    return crud.buscar_transacciones(db, current_user.id, q)
```



Código completo y corregido de los últimos endpoints:

python

```
# Asegúrate de tener todas las importaciones al inicio
from fastapi import FastAPI, Depends, HTTPException, status, Query
from fastapi.security import OAuth2PasswordRequestForm
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta, datetime
from pydantic import Field # Si lo necesitas en otros lugares

from app import crud, models, auth, schemas, utils
from app.database import engine, get_db, create_tables

# ... todo tu código existente ...

# Al final, los nuevos endpoints:
@app.get("/transacciones/estadisticas")
def obtener_estadisticas(
```

```

current_user: models.Usuario = Depends(auth.get_current_active_user),
db: Session = Depends(get_db)
):
    """Obtener estadísticas detalladas del usuario"""
    try:
        estadisticas = crud.get_estadisticas_usuario(db, current_user.id)
        return estadisticas
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error al obtener estadísticas: {str(e)}")

@app.get("/transacciones/por-mes/{año}/{mes}")
def transacciones_por_mes(
    año: int,
    mes: int,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Obtener transacciones de un mes específico"""
    # Validar mes
    if mes < 1 or mes > 12:
        raise HTTPException(status_code=400, detail="Mes inválido (1-12)")

    # Validar año (opcional)
    if año < 2000 or año > 2100:
        raise HTTPException(status_code=400, detail="Año inválido")

    try:
        transacciones = crud.get_transacciones_por_mes(db, current_user.id, año, mes)
        return transacciones
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error al obtener transacciones: {str(e)}")

@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str = Query(..., min_length=2, max_length=100, description="Término de búsqueda"),
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto en beneficiario, ordenante, descripción, et

```

```

c. """
    try:
        resultados = crud.buscar_transacciones(db, current_user.id, q)
        return resultados
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error en la búsqueda: {str
(e)}")

```

Para probar los nuevos endpoints:

bash

```

# 1. Estadísticas
curl -X GET "http://localhost:8000/transacciones/estadisticas" \
-H "Authorization: Bearer TU_TOKEN"

# 2. Transacciones por mes (ejemplo: marzo 2024)
curl -X GET "http://localhost:8000/transacciones/por-mes/2024/3" \
-H "Authorization: Bearer TU_TOKEN"

# 3. Buscar transacciones
curl -X GET "http://localhost:8000/transacciones/buscar?q=pago" \
-H "Authorization: Bearer TU_TOKEN"

```

La solución más limpia es usar `Query` de FastAPI para los parámetros de consulta, que es justamente lo que necesitas en ese endpoint.

```

File "D:\projectoBanc\venv\lib\site-packages\fastapi\routing.py",
line 921, in decorator
    self.add_api_route(
File "D:\projectoBanc\venv\lib\site-packages\fastapi\routing.py",
line 860, in add_api_route
    route = route_class(
File "D:\projectoBanc\venv\lib\site-packages\fastapi\routing.py",
line 490, in __init__
    self.dependant = get_dependant(path=self.path_format,
call=self.endpoint)
File "D:\projectoBanc\venv\lib\site-
packages\fastapi\dependencies\utils.py", line 290, in
get_dependant

```

```

        add_param_to_fields(field=param_field, dependant=dependant)
File "D:\projectoBanc\venv\lib\site-
packages\fastapi\dependencies\utils.py", line 470, in
add_param_to_fields
    if field_info.in_ == params.ParamTypes.path:
AttributeError: 'FieldInfo' object has no attribute 'in_'

```

El error indica un problema de compatibilidad entre versiones de FastAPI y Pydantic. Vamos a solucionarlo:

Solución: Corregir la importación y uso de Field/Query

1. Primero, verifica tus versiones:

```
bash
```

```
pip show fastapi pydantic
```

2. Actualiza las importaciones en main.py :

```
python
```

```

from fastapi import FastAPI, Depends, HTTPException, status
from fastapi import Query # Importa Query directamente
from fastapi.security import OAuth2PasswordRequestForm
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta, datetime

# NO importes Field de pydantic aquí si no es necesario
# from pydantic import Field ← Comenta o elimina esta línea

from app import crud, models, auth, schemas, utils
from app.database import engine, get_db, create_tables

# ... resto del código ...

```

3. Corrige el endpoint problemático:

python

```
@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str = Query(..., min_length=2, max_length=100, description="Término de búsqueda"),
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto en beneficiario, ordenante, descripción, etc."""
    try:
        resultados = crud.buscar_transacciones(db, current_user.id, q)
        return resultados
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error en la búsqueda: {str(e)}")
```

4. Si el error persiste, usa una versión más simple sin validaciones:

python

```
@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionResponse])
def buscar_transacciones(
    q: str, # Sin validaciones complejas
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto"""
    if not q or len(q) < 2:
        raise HTTPException(status_code=400, detail="La búsqueda debe tener al menos 2 caracteres")

    try:
        resultados = crud.buscar_transacciones(db, current_user.id, q)
        return resultados
    except Exception as e:
```

```
raise HTTPException(status_code=500, detail=f"Error en la búsqueda: {str
(e)}")
```

5. Solución alternativa con parámetro opcional:

python

```
@app.get("/transacciones/buscar", response_model=List[schemas.TransaccionRespons
e])
def buscar_transacciones(
    q: Optional[str] = None, # Hacerlo opcional
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Buscar transacciones por texto"""
    if not q:
        # Si no hay búsqueda, devolver todas (con límite)
        return crud.get_transacciones_usuario(db, current_user.id, limit=50)

    if len(q) < 2:
        raise HTTPException(status_code=400, detail="La búsqueda debe tener al men
os 2 caracteres")

    try:
        resultados = crud.buscar_transacciones(db, current_user.id, q)
        return resultados
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error en la búsqueda: {str
(e)}")
```

6. Si el error es de compatibilidad, actualiza los paquetes:

bash

```
# Actualizar FastAPI y Pydantic
pip install --upgrade fastapi pydantic

# Si eso no funciona, instalar versiones compatibles
pip install fastapi==0.104.1 pydantic==2.5.0
```

7. Verifica que no haya conflictos en `schemas.py` :

En tu `schemas.py` , asegúrate de que `Field` se importa correctamente de Pydantic v2:

```
python
```

```
from pydantic import BaseModel, Field, ConfigDict, field_validator
from typing import Optional, List
from datetime import datetime
from enum import Enum
import re
import json
```

8. Reinicia el servidor:

```
bash
```

```
# Detén el servidor (Ctrl+C) y vuelve a iniciar
python main.py
```

9. Prueba el endpoint con curl:

```
bash
```

```
# Probar búsqueda
curl -X GET "http://localhost:8000/transacciones/buscar?q=pago" \
-H "Authorization: Bearer TU_TOKEN"

# Si usaste la versión con q opcional
curl -X GET "http://localhost:8000/transacciones/buscar" \
-H "Authorization: Bearer TU_TOKEN"
```

El error `'FieldInfo' object has no attribute 'in_'` es típico cuando hay mezcla de versiones de Pydantic (v1 vs v2). La solución más rápida es usar `Query` de FastAPI en lugar de `Field` para los parámetros de consulta.

crees que deba borrar la db para que se haga nuevamente con

las nuevas campos si se modifiko algo

No, no es necesario borrar la base de datos si solo estás modificando la lógica de negocio (crud, schemas, endpoints). Solo necesitas borrar la DB si:

● **Cuándo SÍ debes borrar la DB:**

1. **Cambios en modelos** (si agregaste/quitaste columnas en `models.py`)
 - Ej: Agregaste `fecha_confirmacion` O etiquetas
 - Ej: Cambiaste tipo de dato de una columna
2. **Cambios en relaciones** entre tablas
 - Ej: Nuevas claves foráneas
3. **Cambios en constraints** (unique, nullable, etc.)

● **Cuándo NO debes borrar la DB:**

- ✓ Modificaciones en `crud.py`
- ✓ Modificaciones en `schemas.py` (validaciones)
- ✓ Nuevos endpoints en `main.py`
- ✓ Mejoras en `auth.py`
- ✓ Cambios en `utils.py`

📋 **Verifica si tus modelos cambiaron:**

Revisa tu `models.py` actual comparado con el anterior. Si tienes cambios como estos, **SÍ** necesitas recrear la DB:

python

```
# Si AGREGAS nuevas columnas:
class Transaccion(Base):
    # ... columnas existentes ...

    # NUEVAS columnas (esto requiere recrear DB)
    fecha_confirmacion = Column(DateTime, nullable=True) # ← NUEVA
    etiquetas = Column(String, default="[]") # ← NUEVA
```

```
# Si MODIFICAS columnas existentes:  
monto = Column(Float, nullable=False) # ← Antes era nullable=True  
numero_transaccion = Column(String, unique=True) # ← Antes no era unique
```

Cómo verificar sin borrar:

Opción 1: Ver estructura actual

```
bash  
  
# Con sqlite3  
sqlite3 bancaria.db ".schema transacciones"  
  
# Verás la estructura actual de la tabla
```

Opción 2: Prueba con un nuevo registro

```
python  
  
# Crea una transacción de prueba  
curl -X POST "http://localhost:8000/transacciones/" \  
-H "Authorization: Bearer TU_TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "banco": "Test",  
  "fecha": "15/03/2024",  
  "beneficiario": "ES123456789",  
  "ordenante": "ES987654321",  
  "monto": 100.50,  
  "moneda": "EUR",  
  "numero_transaccion": "TEST123",  
  "tipo": "RECIBIDA"  
}'
```

Si decides que necesitas recrear la DB:

```
bash
```

```
# 1. Haz backup por si acaso
cp bancaria.db bancaria_backup.db

# 2. Borra La DB
rm bancaria.db

# 3. Reinicia La app (se recreará automáticamente con create_tables())
python main.py

# 4. Verifica que se creó
sqlite3 bancaria.db ".tables"
```

Recomendación:

NO borres la DB aún. Primero:

1. Intenta usar la app con los nuevos endpoints
2. Si ves errores de "columna no existe", entonces Sí necesitas recrearla
3. Los errores típicos serían:

text

```
sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) no such column: transacciones.fecha_confirmacion
```

Para futuros cambios, usa migraciones:

Instala Alembic para manejar migraciones sin perder datos:

bash

```
pip install alembic
alembic init migrations
# Configura alembic y Luego:
alembic revision --autogenerate -m "agrega fecha_confirmacion"
alembic upgrade head
```

Conclusión: Si solo tocaste `crud.py`, `schemas.py` y `main.py`, **no necesitas borrar nada**. Tus datos están seguros.

```

File "D:\projectoBanc\venv\lib\site-
packages\uvicorn\protocols\http\h11_impl.py", line 408, in run_asgi
    result = await app( # type: ignore[func-returns-value]
File "D:\projectoBanc\venv\lib\site-
packages\uvicorn\middleware\proxy_headers.py", line 84, in
__call__
    return await self.app(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-
packages\fastapi\applications.py", line 1106, in __call__
    await super().__call__(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-
packages\starlette\applications.py", line 122, in __call__
    await self.middleware_stack(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-
packages\starlette\middleware\errors.py", line 184, in __call__
    raise exc
File "D:\projectoBanc\venv\lib\site-
packages\starlette\middleware\errors.py", line 162, in __call__
    await self.app(scope, receive, _send)
File "D:\projectoBanc\venv\lib\site-
packages\starlette\middleware\cors.py", line 91, in __call__
    await self.simple_response(scope, receive, send,
request_headers=headers)
File "D:\projectoBanc\venv\lib\site-
packages\starlette\middleware\cors.py", line 146, in
simple_response
    await self.app(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-
packages\starlette\middleware\exceptions.py", line 79, in __call__
    raise exc
File "D:\projectoBanc\venv\lib\site-
packages\starlette\middleware\exceptions.py", line 68, in __call__
    await self.app(scope, receive, sender)
File "D:\projectoBanc\venv\lib\site-
packages\fastapi\middleware\asyncexitstack.py", line 20, in
__call__
    raise e

```

```

File "D:\projectoBanc\venv\lib\site-
packages\fastapi\middleware\asyncexitstack.py", line 17, in
__call__
    await self.app(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-packages\starlette\routing.py",
line 718, in __call__
    await route.handle(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-packages\starlette\routing.py",
line 276, in handle
    await self.app(scope, receive, send)
File "D:\projectoBanc\venv\lib\site-packages\starlette\routing.py",
line 66, in app
    response = await func(request)
File "D:\projectoBanc\venv\lib\site-packages\fastapi\routing.py",
line 292, in app
    content = await serialize_response(
File "D:\projectoBanc\venv\lib\site-packages\fastapi\routing.py",
line 155, in serialize_response
    raise ResponseValidationError(
fastapi.exceptions.ResponseValidationError: 2 validation errors:
  {'type': 'value_error', 'loc': ('response', 'fecha'), 'msg': 'Value
error, Formato de fecha inválido. Use DD/MM/YYYY', 'input':
'12/3/2026', 'ctx': {'error': ValueError('Formato de fecha inválido.
Use DD/MM/YYYY')}}, 'url':
'https://errors.pydantic.dev/2.5/v/value_error'}
  {'type': 'list_type', 'loc': ('response', 'etiquetas'), 'msg': 'Input
should be a valid list', 'input': '[]', 'url':
'https://errors.pydantic.dev/2.5/v/list_type'}

```

¡Excelente! El error nos muestra exactamente el problema. La fecha '12/3/2026' no cumple con el formato DD/MM/YYYY que esperas (debería ser 12/03/2026 con dos dígitos para el mes). También hay un problema con las etiquetas que vienen como string '[]' cuando deberían ser lista.

Solución: Corregir el formato de fecha y manejo de etiquetas

1. Primero, corrige el `utils.py` para que devuelva fechas en formato correcto:

python

```
import re
from datetime import datetime
import unicodedata

def parse_transaccion_texto(texto: str) -> dict:
    """
    Parsea texto de transacción bancaria y extrae los datos
    Retorna un diccionario con los campos en el formato correcto
    """
    resultado = {
        "banco": "",
        "fecha": "",
        "beneficiario": "",
        "ordenante": "",
        "monto": 0.0,
        "moneda": "EUR",
        "numero_transaccion": "",
        "concepto": ""
    }

    # Limpiar texto
    texto = texto.strip()
    lineas = texto.split('\n')

    for linea in lineas:
        linea = linea.strip()
        linea_lower = linea.lower()

        # Buscar fecha - varios formatos posibles
        # Formato: DD/MM/YYYY o DD-MM-YYYY
        fecha_match = re.search(r'(\d{1,2})[/-](\d{1,2})[/-](\d{4})', linea)
        if fecha_match:
            dia, mes, año = fecha_match.groups()
            # Formatear a DD/MM/YYYY con dos dígitos
            resultado["fecha"] = f"{int(dia):02d}/{int(mes):02d}/{año}"
            continue

        # Buscar monto
        monto_match = re.search(r'(\d+[\.,]\d+)\s*(EUR|USD|CUP|MLC)', linea, re.IGNORECASE)
```

```

ORECASE)
    if monto_match:
        monto_str, moneda = monto_match.groups()
        # Convertir a float (reemplazar coma por punto)
        monto_str = monto_str.replace(',', '.')
        resultado["monto"] = round(float(monto_str), 2)
        resultado["moneda"] = moneda.upper()
        continue

    # Buscar números de cuenta (IBAN)
    cuenta_match = re.search(r'([A-Z]{2}\d{22})', linea)
    if cuenta_match:
        cuenta = cuenta_match.group(1)
        if "beneficiario" in linea_lower or "destino" in linea_lower:
            resultado["beneficiario"] = cuenta
        elif "ordenante" in linea_lower or "origen" in linea_lower:
            resultado["ordenante"] = cuenta
        elif not resultado["beneficiario"]:
            resultado["beneficiario"] = cuenta
        elif not resultado["ordenante"]:
            resultado["ordenante"] = cuenta
        continue

    # Buscar número de transacción/referencia
    ref_match = re.search(r'(?:(referencia|ref|nº|n°)|[:\s]*([A-Z0-9]{8,}))', linea, re.IGNORECASE)
    if ref_match:
        resultado["numero_transaccion"] = ref_match.group(1)
        continue

    # Buscar concepto
    if "concepto" in linea_lower or "descripcion" in linea_lower:
        concepto = re.sub(r'^^[^:]*:', '', linea).strip()
        if concepto:
            resultado["concepto"] = concepto

    # Si no se encontró número de transacción, generar uno
    if not resultado["numero_transaccion"]:
        resultado["numero_transaccion"] = f"TXN{datetime.now().strftime('%Y%m%d%H%M%S')}}}"

    return resultado

```

2. Corrige el crud.py para manejar correctamente las etiquetas:

python

```
def parse_and_create_transaccion(db: Session, texto: str, tipo: schemas.TipoTransaccion,
                                usuario_id: int, descripcion: Optional[str] = None):
    """
    Crear transacción desde texto bancario
    """
    # Parsear el texto
    datos = parse_transaccion_texto(texto)

    print("\n📄 Datos parseados:", datos)

    # Validar que tenemos fecha
    if not datos.get('fecha'):
        # Usar fecha actual si no se encontró
        datos['fecha'] = datetime.now().strftime('%d/%m/%Y')

    # Crear objeto TransaccionCreate con los datos parseados
    transaccion_data = {
        "banco": datos.get('banco', 'Desconocido'),
        "fecha": datos['fecha'],
        "beneficiario": datos.get('beneficiario', ''),
        "ordenante": datos.get('ordenante', ''),
        "monto": datos.get('monto', 0),
        "moneda": datos.get('moneda', 'EUR'),
        "numero_transaccion": datos.get('numero_transaccion', ''),
        "tipo": tipo,
        "descripcion": descripcion or datos.get('concepto', ''),
        "etiquetas": [] # Siempre lista vacía inicialmente
    }

    # Validar con el schema
    try:
        transaccion_validada = schemas.TransaccionCreate(**transaccion_data)
        print("✅ Datos validados:", transaccion_validada)
    except Exception as e:
        print("❌ Error validando:", str(e))
        raise ValueError(f"Error validando datos: {str(e)}")

    # Verificar si ya existe
```



```

existente = db.query(models.Transaccion).filter(
    models.Transaccion.numero_transaccion == transaccion_validada.numero_trans
accion,
    models.Transaccion.usuario_id == usuario_id
).first()

if existente:
    raise ValueError(f"La transacción {transaccion_validada.numero_transaccio
n} ya existe")

# Crear en BD
db_transaccion = models.Transaccion(
    usuario_id=usuario_id,
    banco=transaccion_validada.banco,
    fecha=transaccion_validada.fecha,
    beneficiario=transaccion_validada.beneficiario,
    ordenante=transaccion_validada.ordenante,
    monto=transaccion_validada.monto,
    moneda=transaccion_validada.moneda,
    numero_transaccion=transaccion_validada.numero_transaccion,
    tipo=transaccion_validada.tipo,
    estado=models.EstadoTransaccion.PENDIENTE,
    descripcion=transaccion_validada.descripcion,
    etiquetas="[]" # Guardar como JSON string
)

try:
    db.add(db_transaccion)
    db.commit()
    db.refresh(db_transaccion)
    print("✅ Transacción guardada en BD")
    return db_transaccion
except Exception as e:
    db.rollback()
    print("❌ Error guardando:", str(e))
    raise e

```

3. En schemas.py , asegura que la validación de fecha sea flexible:

python

```

class TransaccionBase(BaseModel):
    banco: str = Field(..., min_length=2, max_length=100)
    fecha: str = Field(..., description="Fecha en formato DD/MM/YYYY")
    beneficiario: str = Field(..., min_length=4, max_length=50)
    ordenante: str = Field(..., min_length=4, max_length=50)
    monto: float = Field(..., gt=0, le=10000000)
    moneda: str = Field(..., min_length=1, max_length=10)
    numero_transaccion: str = Field(..., min_length=4, max_length=50)

    @field_validator('fecha')
    @classmethod
    def validar_fecha(cls, v: str) -> str:
        """Validar y normalizar formato DD/MM/YYYY"""
        v = v.strip()

        # Si ya está en formato correcto
        if re.match(r"^\d{2}/\d{2}/\d{4}$", v):
            return v

        # Intentar convertir desde otros formatos
        # DD/MM/YY o DD-MM-YYYY o YYYY-MM-DD
        patrones = [
            (r"^\d{1,2}/(\d{1,2})/(\d{4})$", lambda d,m,a: f"{int(d):02d}/{int(m):02d}/{a}"),
            (r"^\d{1,2})-(\d{1,2})-(\d{4})$", lambda d,m,a: f"{int(d):02d}/{int(m):02d}/{a}"),
            (r"^\d{4})-(\d{1,2})-(\d{1,2})$", lambda a,m,d: f"{int(d):02d}/{int(m):02d}/{a}"),
        ]

        for pattern, formatter in patrones:
            match = re.match(pattern, v)
            if match:
                return formatter(*match.groups())

        raise ValueError('Formato de fecha inválido. Use DD/MM/YYYY')

class TransaccionResponse(TransaccionBase):
    id: int
    usuario_id: int
    tipo: TipoTransaccion
    estado: EstadoTransaccion
    descripcion: str

```

```

etiquetas: List[str] # Siempre será Lista en La respuesta
fecha_procesamiento: datetime
fecha_confirmacion: Optional[datetime] = None

@field_validator('etiquetas', mode='before')
@classmethod
def parse_etiquetas(cls, v):
    """Convertir string JSON a lista si es necesario"""
    if isinstance(v, str):
        try:
            return json.loads(v)
        except:
            return []
    return v or []

model_config = ConfigDict(from_attributes=True)

```

4. Prueba con el texto corregido:

bash

```

curl -X POST "http://localhost:8000/transacciones/parsear" \
-H "Authorization: Bearer TU_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "texto": "Transferencia realizada el 12/03/2026\nOrdenante: ES1234567890123456
7890\nBeneficiario: ES98765432109876543210\nImporte: 150.50 EUR\nConcepto: Pago fa
ctura",
  "tipo": "RECIBIDA",
  "descripcion": "Prueba"
}'

```

El problema principal era que tu parser estaba generando fechas como '12/3/2026' en lugar de '12/03/2026'. Con las correcciones, ahora normalizará las fechas al formato correcto.

me da este error INFO: 127.0.0.1:49910 - "POST /transacciones/parsear HTTP/1.1" 422 Unprocessable Entity creo que debo eliminar los espacios en blanco porque los mensajes de las trananferencias tienes varias tabulaciones

¡Excelente observación! El error 422 es de validación, probablemente por los espacios/tabulaciones. Vamos a mejorar el parser para manejar cualquier formato de texto.

Solución: Mejorar el parser para manejar espacios y tabulaciones

1. Actualiza `utils.py` con un parser más robusto:

```
python

import re
from datetime import datetime
import unicodedata

def limpiar_texto(texto: str) -> str:
    """Limpia el texto eliminando espacios extras y normalizando"""
    if not texto:
        return ""

    # Reemplazar tabulaciones y múltiples espacios por un solo espacio
    texto = re.sub(r'\s+', ' ', texto)

    # Eliminar espacios al inicio y final
    texto = texto.strip()

    return texto

def parse_transaccion_texto(texto: str) -> dict:
    """
    Parsea texto de transacción bancaria y extrae los datos
    Maneja cualquier formato de espacios y tabulaciones
    """
    resultado = {
        "banco": "",
        "fecha": "",
        "beneficiario": "",
        "ordenante": "",
        "monto": 0.0,
        "moneda": "EUR",
        "numero_transaccion": "",
```

```

    "concepto": ""
}

# Limpiar texto primero
texto = limpiar_texto(texto)

# Dividir por puntos, comas o espacios para análisis
partes = re.split(r'[.,;\s]+', texto)

print("📄 Texto original:", texto)
print("📄 Partes:", partes)

# Buscar fecha - varios formatos
# Formato: DD/MM/YYYY, DD-MM-YYYY, DD.MM.YYYY
fecha_pattern = r'(\d{1,2})[/\-\.](\d{1,2})[/\-\.](\d{4})'
fecha_match = re.search(fecha_pattern, texto)
if fecha_match:
    dia, mes, año = fecha_match.groups()
    resultado["fecha"] = f"{int(dia):02d}/{int(mes):02d}/{año}"
    print(f"📅 Fecha encontrada: {resultado['fecha']}")

# Buscar monto con moneda
# Patrones: 150.50 EUR, 150,50€, EUR 150.50, etc.
monto_patterns = [
    r'(\d+[. ,]\d+)\s*(EUR|USD|CUP|MLC|€|\$)',
    r'(EUR|USD|CUP|MLC|€|\$)\s*(\d+[. ,]\d+)',
    r'importe[:\s]*(\d+[. ,]\d+)',
    r'monto[:\s]*(\d+[. ,]\d+)'
]

for pattern in monto_patterns:
    monto_match = re.search(pattern, texto, re.IGNORECASE)
    if monto_match:
        grupos = monto_match.groups()
        if len(grupos) == 2:
            # Determinar cuál es monto y cuál moneda
            if grupos[0].replace(',', '').replace('.', '').isdigit():
                monto_str, moneda = grupos
            else:
                moneda, monto_str = grupos

            # Limpiar monto
            monto_str = monto_str.replace(',', '').replace('.', '')
            # Eliminar símbolos de moneda

```

```

monto_str = re.sub(r'[€$]', '', monto_str)

try:
    resultado["monto"] = round(float(monto_str), 2)

    # Normalizar moneda
    moneda = moneda.upper()
    if '€' in moneda or 'EUR' in moneda:
        resultado["moneda"] = "EUR"
    elif '$' in moneda or 'USD' in moneda:
        resultado["moneda"] = "USD"
    elif 'CUP' in moneda:
        resultado["moneda"] = "CUP"
    elif 'MLC' in moneda:
        resultado["moneda"] = "MLC"

    print(f"💰 Monto encontrado: {resultado['monto']} {resultado
['moneda']}")
except:
    pass
break

# Buscar números de cuenta (IBAN español)
# Patrón: ES + 22 dígitos (pueden tener espacios)
iban_pattern = r'ES\s*\d{2}\s*\d{4}\s*\d{4}\s*\d{4}\s*\d{4}\s*\d{4}'
cuentas = re.findall(iban_pattern, texto, re.IGNORECASE)

# Limpiar IBANs (quitar espacios)
cuentas_limpias = [re.sub(r'\s+', '', c) for c in cuentas]

for cuenta in cuentas_limpias:
    texto_lower = texto.lower()
    # Determinar si es ordenante o beneficiario por contexto
    if "ordenante" in texto_lower or "origen" in texto_lower or "de" in texto_
lower:
        if not resultado["ordenante"]:
            resultado["ordenante"] = cuenta
            print(f"👤 Ordenante encontrado: {resultado['ordenante']}")
        elif "beneficiario" in texto_lower or "destino" in texto_lower or "para" i
n texto_lower:
            if not resultado["beneficiario"]:
                resultado["beneficiario"] = cuenta
                print(f"👤 Beneficiario encontrado: {resultado['beneficiario']}")
            else:

```

```

# Si no hay contexto, asignar según orden
if not resultado["ordenante"]:
    resultado["ordenante"] = cuenta
elif not resultado["beneficiario"]:
    resultado["beneficiario"] = cuenta

# Buscar número de transacción/referencia
ref_patterns = [
    r'(?:(referencia|ref|nº|n°|#)[:\s]*([A-Z0-9]{8,}))',
    r'(?:(operación|operacion)[:\s]*([A-Z0-9]{8,}))',
    r'(\d{10,})' # Cualquier número de 10+ dígitos
]

for pattern in ref_patterns:
    ref_match = re.search(pattern, texto, re.IGNORECASE)
    if ref_match:
        resultado["numero_transaccion"] = ref_match.group(1)
        print(f"🔍 Referencia encontrada: {resultado['numero_transaccion']}")
        break

# Buscar concepto (Lo que queda después de "concepto" o "descripción")
concepto_patterns = [
    r'(?:(concepto|descripción|descripcion)[:\s]*(.+?))?(?=(?:\.\|$|\n))',
    r'para[:\s]*(.+?)(?=(?:\.\|$|\n))'
]

for pattern in concepto_patterns:
    concepto_match = re.search(pattern, texto, re.IGNORECASE)
    if concepto_match:
        resultado["concepto"] = concepto_match.group(1).strip()
        print(f"📄 Concepto encontrado: {resultado['concepto']}")
        break

# Si no se encontró número de transacción, generar uno
if not resultado["numero_transaccion"]:
    resultado["numero_transaccion"] = f"TXN{datetime.now().strftime('%Y%m%d%H%M%S')}"
    print(f"🆕 Referencia generada: {resultado['numero_transaccion']}")

# Si no se encontró fecha, usar actual
if not resultado["fecha"]:
    resultado["fecha"] = datetime.now().strftime('%d/%m/%Y')
    print(f"📅 Fecha actual usada: {resultado['fecha']}")

```

```

return resultado

def enmascarar_numero_cuenta(cuenta: str) -> str:
    """Enmascara número de cuenta mostrando solo primeros y últimos dígitos"""
    if not cuenta or len(cuenta) < 8:
        return cuenta

    # Limpiar espacios
    cuenta = re.sub(r'\s+', '', cuenta)

    # Mostrar primeros 4 y últimos 4
    if len(cuenta) > 8:
        return cuenta[:4] + "*" * (len(cuenta) - 8) + cuenta[-4:]
    return cuenta

```

2. En main.py , mejora el endpoint de parseo:

python

```

@app.post("/transacciones/parsear", response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    try:
        print("="*60)
        print("📄 Recibida petición de parseo")
        print(f"Texto: {repr(transaccion_req.texto)}") # repr muestra caracteres
        especiales
        print(f"Tipo: {transaccion_req.tipo}")

        # Limpiar texto antes de parsear
        texto_limpio = utils.limpiar_texto(transaccion_req.texto)
        print(f"Texto limpio: {repr(texto_limpio)}")

        # Parsear
        datos = utils.parse_transaccion_texto(texto_limpio)
        print(f"Datos parseados: {datos}")

        # Verificar si ya existe
        existente = db.query(models.Transaccion).filter(

```



```

models.Transaccion.numero_transaccion == datos["numero_transaccion"],
models.Transaccion.usuario_id == current_user.id
).first()

if existente:
    raise HTTPException(
        status_code=400,
        detail=f"La transacción {datos['numero_transaccion']} ya existe"
    )

# Crear transacción
transaccion = crud.parse_and_create_transaccion(
    db=db,
    texto=texto_limpio,
    tipo=transaccion_req.tipo,
    usuario_id=current_user.id,
    descripcion=transaccion_req.descripcion
)

print("✅ Transacción creada exitosamente")
print("="*60)
return transaccion

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error: {str(e)}")
    raise HTTPException(
        status_code=400,
        detail=f"Error al procesar: {str(e)}"
    )

```

3. Prueba con diferentes formatos:

bash

Con tabulaciones

```

curl -X POST "http://localhost:8000/transacciones/parsear" \
  -H "Authorization: Bearer TU_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "texto": "Transferencia\t\tbancaria\nFecha:\t15/03/2024\nOrdenante:\tES1234567
8901234567890\nBeneficiario:\tES98765432109876543210\nImporte:\t150.50 EUR\nConcep

```

```
to:\tPago de servicios",
  "tipo": "RECIBIDA"
}'
```

Con múltiples espacios

```
curl -X POST "http://localhost:8000/transacciones/parsear" \
-H "Authorization: Bearer TU_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "texto": "Transferencia bancaria Fecha: 15/03/2024 Ordenante: E
S12345678901234567890 Beneficiario: ES98765432109876543210 Importe: 15
0.50 EUR Concepto: Pago de servicios",
  "tipo": "RECIBIDA"
}'
```

El error 422 debería desaparecer porque ahora el texto se limpia correctamente antes de procesarlo. La función `limpiar_texto()` elimina todos los espacios extras y tabulaciones, normalizando el texto para facilitar el parseo.

```
estoy obteniendo unprocesable entiti y me devuelve este error "{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        73
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Invalid control character at"
      }
    }
  ]
}"
```

El error "Invalid control character" indica que hay caracteres de control (tabulaciones, saltos de línea, etc.) en el JSON que estás enviando. El JSON no permite estos caracteres sin escapar.

Solución: Escapar correctamente el texto con caracteres especiales

Opción 1: Usar json.dumps() en Python para escapar automáticamente

python

```
import json
import requests

texto_con_tabulaciones = """Transferencia          bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50   EUR
Concepto:       Pago    de      servicios"""

# Escapar el texto automáticamente
payload = {
    "texto": texto_con_tabulaciones,
    "tipo": "RECIBIDA"
}

# json.dumps() escapará automáticamente los caracteres especiales
json_payload = json.dumps(payload)
print(json_payload) # Verás que las tabulaciones se convierten en \t

headers = {
    "Authorization": f"Bearer {token}",
    "Content-Type": "application/json"
}

response = requests.post(
    f"{BASE_URL}/transacciones/parsear",
    headers=headers,
    data=json_payload # Usar data, no json
)
```

Opción 2: Para curl, escapar manualmente

```
bash
```

```
# En Windows CMD (escapar con ^)
curl -X POST "http://localhost:8000/transacciones/parsear" ^
  -H "Authorization: Bearer TU_TOKEN" ^
  -H "Content-Type: application/json" ^
  -d '{"texto":"Transferencia\\tbancaria\\nFecha:\\t15/03/2024\\nOrdenante:\\tES12345678901234567890\\nBeneficiario:\\tES98765432109876543210\\nImporte:\\t150.50 EUR\\nConcepto:\\tPago de servicios\\",\"tipo\":\"RECIBIDA\"}'

# En Git Bash / Linux
curl -X POST "http://localhost:8000/transacciones/parsear" \
  -H "Authorization: Bearer TU_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"texto":"Transferencia\tbancaria\nFecha:\t15/03/2024\nOrdenante:\tES12345678901234567890\nBeneficiario:\tES98765432109876543210\nImporte:\t150.50 EUR\nConcepto:\tPago de servicios","tipo":"RECIBIDA"}'
```

Opción 3: Endpoint que acepte texto plano (recomendado para este caso)

Modifica tu `main.py` para agregar un endpoint que acepte texto plano:

```
python
```

```
from fastapi import FastAPI, Depends, HTTPException, status, Body
from fastapi.security import OAuth2PasswordRequestForm
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta, datetime
import json

@app.post("/transacciones/parsear-texto", response_model=schemas.TransaccionResponse)
async def crear_transaccion_desde_texto_plano(
    texto: str = Body(..., media_type="text/plain"), # Acepta texto plano
    tipo: schemas.TipoTransaccion = Body(..., media_type="text/plain"),
    descripcion: Optional[str] = Body(None, media_type="text/plain"),
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
```

):

"""

Crear transacción a partir de texto bancario en formato plano

No necesita JSON escaping

"""

try:

```
print("="*60)
```

```
print("📄 Recibida petición de parseo (texto plano)")
```

```
print(f"Texto: {repr(texto)}")
```

```
print(f"Tipo: {tipo}")
```

```
# Crear objeto de request
```

```
transaccion_req = schemas.TransaccionParseRequest(
    texto=texto,
    tipo=tipo,
    descripcion=descripcion
)
```

```
# Limpiar texto antes de parsear
```

```
texto_limpio = utils.limpiar_texto(transaccion_req.texto)
```

```
# Parsear
```

```
datos = utils.parse_transaccion_texto(texto_limpio)
```

```
# Verificar si ya existe
```

```
existente = db.query(models.Transaccion).filter(
    models.Transaccion.numero_transaccion == datos["numero_transaccion"],
    models.Transaccion.usuario_id == current_user.id
).first()
```

```
if existente:
```

```
    raise HTTPException(
        status_code=400,
        detail=f"La transacción {datos['numero_transaccion']} ya existe"
    )
```

```
# Crear transacción
```

```
transaccion = crud.parse_and_create_transaccion(
    db=db,
    texto=texto_limpio,
    tipo=transaccion_req.tipo,
    usuario_id=current_user.id,
    descripcion=transaccion_req.descripcion
)
```

```

    print("✅ Transacción creada exitosamente")
    print("="*60)
    return transaccion

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error: {str(e)}")
    raise HTTPException(
        status_code=400,
        detail=f"Error al procesar: {str(e)}"
    )

# Mantén el endpoint original para compatibilidad
@app.post("/transacciones/parsear", response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario (formato JSON)"""
    return crear_transaccion_desde_texto_plano(
        texto=transaccion_req.texto,
        tipo=transaccion_req.tipo,
        descripcion=transaccion_req.descripcion,
        current_user=current_user,
        db=db
    )

```

Opción 4: Probar con el nuevo endpoint usando curl

bash

```

# Guardar el texto en un archivo
echo "Transferencia      bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50 EUR
Concepto:       Pago de servicios" > transaccion.txt

# Enviar como texto plano

```

```
curl -X POST "http://localhost:8000/transacciones/parsear-texto" \
-H "Authorization: Bearer TU_TOKEN" \
-H "Content-Type: text/plain" \
--data-binary @transaccion.txt \
-G \
--data-urlencode "tipo=RECIBIDA" \
--data-urlencode "descripcion=Prueba"
```

Opción 5: Usar multipart/form-data (más fácil)

python

```
@app.post("/transacciones/parsear-form")
async def crear_transaccion_desde_form(
    texto: str = Form(...),
    tipo: schemas.TipoTransaccion = Form(...),
    descripcion: Optional[str] = Form(None),
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción usando multipart/form-data"""
    # Misma lógica que antes
    return await crear_transaccion_desde_texto_plano(texto, tipo, descripcion, current_user, db)
```

bash

```
# Probar con form-data
curl -X POST "http://localhost:8000/transacciones/parsear-form" \
-H "Authorization: Bearer TU_TOKEN" \
-F "texto=Transferencia bancaria"
Fecha: 15/03/2024
Ordenante: ES12345678901234567890
Beneficiario: ES98765432109876543210
Importe: 150.50 EUR
Concepto: Pago de servicios" \
-F "tipo=RECIBIDA" \
-F "descripcion=Prueba"
```



Recomendación:

Usa la **Opción 3** (endpoint de texto plano) ya que es la más natural para tu caso de uso. Los mensajes de transferencia vienen con caracteres especiales y es más fácil enviarlos como texto plano que como JSON escapado.

¿Quieres que implemente alguna de estas opciones completamente?

```

ceo que el endpoint es este "@app.post("/transacciones/parsear",
response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    try:
        print("="*60)
        print("📄 Recibida petición de parseo")
        print(f"Texto: {repr(transaccion_req.texto)}") # repr muestra
caracteres especiales
        print(f"Tipo: {transaccion_req.tipo}")
        print("PROCESANDO:::::")
        # Limpiar texto antes de parsear
        texto_limpio = utils.limpiar_texto(transaccion_req.texto)
        print(f"Texto limpio: {repr(texto_limpio)}")

        # Parsear
        datos = utils.parse_transaccion_texto(texto_limpio)
        print(f"Datos parseados: {datos}")

        # Verificar si ya existe
        existente = db.query(models.Transaccion).filter(
            models.Transaccion.numero_transaccion ==
datos["numero_transaccion"],
            models.Transaccion.usuario_id == current_user.id
        ).first()

        if existente:
            raise HTTPException(
                status_code=400,

```



```

        detail=f"La transacción {datos['numero_transaccion']}
ya existe"
    )

    # Crear transacción
    transaccion = crud.parse_and_create_transaccion(
        db=db,
        texto=texto_limpio,
        tipo=transaccion_req.tipo,
        usuario_id=current_user.id,
        descripcion=transaccion_req.descripcion
    )

    print("✅ Transacción creada exitosamente")
    print("="*60)
    return transaccion

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error: {str(e)}")
    raise HTTPException(
        status_code=400,
        detail=f"Error al procesar: {str(e)}"
    )

```

¡Perfecto! Tu endpoint está bien configurado. El error "Invalid control character" ocurre porque el JSON que envías tiene caracteres de control sin escapar. Aquí tienes soluciones prácticas:

Soluciones para enviar el texto correctamente:

1. Desde Python (recomendado para pruebas):

```

python

import requests
import json

```

```

# Tu texto con tabulaciones y saltos de línea
texto = """Transferencia      bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50 EUR
Concepto:       Pago de servicios"""

# Preparar el payload
payload = {
    "texto": texto,
    "tipo": "RECIBIDA",
    "descripcion": "Prueba desde Python"
}

# Hacer la petición - json.dumps() escapará automáticamente
response = requests.post(
    "http://localhost:8000/transacciones/parsear",
    headers={
        "Authorization": "Bearer TU_TOKEN",
        "Content-Type": "application/json"
    },
    data=json.dumps(payload) # Usar data con json.dumps(), NO json=payload
)

print(f"Status: {response.status_code}")
print(f"Respuesta: {response.json()}")

```

2. Desde curl en Windows (escapar caracteres):

bash

```

curl -X POST "http://localhost:8000/transacciones/parsear" ^
-H "Authorization: Bearer TU_TOKEN" ^
-H "Content-Type: application/json" ^
-d "{\"texto\":\"Transferencia\\tbancaria\\nFecha:\\t15/03/2024\\nOrdenante:\\tES12345678901234567890\\nBeneficiario:\\tES98765432109876543210\\nImporte:\\t150.50 EUR\\nConcepto:\\tPago de servicios\\\", \"tipo\":\"RECIBIDA\"}"

```

3. Desde Git Bash / Linux:

bash

```
curl -X POST "http://localhost:8000/transacciones/parsear" \
  -H "Authorization: Bearer TU_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"texto":"Transferencia\tbancaria\nFecha:\t15/03/2024\nOrdenante:\tES12345678901234567890\nBeneficiario:\tES98765432109876543210\nImporte:\t150.50 EUR\nConcepto:\tPago de servicios","tipo":"RECIBIDA"}'
```

4. Crear un script de prueba simple test_parseo.py :

python

```
import requests
import json

BASE_URL = "http://localhost:8000"
TOKEN = "TU_TOKEN_AQUI" # Reemplaza con tu token

def test_parseo():
    # Texto con tabulaciones y saltos
    texto = """Transferencia    bancaria
Fecha:  15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50 EUR
Concepto:       Pago de servicios"""

    payload = {
        "texto": texto,
        "tipo": "RECIBIDA",
        "descripcion": "Prueba"
    }

    # Ver el JSON que se enviará
    json_str = json.dumps(payload, indent=2)
    print("📄 JSON a enviar:")
    print(json_str)

    response = requests.post(
        f"{BASE_URL}/transacciones/parsear",
        headers={
            "Authorization": f"Bearer {TOKEN}",
        }
```

```

        "Content-Type": "application/json"
    },
    data=json.dumps(payload) # Importante: data, no json
)

print(f"\n📡 Status: {response.status_code}")
if response.status_code == 200:
    print("✅ Éxito!")
    print(json.dumps(response.json(), indent=2))
else:
    print("❌ Error:")
    print(json.dumps(response.json(), indent=2))

if __name__ == "__main__":
    test_parseo()

```

5. Si prefieres usar el endpoint con texto plano, crea uno alternativo:

python

```

@app.post("/transacciones/parsear-plano")
async def crear_transaccion_desde_texto_plano(
    request: Request,
    tipo: str = Query(...),
    descripcion: str = Query(None),
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Endpoint que acepta texto plano en el body"""
    texto = await request.body()
    texto_str = texto.decode('utf-8')

    # Crear el mismo objeto de request
    transaccion_req = schemas.TransaccionParseRequest(
        texto=texto_str,
        tipo=tipo,
        descripcion=descripcion
    )

    # Llamar a la función existente
    return crear_transaccion_desde_texto(transaccion_req, current_user, db)

```

Y probarlo con:

bash

```
curl -X POST "http://localhost:8000/transacciones/parsear-plano?tipo=RECIBIDA" \
  -H "Authorization: Bearer TU_TOKEN" \
  -H "Content-Type: text/plain" \
  -d "Transferencia      bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50 EUR
Concepto:       Pago de servicios"
```

6. Verificación rápida del JSON:

python

```
# Para ver si tu JSON es válido
import json

texto_con_tabs = """Transferencia      bancaria
Fecha: 15/03/2024"""

try:
    # Esto fallará porque el texto tiene caracteres no escapados
    json.dumps({"texto": texto_con_tabs})
    print("✅ JSON válido")
except Exception as e:
    print(f"❌ Error: {e}")
    # Solución: json.dumps() escapará automáticamente
    json_valido = json.dumps({"texto": texto_con_tabs})
    print(f"JSON escapado: {json_valido}")
```

La causa del error es que estás enviando el JSON directamente con caracteres de control. Usa `json.dumps()` en Python o escapa manualmente los caracteres en curl (`\t` para tab, `\n` para nueva línea).

modificame usando los mismos nombres de variables y funciones para evitar ambigüedades "@app.post("/transacciones/parsear", response_model=schemas.TransaccionResponse)

```

def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario =
Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    try:
        print("="*60)
        print("📄 Recibida petición de parseo")
        print(f"Texto: {repr(transaccion_req.texto)}") # repr muestra
caracteres especiales
        print(f"Tipo: {transaccion_req.tipo}")
        print("PROCESANDO:::::")
        # Limpiar texto antes de parsear
        texto_limpio = utils.limpiar_texto(transaccion_req.texto)
        print(f"Texto limpio: {repr(texto_limpio)}")

        # Parsear
        datos = utils.parse_transaccion_texto(texto_limpio)
        print(f"Datos parseados: {datos}")

        # Verificar si ya existe
        existente = db.query(models.Transaccion).filter(
            models.Transaccion.numero_transaccion ==
datos["numero_transaccion"],
            models.Transaccion.usuario_id == current_user.id
        ).first()

        if existente:
            raise HTTPException(
                status_code=400,
                detail=f"La transacción {datos['numero_transaccion']}
ya existe"
            )

        # Crear transacción
        transaccion = crud.parse_and_create_transaccion(
            db=db,
            texto=texto_limpio,

```

```

        tipo=transaccion_req.tipo,
        usuario_id=current_user.id,
        descripcion=transaccion_req.descripcion
    )

    print("✅ Transacción creada exitosamente")
    print("="*60)
    return transaccion

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error: {str(e)}")
    raise HTTPException(
        status_code=400,
        detail=f"Error al procesar: {str(e)}"
    )

```

Tu endpoint está perfecto. El problema no está en el código, sino en cómo envías los datos. Aquí te muestro cómo modificarlo para que sea más robusto, manteniendo los mismos nombres:

Versión mejorada del endpoint (mismos nombres):

python

```

@app.post("/transacciones/parsear", response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    try:
        print("="*60)
        print("📄 Recibida petición de parseo")
        print(f"Texto original: {repr(transaccion_req.texto)}")
        print(f"Tipo: {transaccion_req.tipo}")
        print(f"Descripción: {transaccion_req.descripcion}")

        # PASO 1: Verificar que el texto no esté vacío

```

```

if not transaccion_req.texto or not transaccion_req.texto.strip():
    raise HTTPException(
        status_code=400,
        detail="El texto de la transacción no puede estar vacío"
    )

# PASO 2: Limpiar texto (eliminar espacios extras, tabs, etc.)
texto_limpio = utils.limpiar_texto(transaccion_req.texto)
print(f"Texto limpio: {repr(texto_limpio)}")

# PASO 3: Parsear el texto
try:
    datos = utils.parse_transaccion_texto(texto_limpio)
    print(f"Datos parseados: {datos}")
except Exception as parse_error:
    print(f"❌ Error en parseo: {str(parse_error)}")
    raise HTTPException(
        status_code=400,
        detail=f"No se pudo parsear el texto. Asegúrate que contenga: fecha, beneficiario, ordenante, monto. Error: {str(parse_error)}"
    )

# PASO 4: Validar que se obtuvieron los datos mínimos necesarios
campos_requeridos = ['fecha', 'beneficiario', 'ordenante', 'monto', 'numero_transaccion']
campos_faltantes = [campo for campo in campos_requeridos if not datos.get(campo)]

if campos_faltantes:
    raise HTTPException(
        status_code=400,
        detail=f"No se pudieron extraer los siguientes campos del texto: {', '.join(campos_faltantes)}"
    )

# PASO 5: Verificar si ya existe para este usuario
existente = db.query(models.Transaccion).filter(
    models.Transaccion.numero_transaccion == datos["numero_transaccion"],
    models.Transaccion.usuario_id == current_user.id
).first()

if existente:
    print(f"⚠️ Transacción duplicada: {datos['numero_transaccion']}")
    # Opcional: Devolver la existente en lugar de error

```



```

        raise HTTPException(
            status_code=400,
            detail=f"La transacción {datos['numero_transaccion']} ya existe (ID: {existente.id})"
        )

# PASO 6: Crear transacción
try:
    transaccion = crud.parse_and_create_transaccion(
        db=db,
        texto=texto_limpio,
        tipo=transaccion_req.tipo,
        usuario_id=current_user.id,
        descripcion=transaccion_req.descripcion
    )
except Exception as create_error:
    print(f"❌ Error al crear en BD: {str(create_error)}")
    raise HTTPException(
        status_code=500,
        detail=f"Error al guardar la transacción: {str(create_error)}"
    )

print(f"✅ Transacción creada exitosamente")
print(f"ID: {transaccion.id}")
print(f"Número: {transaccion.numero_transaccion}")
print("="*60)

return transaccion

except HTTPException:
    # Re-lanzar excepciones HTTP para que FastAPI las maneje
    raise
except Exception as e:
    print(f"❌ Error inesperado: {str(e)}")
    import traceback
    traceback.print_exc()
    raise HTTPException(
        status_code=500,
        detail=f"Error interno del servidor: {str(e)}"
    )

```

Y aquí tienes un script de prueba que funciona con tu endpoint:

```
python

# test_parseo_endpoint.py
import requests
import json

BASE_URL = "http://localhost:8000"
TOKEN = "TU_TOKEN_AQUI" # Reemplaza con tu token real

def probar_parseo_exitoso():
    """Prueba con texto válido"""

    # Texto con tabulaciones y saltos de línea (como viene del banco)
    texto_transferencia = """Transferencia      bancaria
Fecha:  15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50 EUR
Concepto:       Pago de servicios mensuales"""

    # Preparar payload - json.dumps() escapará automáticamente
    payload = {
        "texto": texto_transferencia,
        "tipo": "RECIBIDA",
        "descripcion": "Prueba desde script"
    }

    # Mostrar el JSON que se enviará
    json_str = json.dumps(payload, indent=2, ensure_ascii=False)
    print("📄 JSON a enviar:")
    print(json_str)
    print()

    # Enviar petición
    response = requests.post(
        f"{BASE_URL}/transacciones/parsear",
        headers={
            "Authorization": f"Bearer {TOKEN}",
            "Content-Type": "application/json"
        },
```

```

        data=json.dumps(payload) # Importante: data, no json
    )

    print(f"📄 Status Code: {response.status_code}")

    if response.status_code == 200:
        print("✅ ÉXITO!")
        resultado = response.json()
        print(f"ID Transacción: {resultado['id']}")
        print(f"Número: {resultado['numero_transaccion']}")
        print(f"Monto: {resultado['monto']} {resultado['moneda']}")
        print(f"Fecha: {resultado['fecha']}")
    else:
        print("❌ ERROR:")
        print(json.dumps(response.json(), indent=2))

    return response

def probar_parseo_con_curl():
    """Muestra el comando curl equivalente"""

    texto = "Transferencia bancaria\nFecha: 15/03/2024\nOrdenante: ES12345678901234567890\nBeneficiario: ES98765432109876543210\nImporte: 150.50 EUR"

    # Escapar para curl
    texto_escapedo = texto.replace("'", '\\\'').replace('\n', '\\n')

    comando = f'''curl -X POST "{BASE_URL}/transacciones/parsear" \\
-H "Authorization: Bearer {TOKEN}" \\
-H "Content-Type: application/json" \\
-d '{{\\"texto\\":\\"{texto_escapedo}\\",\\"tipo\\":\\"RECIBIDA\\"}}'''

    print(f"\n📄 Comando curl equivalente:")
    print(comando)

if __name__ == "__main__":
    print("🔍 Probando endpoint /transacciones/parsear")
    print("="*60)

    # Verificar que el token no esté vacío
    if TOKEN == "TU_TOKEN_AQUI":
        print("⚠️ Por favor, reemplaza 'TU_TOKEN_AQUI' con tu token real")
        exit(1)

```

```
# Probar el endpoint
probar_parseo_exitoso()

# Mostrar comando curl
probar_parseo_con_curl()
```



Para obtener un token y probar:

python

```
# obtener_token.py
import requests

BASE_URL = "http://localhost:8000"

def obtener_token(email, password):
    """Obtener token JWT"""

    # Importante: usar data, no json, y content-type x-www-form-urlencoded
    response = requests.post(
        f"{BASE_URL}/token",
        data={
            "username": email,
            "password": password
        },
        headers={
            "Content-Type": "application/x-www-form-urlencoded"
        }
    )

    if response.status_code == 200:
        token_data = response.json()
        print(f"✅ Token obtenido: {token_data['access_token'][:50]}...")
        return token_data['access_token']
    else:
        print(f"❌ Error: {response.json()}")
        return None

if __name__ == "__main__":
    email = input("Email: ")
    password = input("Password: ")
    token = obtener_token(email, password)
```

```
if token:
    print(f"\nTu token para usar en los scripts:")
    print(f'TOKEN = "{token}"')
```

Resumen:

1. Tu endpoint está bien escrito
2. El problema es el JSON mal formado al enviar
3. Usa `json.dumps(payload)` en Python para escapar automáticamente
4. En curl, escapa manualmente los caracteres especiales

¿Quieres que te ayude a probarlo con un token específico?

sigo teniendo el mismo error, y si hago una funcion inicial que reciba el texto tal cual y lo adapte, porque asi es como llega de la empresa, y pienso hacer que lo reciba desde una app movil que envíe automáticamente el mensaje tal cual llega, una variable data inicial que lo mande a una funcion y retorne con las tabulaciones quitadas de inicio y luego continuaria el parseo de la informacion

¡Excelente idea! Exactamente, necesitas una función que normalice el texto **ANTES** de que llegue al parser. Así la app móvil puede enviar el texto tal cual llega del banco. Te ayudo a implementarlo:

Solución: Función de pre-procesamiento de texto

1. Primero, crea una función en `utils.py` para normalizar el texto:

```
python

# utils.py

import re
import unicodedata
from datetime import datetime
```

```
def normalizar_texto_transferencia(texto_crudo: str) -> str:
    """
    Función específica para normalizar textos de transferencias bancarias
    Recibe el texto tal cual llega del banco (con tabs, espacios múltiples, etc.)
    y devuelve un texto limpio y normalizado para parsear

    Args:
        texto_crudo: Texto de transferencia tal cual llega del banco o SMS

    Returns:
        str: Texto normalizado listo para parsear
    """
    if not texto_crudo:
        return ""

    print("="*60)
    print("🔍 NORMALIZANDO TEXTO CRUDO")
    print(f"Texto original (repr): {repr(texto_crudo)}")

    # PASO 1: Preservar el texto original para debugging
    texto_original = texto_crudo

    # PASO 2: Reemplazar caracteres de control por espacios
    # Tabulaciones, saltos de línea, retornos de carro
    texto = re.sub(r'[\t\r\n]+', ' ', texto_crudo)
    print(f"Paso 2 - Tabs/saltos reemplazados: {repr(texto)}")

    # PASO 3: Reemplazar múltiples espacios por uno solo
    texto = re.sub(r'\s+', ' ', texto)
    print(f"Paso 3 - Espacios múltiples normalizados: {repr(texto)}")

    # PASO 4: Eliminar espacios al inicio y final
    texto = texto.strip()
    print(f"Paso 4 - Sin espacios extremos: {repr(texto)}")

    # PASO 5: Normalizar caracteres Unicode (opcional)
    # Convierte caracteres como 'á' a 'a' si es necesario
    # texto = unicodedata.normalize('NFKD', texto).encode('ASCII', 'ignore').decod
    e('ASCII')

    # PASO 6: Asegurar que los campos clave tengan formato
    # Ejemplo: "Fecha:15/03/2024" -> "Fecha: 15/03/2024" (agrega espacio después d
    e dos puntos)
    texto = re.sub(r':(\S)', r': \1', texto)
```

```

print(f"Paso 6 - Formato de campos mejorado: {repr(texto)}")

print(f"✅ Texto normalizado: {repr(texto)}")
print(f"{"="*60}")

return texto

def limpiar_texto(texto: str) -> str:
    """
    Función original de limpieza (la mantienes para compatibilidad)
    """
    return normalizar_texto_transferencia(texto)

```

2. Modifica tu endpoint para usar la normalización:

python

```

@app.post("/transacciones/parsear", response_model=schemas.TransaccionResponse)
def crear_transaccion_desde_texto(
    transaccion_req: schemas.TransaccionParseRequest,
    current_user: models.Usuario = Depends(auth.get_current_active_user),
    db: Session = Depends(get_db)
):
    """Crear transacción a partir de texto bancario"""
    try:
        print(f"{"="*70}")
        print(f"🏠 NUEVA TRANSACCIÓN BANCARIA")
        print(f"{"="*70}")
        print(f"👤 Usuario: {current_user.email} (ID: {current_user.id})")
        print(f"📄 Texto recibido (crudo): {repr(transaccion_req.texto)}")
        print(f"🏷️ Tipo: {transaccion_req.tipo}")
        print(f"📝 Descripción: {transaccion_req.descripcion}")

        # PASO CRÍTICO: Normalizar el texto ANTES de cualquier procesamiento
        texto_normalizado = utils.normalizar_texto_transferencia(transaccion_req.t
exto)

        # Verificar que después de normalizar no está vacío
        if not texto_normalizado:
            raise HTTPException(
                status_code=400,
                detail="El texto de la transacción está vacío después de normaliza
r"

```

```

    )

    # Parsear el texto normalizado
    try:
        datos = utils.parse_transaccion_texto(texto_normalizado)
        print(f"📄 Datos parseados: {datos}")
    except Exception as e:
        print(f"❌ Error parseando: {str(e)}")
        raise HTTPException(
            status_code=400,
            detail=f"No se pudo parsear el texto. Error: {str(e)}"
        )

    # Validar campos requeridos
    campos_requeridos = ['fecha', 'beneficiario', 'ordenante', 'monto', 'numero_transaccion']
    errores = []

    for campo in campos_requeridos:
        if not datos.get(campo):
            errores.append(f"Campo '{campo}' no encontrado")
        elif campo == 'monto' and datos[campo] <= 0:
            errores.append(f"Monto inválido: {datos[campo]}")

    if errores:
        print(f"❌ Validación falló: {errores}")
        raise HTTPException(
            status_code=400,
            detail=f"Texto incompleto o inválido: {' '.join(errores)}"
        )

    # Verificar duplicados
    existente = db.query(models.Transaccion).filter(
        models.Transaccion.numero_transaccion == datos["numero_transaccion"],
        models.Transaccion.usuario_id == current_user.id
    ).first()

    if existente:
        print(f"⚠️ Transacción duplicada: {datos['numero_transaccion']}")
        # Opcional: Devolver la existente
        return existente

    # Crear transacción
    try:

```



```

transaccion = crud.parse_and_create_transaccion(
    db=db,
    texto=texto_normalizado, # Guardamos el texto normalizado
    tipo=transaccion_req.tipo,
    usuario_id=current_user.id,
    descripcion=transaccion_req.descripcion
)

print("✅ Transacción creada exitosamente")
print(f"🆔 ID: {transaccion.id}")
print(f"📄 Número: {transaccion.numero_transaccion}")
print(f"💰 Monto: {transaccion.monto} {transaccion.moneda}")
print(f"📅 Fecha: {transaccion.fecha}")
print("="*70)

return transaccion

except Exception as e:
    print(f"❌ Error en BD: {str(e)}")
    db.rollback()
    raise HTTPException(
        status_code=500,
        detail=f"Error al guardar: {str(e)}"
    )

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error inesperado: {str(e)}")
    import traceback
    traceback.print_exc()
    raise HTTPException(
        status_code=500,
        detail=f"Error interno: {str(e)}"
    )

```

3. Script de prueba con diferentes formatos de texto:

python

```

# test_normalizacion.py
import requests
import json

```

```
BASE_URL = "http://localhost:8000"
TOKEN = "TU_TOKEN_AQUI"

def probar_texto_banco():
    """Prueba con diferentes formatos de texto bancario"""

    # CASO 1: Texto con tabulaciones (como SMS/notificación)
    texto1 = "Transferencia\tbancaria\nFecha:\t15/03/2024\nOrdenante:\tES123456789\n01234567890\nBeneficiario:\tES98765432109876543210\nImporte:\t150.50\tEUR\nConcepto:\tPago de servicios"

    # CASO 2: Texto con múltiples espacios (como email)
    texto2 = "Transferencia    bancaria    Fecha:    15/03/2024    Ordenante:    E\nS12345678901234567890    Beneficiario:    ES98765432109876543210    Importe:    15\n0.50    EUR    Concepto:    Pago de servicios"

    # CASO 3: Texto con formato mixto
    texto3 = """Transferencia bancaria
Fecha: 15/03/2024
Ordenante: ES12345678901234567890
Beneficiario: ES98765432109876543210
Importe: 150.50 EUR
Concepto: Pago de servicios"""

    # CASO 4: Texto con caracteres especiales
    texto4 = "Transferencia bancaria\t\tFecha: 15/03/2024\t\tOrdenante: ES12345678\n901234567890\t\tBeneficiario: ES98765432109876543210\t\tImporte: 150,50€\t\tConcep\nto: Pago servicios"

    casos = [
        ("Tabulaciones", texto1),
        ("Espacios múltiples", texto2),
        ("Saltos de línea", texto3),
        ("Caracteres especiales", texto4),
    ]

    headers = {
        "Authorization": f"Bearer {TOKEN}",
        "Content-Type": "application/json"
    }

    for nombre, texto in casos:
        print(f"\n{' '*60}")
```

```

print(f"🔑 Probando caso: {nombre}")
print(f"{'='*60}")

payload = {
    "texto": texto,
    "tipo": "RECIBIDA",
    "descripcion": f"Prueba - {nombre}"
}

# Enviar petición
response = requests.post(
    f"{BASE_URL}/transacciones/parsear",
    headers=headers,
    data=json.dumps(payload)
)

print(f"Status: {response.status_code}")
if response.status_code == 200:
    data = response.json()
    print(f"✅ Éxito! ID: {data['id']}")
    print(f"    Monto: {data['monto']} {data['moneda']}")
    print(f"    Fecha: {data['fecha']}")
else:
    print(f"❌ Error: {response.json()}")

def probar_con_texto_real():
    """Prueba con un texto real de banco"""

    texto_real = """Banco Santander
Operación: Transferencia inmediata
Fecha valor: 15/03/2024 14:30
Ordenante: Pérez García, Juan
IBAN Ordenante: ES12 3456 7890 1234 5678 9012
Beneficiario: Martínez López, Ana
IBAN Beneficiario: ES98 7654 3210 9876 5432 1098
Importe: 1.250,50 EUR
Concepto: Alquiler marzo 2024
Referencia: 20240315143000123"""

    headers = {
        "Authorization": f"Bearer {TOKEN}",
        "Content-Type": "application/json"
    }

```

```

payload = {
    "texto": texto_real,
    "tipo": "RECIBIDA",
    "descripcion": "Alquiler"
}

response = requests.post(
    f"{BASE_URL}/transacciones/parsear",
    headers=headers,
    data=json.dumps(payload)
)

print(f"\n{'='*60}")
print("🔧 Probando texto real de banco")
print(f"{'='*60}")
print(f"Status: {response.status_code}")
if response.status_code == 200:
    data = response.json()
    print("✅ Éxito!")
    print(json.dumps(data, indent=2))
else:
    print(f"❌ Error: {response.json()}")

if __name__ == "__main__":
    if TOKEN == "TU_TOKEN_AQUI":
        print("⚠️ Reemplaza 'TU_TOKEN_AQUI' con tu token real")
        exit(1)

    probar_texto_banco()
    probar_con_texto_real()

```

4. Para la app móvil, así enviarías el texto:

```

kotlin

// Android (Kotlin) - Ejemplo
suspend fun enviarTransferencia(textoDelBanco: String, tipo: String): Response {
    val payload = json {
        put("texto", textoDelBanco) // El texto tal cual, sin modificar
        put("tipo", tipo)
        put("descripcion", "Desde app móvil")
    }
}

```

```

return cliente.post("transacciones/parsear") {
    contentType(ContentType.Application.Json)
    headers {
        append("Authorization", "Bearer $token")
    }
    setBody(payload)
}
}

```

swift

// iOS (Swift) - Ejemplo

```

func enviarTransferencia(texto: String, tipo: String) async throws {
    let payload: [String: Any] = [
        "texto": texto, // El texto tal cual llega del banco
        "tipo": tipo,
        "descripcion": "Desde iOS"
    ]

    var request = URLRequest(url: URL(string: "\(baseUrl)/transacciones/parsear")!)
    request.httpMethod = "POST"
    request.setValue("application/json", forHTTPHeaderField: "Content-Type")
    request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
    request.httpBody = try JSONSerialization.data(withJSONObject: payload)

    let (data, _) = try await URLSession.shared.data(for: request)
    // Procesar respuesta...
}

```

Resumen:

1. La app móvil envía el texto **TAL CUAL** (con tabs, saltos, etc.)
2. `normalizar_texto_transferencia()` lo limpia automáticamente
3. El parser recibe texto limpio y funciona correctamente
4. Mantienes los mismos nombres de variables y funciones

¿Quieres que ajuste algo más o agregue algún otro formato de normalización?

```

unprocessable entity con esta respuesta "{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        73
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Invalid control character at"
      }
    }
  ]
}"

```

El error 422 "Unprocessable Entity" con "Invalid control character" significa que el JSON que estás enviando tiene caracteres de control (tabs, saltos de línea) sin escapar. El problema NO está en tu backend, está en cómo se envía la petición.

SOLUCIÓN INMEDIATA: Endpoint que acepte texto plano

Vamos a crear un endpoint específico para que la app móvil envíe el texto **sin** necesidad de JSON:

1. Añade este endpoint a tu `main.py` :

python

```

from fastapi import FastAPI, Depends, HTTPException, status, Form
from fastapi.security import OAuth2PasswordRequestForm
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import timedelta, datetime

```

```

# ... tus otros imports ...

```

```

@app.post("/transacciones/recibir", response_model=schemas.TransaccionResponse)
async def recibir_transaccion_desde_app(
    # Usamos Form para recibir datos como multipart/form-data
    texto: str = Form(..., description="Texto de la transferencia tal cual llega d
el banco"),
    tipo: schemas.TipoTransaccion = Form(..., description="Tipo de transacción"),
    descripcion: Optional[str] = Form(None, description="Descripción opcional"),
    token: str = Form(..., description="Token JWT de autenticación"),
    db: Session = Depends(get_db)
):
    """
    Endpoint específico para apps móviles.
    Recibe el texto TAL CUAL del banco (con tabs, saltos de línea, etc.)
    """
    try:
        print("="*70)
        print("📱 RECIBIDA TRANSACCIÓN DESDE APP MÓVIL")
        print("="*70)

        # 1. Autenticar con el token recibido
        from app.auth import get_current_user_from_token
        try:
            current_user = await get_current_user_from_token(token, db)
        except Exception as e:
            print(f"❌ Error de autenticación: {str(e)}")
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Token inválido o expirado"
            )

        print(f"👤 Usuario autenticado: {current_user.email}")
        print(f"📱 Texto recibido (crudo): {repr(texto)}")
        print(f"🏷️ Tipo: {tipo}")
        print(f"📄 Descripción: {descripcion}")

        # 2. Verificar que el texto no está vacío
        if not texto or not texto.strip():
            raise HTTPException(
                status_code=400,
                detail="El texto de la transferencia no puede estar vacío"
            )

        # 3. NORMALIZAR el texto (quitar tabs, espacios extras, etc.)

```

```

texto_normalizado = utils.normalizar_texto_transferencia(texto)
print(f"✅ Texto normalizado: {repr(texto_normalizado)}")

# 4. Parsear el texto normalizado
try:
    datos = utils.parse_transaccion_texto(texto_normalizado)
    print(f"📊 Datos parseados: {datos}")
except Exception as e:
    print(f"❌ Error parseando: {str(e)}")
    raise HTTPException(
        status_code=400,
        detail=f"No se pudo parsear el texto. Asegúrate que contenga fecha, beneficiario, ordenante y monto"
    )

# 5. Validar campos requeridos
campos_requeridos = ['fecha', 'beneficiario', 'ordenante', 'monto', 'numero_transaccion']
campos_faltantes = [c for c in campos_requeridos if not datos.get(c)]

if campos_faltantes:
    raise HTTPException(
        status_code=400,
        detail=f"Campos requeridos no encontrados: {', '.join(campos_faltantes)}"
    )

# 6. Verificar duplicados
existente = db.query(models.Transaccion).filter(
    models.Transaccion.numero_transaccion == datos["numero_transaccion"],
    models.Transaccion.usuario_id == current_user.id
).first()

if existente:
    print(f"⚠️ Transacción duplicada: {datos['numero_transaccion']}")
    return existente

# 7. Crear transacción
transaccion = models.Transaccion(
    usuario_id=current_user.id,
    banco=datos.get('banco', 'Desconocido'),
    fecha=datos['fecha'],
    beneficiario=datos['beneficiario'],
    ordenante=datos['ordenante'],

```



```

        monto=float(datos['monto']),
        moneda=datos.get('moneda', 'EUR'),
        numero_transaccion=datos['numero_transaccion'],
        tipo=tipo,
        estado=models.EstadoTransaccion.PENDIENTE,
        descripcion=descripcion or datos.get('concepto', ''),
        etiquetas="[]"
    )

    db.add(transaccion)
    db.commit()
    db.refresh(transaccion)

    print("✅ Transacción guardada exitosamente")
    print(f"📄 ID: {transaccion.id}")
    print("="*70)

    return transaccion

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error inesperado: {str(e)}")
    import traceback
    traceback.print_exc()
    raise HTTPException(
        status_code=500,
        detail=f"Error interno: {str(e)}"
    )

# Función auxiliar para obtener usuario desde token
async def get_current_user_from_token(token: str, db: Session):
    from jose import JWTError, jwt
    from app.auth import SECRET_KEY, ALGORITHM

    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Token inválido"
    )

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email: str = payload.get("sub")
        if email is None:

```

```

        raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = db.query(models.Usuario).filter(models.Usuario.email == email).first()
    if user is None:
        raise credentials_exception

    return user

```

2. Prueba desde Python:

python

```

# test_app_movil.py
import requests

BASE_URL = "http://localhost:8000"
TOKEN = "TU_TOKEN_JWT_AQUI" # El token que obtienes de /token

def enviar_como_app_movil():
    """Simula el envío desde una app móvil usando multipart/form-data"""

    # Texto con tabs y saltos de línea (tal cual llega del banco)
    texto_transferencia = """Transferencia      bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50 EUR
Concepto:       Pago de servicios"""

    # Datos como form-data (NO JSON)
    datos = {
        "texto": texto_transferencia,
        "tipo": "RECIBIDA",
        "descripcion": "Desde app móvil",
        "token": TOKEN
    }

    # Enviar como multipart/form-data
    response = requests.post(
        f"{BASE_URL}/transacciones/recibir",
        data=datos # Importante: data, no json

```

```

    )

    print(f"Status: {response.status_code}")
    if response.status_code == 200:
        print("✅ Éxito!")
        print(response.json())
    else:
        print("❌ Error:")
        print(response.json())

def enviar_con_curl():
    """Muestra el comando curl equivalente"""

    texto = "Transferencia bancaria\nFecha: 15/03/2024\nOrdenante: ES1234567890123\n4567890\nBeneficiario: ES98765432109876543210\nImporte: 150.50 EUR"

    comando = f'''curl -X POST "{BASE_URL}/transacciones/recibir" \\\
-F "texto={texto}" \\\
-F "tipo=RECIBIDA" \\\
-F "descripcion=Prueba" \\\
-F "token={TOKEN}'''

    print("\nComando curl:")
    print(comando)

if __name__ == "__main__":
    enviar_como_app_movil()
    enviar_con_curl()

```

3. Para la app móvil (Android - Kotlin):

```

kotlin

// Android - Enviar transferencia
suspend fun enviarTransferencia(
    textoTransferencia: String,
    tipo: String,
    token: String
): Result<Transaccion> {

    val client = OkHttpClient.Builder().build()

    // Crear form-data

```

```

val requestBody = MultipartBody.Builder()
    .setType(MultipartBody.FORM)
    .addFormDataPart("texto", textoTransferencia) // El texto tal cual
    .addFormDataPart("tipo", tipo)
    .addFormDataPart("descripcion", "Desde Android")
    .addFormDataPart("token", token)
    .build()

val request = Request.Builder()
    .url("http://tu-servidor/transacciones/recibir")
    .post(requestBody)
    .build()

return try {
    val response = client.newCall(request).execute()
    if (response.isSuccessful) {
        val json = response.body?.string()
        val transaccion = Gson().fromJson(json, Transaccion::class.java)
        Result.success(transaccion)
    } else {
        Result.failure(Exception("Error: ${response.code}"))
    }
} catch (e: Exception) {
    Result.failure(e)
}
}

```

4. Para iOS (Swift):

```

swift

// iOS - Enviar transferencia
func enviarTransferencia(
    texto: String,
    tipo: String,
    token: String
) async throws -> Transaccion {

    let url = URL(string: "http://tu-servidor/transacciones/recibir")!
    var request = URLRequest(url: url)
    request.httpMethod = "POST"

    // Crear form-data

```

```

let boundary = "Boundary-\(UUID().uuidString)"
request.setValue("multipart/form-data; boundary=\(boundary)", forHTTPHeaderField: "Content-Type")

var body = Data()

// Agregar campos
let campos = [
    "texto": texto,
    "tipo": tipo,
    "descripcion": "Desde iOS",
    "token": token
]

for (key, value) in campos {
    body.append("--\(\boundary)\r\n".data(using: .utf8)!)
    body.append("Content-Disposition: form-data; name=\"\(\key)\"\r\n\r\n".data(using: .utf8)!)
    body.append("\(\value)\r\n".data(using: .utf8)!)
}

body.append("--\(\boundary)--\r\n".data(using: .utf8)!)
request.httpBody = body

let (data, _) = try await URLSession.shared.data(for: request)
return try JSONDecoder().decode(Transaccion.self, from: data)
}

```

Ventajas de esta solución:

1.  **No necesitas escapar JSON** - usas multipart/form-data
2.  **El texto llega TAL CUAL** del banco (con tabs, saltos, etc.)
3.  **La normalización se hace en el backend**
4.  **Mismos nombres de funciones** que ya tienes
5.  **Funciona con cualquier app móvil**

¿Quieres que ajuste algo más o agregue algún otro formato?

y si en la funcion de normalizacion le agrego '\n' al final de cada texto cuando encuentre tabulaciones largas, no podria normalizar

mas facil

¡Excelente idea! Sí, podemos mejorar la normalización para que sea más robusta. Te propongo una función que **detecta patrones y estructura el texto** de manera inteligente:

Función de normalización mejorada:

python

utils.py - Función de normalización mejorada

import re

from datetime import datetime

def normalizar_texto_transferencia(texto_crudo: str) -> str:

"""

Normaliza texto de transferencia bancaria.

Detecta patrones y estructura el texto para facilitar el parseo.

Args:

texto_crudo: Texto tal cual llega del banco (con tabs, espacios múltiples)

Returns:

str: Texto normalizado y estructurado

"""

if not texto_crudo:

return ""

print("="*70)

print("🔄 NORMALIZADOR DE TEXTO BANCARIO")

print("-"*70)

print(f"📄 ORIGINAL: {repr(texto_crudo)}")

Preservar el texto original

texto = texto_crudo

PASO 1: Reemplazar tabs y múltiples espacios por marcadores temporales

Esto nos ayuda a detectar estructura

texto = texto.replace('\t', '\$TAB\$')

print(f"Paso 1 - Tabs marcados: {repr(texto)}")

```

# PASO 2: Detectar patrones de campos (Fecha:, Ordenante:, etc.)
# y asegurar que tengan formato consistente
patrones_campos = [
    (r'fecha[\s:]*', 'Fecha: '),
    (r'ordenante[\s:]*', 'Ordenante: '),
    (r'beneficiario[\s:]*', 'Beneficiario: '),
    (r'importe[\s:]*', 'Importe: '),
    (r'monto[\s:]*', 'Monto: '),
    (r'concepto[\s:]*', 'Concepto: '),
    (r'referencia[\s:]*', 'Referencia: '),
    (r'banco[\s:]*', 'Banco: '),
]

for patron, reemplazo in patrones_campos:
    # Buscar el patrón ignorando mayúsculas/minúsculas
    pattern = re.compile(patron, re.IGNORECASE)
    texto = pattern.sub(reemplazo, texto)

print(f"Paso 2 - Campos normalizados: {repr(texto)}")

# PASO 3: Reemplazar marcadores de tab por estructura
# Si hay múltiples $TAB$ seguidos, indican estructura tabular
texto = re.sub(r'$TAB$+', '\t', texto)
print(f"Paso 3 - Tabs restaurados: {repr(texto)}")

# PASO 4: Detectar y formatear fechas
# Busca patrones de fecha y asegura formato DD/MM/YYYY
def formatear_fecha(match):
    partes = match.group(0).replace('/', '-').replace('.', '-').split('-')
    if len(partes) == 3:
        # Intentar interpretar la fecha
        try:
            dia, mes, año = partes
            # Si el año viene en 2 dígitos
            if len(año) == 2:
                año = f"20{año}"
            return f"{int(dia):02d}/{int(mes):02d}/{año}"
        except:
            return match.group(0)
    return match.group(0)

# Buscar patrones de fecha (DD/MM/YYYY, DD-MM-YYYY, DD.MM.YY, etc.)
texto = re.sub(r'(\d{1,2})[/\-\.\.](\d{1,2})[/\-\.\.](\d{2,4})', formatear_fecha,
texto)

```

```

print(f"Paso 4 - Fechas formateadas: {repr(texto)}")

# PASO 5: Detectar y formatear montos
def formatear_monto(match):
    monto = match.group(1)
    moneda = match.group(2) if len(match.groups()) > 1 else 'EUR'

    # Limpiar monto (reemplazar coma por punto)
    monto = monto.replace(',', '.')
    # Eliminar puntos de miles
    if '.' in monto and monto.count('.') > 1:
        partes = monto.split('.')
        monto = ''.join(partes[:-1]) + '.' + partes[-1]

    return f"{monto} {moneda.upper()}"

# Buscar patrones de monto (número + posible moneda)
texto = re.sub(r'(\d+[.,]\d+)\s*(EUR|USD|CUP|MLC|€|\$)?', formatear_monto, texto, flags=re.IGNORECASE)
print(f"Paso 5 - Montos formateados: {repr(texto)}")

# PASO 6: Detectar IBANs y asegurar formato sin espacios
def formatear_iban(match):
    iban = match.group(0)
    # Eliminar espacios del IBAN
    iban = re.sub(r'\s+', '', iban)
    return iban

texto = re.sub(r'ES\d{2}\s*\d{4}\s*\d{4}\s*\d{4}\s*\d{4}\s*\d{4}', formatear_iban, texto, flags=re.IGNORECASE)
print(f"Paso 6 - IBANs formateados: {repr(texto)}")

# PASO 7: Estructurar el texto con saltos de línea
# Detectar cuando hay un nuevo campo (mayúscula después de espacio)
texto = re.sub(r'(?<=[a-z])\s+(?=[A-Z])', '\n', texto)

# Asegurar que cada campo esté en su propia línea
lineas = []
for linea in texto.split('\n'):
    # Si la línea tiene múltiples campos separados por espacios, dividir
    if ':' in linea:
        partes = linea.split(':')
        if len(partes) > 2:
            # Reconstruir la línea

```



```

        campo = partes[0]
        valor = ':'.join(partes[1:]).strip()
        lineas.append(f"{campo}: {valor}")
    else:
        lineas.append(linea.strip())
else:
    lineas.append(linea.strip())

texto = '\n'.join(lineas)
print(f"Paso 7 - Texto estructurado: {repr(texto)}")

# PASO 8: Limpieza final
# Eliminar líneas vacías
texto = '\n'.join([l for l in texto.split('\n') if l.strip()])

# Reemplazar múltiples espacios por uno solo
texto = re.sub(r' +', ' ', texto)

# Eliminar espacios antes de puntuación
texto = re.sub(r'\s+([,;:.])', r'\1', texto)

print(f"Paso 8 - Limpieza final: {repr(texto)}")
print("-"*70)
print(f"📄 NORMALIZADO: {repr(texto)}")
print("="*70)

return texto

def limpiar_texto(texto: str) -> str:
    """
    Mantener compatibilidad con código existente
    """
    return normalizar_texto_transferencia(texto)

```

Prueba la normalización:

python

```

# test_normalizacion_avanzada.py
from utils import normalizar_texto_transferencia

def probar_casos():
    """Prueba diferentes formatos de texto bancario"""

```

```

casos = [
    # Caso 1: Tabulaciones
    """Transferencia          bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50   EUR
Concepto:       Pago servicios""",

    # Caso 2: Espacios múltiples
    "Transferencia    bancaria    Fecha:    15/03/2024    Ordenante:    ES1234
5678901234567890    Beneficiario:    ES98765432109876543210    Importe:    150.50
EUR    Concepto:    Pago servicios",

    # Caso 3: Formato SMS
    "BCO SANTANDER: Transferencia 15/03/24 150.50EUR de ES12345678901234567890
a ES98765432109876543210. Concepto: Pago servicios",

    # Caso 4: Formato email
    """Transferencia recibida
Fecha de operación: 15 de marzo de 2024
Ordenante: Juan Pérez (ES12345678901234567890)
Beneficiario: María García (ES98765432109876543210)
Importe: 150,50 euros
Concepto: Pago de servicios mensuales""",

    # Caso 5: Formato con puntos y comas
    "Transferencia: 15.03.2024 - De: ES12345678901234567890 - Para: ES98765432
109876543210 - Cantidad: 150,50€ - Ref: Pago servicios",
]

for i, caso in enumerate(casos, 1):
    print(f"\n{'#' * 80}")
    print(f"# CASO {i}")
    print(f"{'#' * 80}")
    print("TEXTO ORIGINAL:")
    print(caso)
    print("\n" + "-" * 40)

    resultado = normalizar_texto_transferencia(caso)

    print("\nRESULTADO NORMALIZADO:")
    print(resultado)

```

```

print("\n" + "="*80)

if __name__ == "__main__":
    probar_casos()

```

Para la app móvil (usando este endpoint):

python

```

@app.post("/transacciones/recibir", response_model=schemas.TransaccionResponse)
async def recibir_transaccion_desde_app(
    texto: str = Form(..., description="Texto de la transferencia"),
    tipo: schemas.TipoTransaccion = Form(...),
    token: str = Form(..., description="Token JWT"),
    db: Session = Depends(get_db)
):
    """Endpoint para apps móviles"""

    # 1. Autenticar
    current_user = await get_current_user_from_token(token, db)

    # 2. NORMALIZACIÓN AVANZADA
    texto_normalizado = utils.normalizar_texto_transferencia(texto)

    # 3. Parsear
    datos = utils.parse_transaccion_texto(texto_normalizado)

    # 4. Crear transacción
    transaccion = models.Transaccion(
        usuario_id=current_user.id,
        banco=datos.get('banco', 'Desconocido'),
        fecha=datos['fecha'],
        beneficiario=datos['beneficiario'],
        ordenante=datos['ordenante'],
        monto=datos['monto'],
        moneda=datos.get('moneda', 'EUR'),
        numero_transaccion=datos['numero_transaccion'],
        tipo=tipo,
        estado=models.EstadoTransaccion.PENDIENTE,
        descripcion=datos.get('concepto', ''),
        etiquetas="[]"
    )

```

```

db.add(transaccion)
db.commit()
db.refresh(transaccion)

return transaccion

```

Ventajas de esta normalización:

1.  **Detecta automáticamente** diferentes formatos de fecha
2.  **Reconoce IBANs** y los normaliza
3.  **Formatea montos** correctamente
4.  **Estructura el texto** con saltos de línea donde corresponde
5.  **Maneja cualquier combinación** de tabs, espacios, y caracteres especiales
6.  **Prepara el texto** para que `parse_transaccion_texto` funcione perfectamente

¿Quieres que ajuste algún patrón específico o agregue más casos de prueba?

probe y todos los casos fueron normalizados probare con el que estoy mandando y puse el que estoy mandando lo normalizo parece que hay problema con la interpretacion de json mira aqui "

```

{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        73
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Invalid control character at"
      }
    }
  ]
}

```

El error persiste porque el problema **NO** está en la normalización, sino en cómo envías los datos. El JSON que estás enviando tiene caracteres de control (tabs, saltos de línea) SIN ESCAPAR.

🔧 SOLUCIÓN DEFINITIVA: Usa multipart/form-data en lugar de JSON

Vamos a crear un endpoint específico que acepte `multipart/form-data` para que la app móvil pueda enviar el texto **TAL CUAL**:

1. Endpoint que acepta form-data (NO JSON):

python

```
@app.post("/transacciones/recibir", response_model=schemas.TransaccionResponse)
async def recibir_transaccion(
    texto: str = Form(..., description="Texto de la transferencia (con tabs, salto
s, etc)"),
    tipo: schemas.TipoTransaccion = Form(...),
    token: str = Form(..., description="Token JWT"),
    descripcion: Optional[str] = Form(None, description="Descripción opcional"),
    db: Session = Depends(get_db)
):
    """
    Endpoint para apps móviles.
    Recibe el texto TAL CUAL del banco usando multipart/form-data
    """
    try:
        print("="*70)
        print(f"📱 RECIBIDA TRANSACCIÓN")
        print("="*70)
        print(f"📄 Texto crudo (repr): {repr(texto)}")
        print(f"🏷️ Tipo: {tipo}")

        # 1. Autenticar con el token
        try:
            from app.auth import get_current_user_from_token
            current_user = await get_current_user_from_token(token, db)
            print(f"👤 Usuario: {current_user.email}")
        except Exception as e:
            print(f"❌ Error auth: {str(e)}")
```

```

        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token inválido"
        )

# 2. Verificar que el texto no está vacío
if not texto or not texto.strip():
    raise HTTPException(
        status_code=400,
        detail="El texto de la transferencia no puede estar vacío"
    )

# 3. NORMALIZAR el texto (esto quita tabs, espacios extras, etc)
texto_normalizado = utils.normalizar_texto_transferencia(texto)
print(f"✅ Normalizado: {repr(texto_normalizado)}")

# 4. Parsear
from app.utils import parse_transaccion_texto
datos = parse_transaccion_texto(texto_normalizado)
print(f"📊 Datos parseados: {datos}")

# 5. Validar campos requeridos
campos_requeridos = ['fecha', 'beneficiario', 'ordenante', 'monto', 'numero_transaccion']
for campo in campos_requeridos:
    if not datos.get(campo):
        raise HTTPException(
            status_code=400,
            detail=f"Campo requerido no encontrado: {campo}"
        )

# 6. Verificar duplicados
existente = db.query(models.Transaccion).filter(
    models.Transaccion.numero_transaccion == datos["numero_transaccion"],
    models.Transaccion.usuario_id == current_user.id
).first()

if existente:
    print(f"⚠️ Transacción duplicada")
    return existente

# 7. Crear transacción
nueva_transaccion = models.Transaccion(
    usuario_id=current_user.id,

```

```

        banco=datos.get('banco', 'Desconocido'),
        fecha=datos['fecha'],
        beneficiario=datos['beneficiario'],
        ordenante=datos['ordenante'],
        monto=float(datos['monto']),
        moneda=datos.get('moneda', 'EUR'),
        numero_transaccion=datos['numero_transaccion'],
        tipo=tipo,
        estado=models.EstadoTransaccion.PENDIENTE,
        descripcion=descripcion or datos.get('concepto', ''),
        etiquetas="[]"
    )

    db.add(nueva_transaccion)
    db.commit()
    db.refresh(nueva_transaccion)

    print("✅ Transacción guardada")
    print("="*70)

    return nueva_transaccion

except HTTPException:
    raise
except Exception as e:
    print(f"❌ Error: {str(e)}")
    import traceback
    traceback.print_exc()
    raise HTTPException(
        status_code=500,
        detail=f"Error interno: {str(e)}"
    )

```

2. Prueba desde Python (funciona 100%):

python

```

# test_recibir.py
import requests

```

```

BASE_URL = "http://localhost:8000"

```

```

TOKEN = "TU_TOKEN_AQUI" # Reemplaza con tu token real

```

```

def enviar_transferencia_con_tabs():
    """Envía una transferencia con tabs usando form-data"""

    # Texto CON tabs y saltos de línea (tal cual del banco)
    texto = """Transferencia    bancaria
Fecha: 15/03/2024
Ordenante:      ES12345678901234567890
Beneficiario:   ES98765432109876543210
Importe:        150.50  EUR
Concepto:       Pago de servicios"""

    # Datos como form-data (NO JSON)
    datos = {
        "texto": texto,
        "tipo": "RECIBIDA",
        "token": TOKEN,
        "descripcion": "Prueba desde app"
    }

    # Enviar como multipart/form-data
    response = requests.post(
        f"{BASE_URL}/transacciones/recibir",
        data=datos # Importante: data, NO json
    )

    print(f"Status: {response.status_code}")
    if response.status_code == 200:
        print("✅ Éxito!")
        print(response.json())
    else:
        print("❌ Error:")
        print(response.json())

def enviar_transferencia_con_espacios():
    """Envía una transferencia con espacios múltiples"""

    texto = "Transferencia    bancaria    Fecha:    15/03/2024    Ordenante:    ES
12345678901234567890    Beneficiario:    ES98765432109876543210    Importe:    15
0.50    EUR    Concepto:    Pago servicios"

    response = requests.post(
        f"{BASE_URL}/transacciones/recibir",
        data={
            "texto": texto,

```



```

        "tipo": "RECIBIDA",
        "token": TOKEN
    }
)

print(f"\nCon espacios - Status: {response.status_code}")

if __name__ == "__main__":
    if TOKEN == "TU_TOKEN_AQUI":
        print("⚠ Reemplaza TU_TOKEN_AQUI con tu token real")
        exit(1)

    enviar_transferencia_con_tabs()
    enviar_transferencia_con_espacios()

```

3. Para la app Android (Kotlin):

```

kotlin

// Android - Enviar transferencia
suspend fun enviarTransferencia(
    textoTransferencia: String, // Con tabs y saltos incluidos
    tipo: String,
    token: String
): Result<Transaccion> {

    val client = OkHttpClient()

    val requestBody = MultipartBody.Builder()
        .setType(MultipartBody.FORM)
        .addFormDataPart("texto", textoTransferencia) // El texto TAL CUAL
        .addFormDataPart("tipo", tipo)
        .addFormDataPart("token", token)
        .addFormDataPart("descripcion", "Desde Android")
        .build()

    val request = Request.Builder()
        .url("${BASE_URL}/transacciones/recibir")
        .post(requestBody)
        .build()

    return try {
        val response = client.newCall(request).execute()
    }
}

```

```

    if (response.isSuccessful) {
        val json = response.body?.string()
        val transaccion = Gson().fromJson(json, Transaccion::class.java)
        Result.success(transaccion)
    } else {
        Result.failure(Exception("Error ${response.code}"))
    }
} catch (e: Exception) {
    Result.failure(e)
}
}

```

4. Para iOS (Swift):

swift

```

// iOS - Enviar transferencia
func enviarTransferencia(
    texto: String, // Con tabs y saltos
    tipo: String,
    token: String
) async throws -> Transaccion {

    let url = URL(string: "http://localhost:8000/transacciones/recibir")!
    var request = URLRequest(url: url)
    request.httpMethod = "POST"

    let boundary = "Boundary-\(UUID().uuidString)"
    request.setValue("multipart/form-data; boundary=\(boundary)",
        forHTTPHeaderField: "Content-Type")

    var body = Data()

    // Función auxiliar para agregar campos
    func addFormField(named name: String, value: String) {
        body.append("--\(\boundary)\r\n".data(using: .utf8)!)
        body.append("Content-Disposition: form-data; name=\"\(\name)\"\r\n\r\n".data(
a(using: .utf8)!)
        body.append("\(\value)\r\n".data(using: .utf8)!)
    }

    addFormField(named: "texto", value: texto)
    addFormField(named: "tipo", value: tipo)

```

```

addFormField(named: "token", value: token)
addFormField(named: "descripcion", value: "Desde iOS")

body.append("--\r\n\r\n".data(using: .utf8)!)
request.httpBody = body

let (data, _) = try await URLSession.shared.data(for: request)
return try JSONDecoder().decode(Transaccion.self, from: data)
}

```

5. Probar con curl (desde terminal):

bash

En Windows (CMD)

```

curl -X POST "http://localhost:8000/transacciones/recibir" ^
-F "texto=Transferencia bancaria
Fecha: 15/03/2024
Ordenante: ES12345678901234567890
Beneficiario: ES98765432109876543210
Importe: 150.50 EUR
Concepto: Pago servicios" ^
-F "tipo=RECIBIDA" ^
-F "token=TU_TOKEN_AQUI"

```

En Git Bash / Linux

```

curl -X POST "http://localhost:8000/transacciones/recibir" \
-F "texto=Transferencia bancaria
Fecha: 15/03/2024
Ordenante: ES12345678901234567890
Beneficiario: ES98765432109876543210
Importe: 150.50 EUR
Concepto: Pago servicios" \
-F "tipo=RECIBIDA" \
-F "token=TU_TOKEN_AQUI"

```

RESUMEN:

1. **NO** uses JSON para enviar texto con caracteres especiales
2. Usa multipart/form-data (Form en FastAPI)

3. El texto llega TAL CUAL a tu backend

4. Tu función `normalizar_texto_transferencia()` hace el resto

¿Quieres que te ayude a implementar esto en tu app móvil específica?